claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd.jsonl`
- SHA-256: `6765a493cfa640e8d23de4364e876fb02fd1933a900b081f9e9a368b9e51d657`
- Source modified: `2026-06-22T09:15:10+00:00`
- Imported at: `2026-07-05T16:48:18+00:00`
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T00:53:29.160Z)

In the badminton-highlight-indexer repo, `tests/test_ablation.py::test_litmus_reproduces_gate_a` and `::test_litmus_report_is_well_formed` FAIL locally with `KeyError: 'shuttle'` when the full n=11 golden corpus is present in `output/`.

Root cause (confirmed): `_golden_present()` (tests/test_ablation.py ~line 366) globs ALL `output/*_golden_features.json` (runs the litmus when ?6 exist), and the litmus path in `backend/eval/ablation.py` accesses `candidate['shuttle']` (also `['person']`, `['audio']`). But the 5 newer real-footage clips added during the R3/R4 corpus growth ? `mahadevpura_1`, `mahadevpura_2`, `GX010128`, `GX030094`, `Badminton_BXH_2` ? have a LEANER candidate schema produced by the velocity-windowing/distill path: keys `['active_frame_count','chunk_index','end','mean_velocity','peak_velocity','start','track_id_count']`, with NO `shuttle`/`person`/`audio` feature dicts. The original n=6 files (e.g. `adarsh_avi_singles`) have the full fusion schema `['audio','end','label','person','shuttle','start']`. The litmus crashes on the leaner files.

This is local-only: CI's `output/` is empty so these tests SKIP (green there). But on the owner's machine `python run_all_tests.py` shows 2 reds.

Pick the right fix (your call): (a) make the litmus loader FILTER to golden_features whose candidates carry the full fusion feature schema (skip windowing-only files) ? most robust; or (b) regenerate the 5 real clips' golden_features through `fusion_golden` so they carry person/audio/shuttle columns (heavier; needs the feature extraction); or (c) guard the litmus to a pinned subset. Prefer (a). Add a regression test for the mixed-schema corpus. Keep it offline/mocked. Verify: full suite + ruff + mypy clean. Branch from origin/master in an isolated worktree, re-verify HEAD before commit, stage explicit paths (shared checkout). Do NOT touch the nightly_regression.py work (separate PR).

2. assistant (2026-06-18T00:53:37.914Z)

I'll start by getting oriented: pull latest, read the relevant code, and check the current-state memory.

3. user (2026-06-18T00:53:43.952Z)

```
From https://github.com/avidullu/badminton-highlight-indexer
 * [new branch]      claude/indexer-doc-review-XVPTh -> origin/claude/indexer-doc-review-XVPTh
   8736e2d..be39a70  master     -> origin/master
---STATUS---
```

docs/golden-regression-fixtures-plan...origin/docs/golden-regression-fixtures-plan [ahead 18, behind 1]

```
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
---HEAD---
f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF) (#203)
```

4. user (2026-06-18T00:53:43.999Z)

File content (272.6KB) exceeds maximum allowed size (256KB). Use offset and limit parameters to read specific portions of the file, or search for specific content instead of reading the whole file.

5. user (2026-06-18T00:53:44.028Z)

```
1       """GPU-free tests for the LOSO/LOGO ablation runner (``backend.eval.ablation``).
2
3       Covers, all on deterministic synthetic candidate rows (no GPU, no real video):
4
5         - LOSO / LOGO correctness ? a cleanly-separating signal *earns its keep*, a noise
signal is
6           *inert*; LOGO collapses a group's columns; the ablated variant subtracts columns /
zeroes
7           the gate knob without mutating ``full``.
8         - the structural-guard exemption ? the SAME signal marked ``structural=True`` is
reported
9           (honest ?) but verdict is always ``structural_protected``, never auto-demoted on "no
lift".
10        - the honest-stats contract ? every row ships a CI + sign test, and no verdict is
stronger
11          than the stats support (the no-over-claim contract).
12        - a valid non-trivial PDF is produced (asserted via pypdf page count > 0).
13        - the golden litmus reproduction ? on the 6 real ``output/*_golden_features.json`` the
runner
14          reproduces the hand-measured Gate-A result (?F1 ? +0.074, 6/6 wins, p ? 0.031).
Skips
15          cleanly when the golden JSONs are absent (CI), runs when present (owner box).
16      """
17      from __future__ import annotations
18
19      import glob
20      import json
21      import os
22
23      import pytest
24
25      from backend.eval import ablation
26      from backend.eval.ablation import (
27          AblationConfig,
28          AblationReport,
29          Signal,
30          ablate_one,
31          run_ablation,
32      )
33      from backend.eval.experiment import Variant, VideoCandidates
34
35
36      # ---------------------------------------------------------------------------- synthetic
fixtures
37
38
39      def _gate_videos(n: int = 6, *, separating: bool = True) -> list[VideoCandidates]:
40          """``n`` videos with a ``net_crossings`` gate signal.
41
42          ``separating=True``: rallies have net_crossings=5, junk has 0 ? so a
``min_crossings=2`` gate
43          perfectly removes the junk and keeping it is load-bearing. ``separating=False``:
every window
44          has net_crossings=5, so the gate is a no-op (ablating it changes nothing ?
inert)."""
45          videos: list[VideoCandidates] = []
46          for k in range(n):
47              intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
48              junk_val = 5.0 if not separating else 0.0
```

```
49              feats = {"net_crossings": [5.0, 5.0, junk_val, junk_val]}
50              gts = [(0.0, 10.0), (20.0, 30.0)]
51              videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
52          return videos
53
54
55      def _learned_videos(n: int = 6) -> list[VideoCandidates]:
56          """``n`` videos where a PERSON feature (``players_in_court``) cleanly separates
rallies from
57          junk while the shuttle/audio columns are constant (non-separating). So a learned
full-set
58          ablation must show person as the load-bearing group and the others as washes ? a
deterministic
59          LOGO/LOSO correctness fixture (mirrors ``test_fusion_compare``'s construction)."""
60          videos: list[VideoCandidates] = []
61          for k in range(n):
62              intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
63              feats: dict[str, list[float]] = {}
64              for fn in ablation.SHUTTLE_COLUMNS:
65                  feats[fn] = [1.0, 1.0, 1.0, 1.0]                    # constant ? non-
separating
66              for fn in ablation.PERSON_COLUMNS:
67                  feats[fn] = [0.0, 0.0, 0.0, 0.0]
68              feats["players_in_court"] = [4.0, 4.0, 0.0, 0.0]         # the separating signal
69              feats["in_court_ratio"] = [1.0, 1.0, 0.0, 0.0]
70              for fn in ablation.AUDIO_COLUMNS:
71                  feats[fn] = [0.0, 0.0, 0.0, 0.0]
72              gts = [(0.0, 10.0), (20.0, 30.0)]
73              videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
74          return videos
75
76
77      # ------------------------------------------------------------------------ LOSO
correctness
78
79
80      def test_separating_gate_earns_keep():
81          """A non-structural gate that cleanly separates rallies must EARN ITS KEEP: removing
it
82          drops F1 with the ?F1 CI clearing 0 and the sign test agreeing (6W/0L)."""
83          videos = _gate_videos(6, separating=True)
84          sig = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
85          cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[sig],
86                               granularity="signal")
87          row = ablate_one(videos, cfg, sig)
88          assert row.delta_f1 > 0.0
89          assert row.ci_excludes_zero is True
90          assert row.sign_wins == 6 and row.sign_losses == 0
91          assert row.sign_p_value <= 0.05
92          assert row.verdict == ablation.EARNS_KEEP
93          assert row.structural_protected is False
94          # C4: the gate cuts over-segmentation (more junk segments without it).
95          assert row.seg_ratio_full < row.seg_ratio_ablated
96
97
98      def test_noop_gate_is_inert():
99          """A gate whose threshold removes nothing (all windows pass) is a wash ? INERT,
never a
100         stronger claim. Honest: a single wash at n=6 is a candidate, not 'proven
useless'."""
101         videos = _gate_videos(6, separating=False)
102         sig = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
103         cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[sig],
```

```
                            granularity="signal")
        row = ablate_one(videos, cfg, sig)
        assert abs(row.delta_f1) < 1e-9
        assert row.ci_excludes_zero is False
        assert row.verdict == ablation.INERT


    def test_ablated_variant_does_not_mutate_full():
        """Building the ablated variant must derive a new Variant, never mutate ``full`` in
place."""
        full = Variant(name="full", feature_names=["a", "b", "c"], min_crossings=2)
        before = full.to_dict()
        ablated = ablation._ablated_variant(full, ("b",), "min_crossings")
        assert ablated.feature_names == ["a", "c"]
        assert ablated.min_crossings == 0
        assert full.to_dict() == before                 # full untouched
        assert full.feature_names == ["a", "b", "c"]


    def test_unknown_gate_knob_raises():
        full = Variant(name="full", feature_names=["a"])
        with pytest.raises(ValueError):
            ablation._ablated_variant(full, (), "min_bogus")


    # ---------------------------------------------------------------------- LOGO
correctness


    def test_logo_collapses_groups():
        """LOGO (``granularity='group'``) ablates a whole producer's columns at once: the
default
        5-signal LOSO inventory collapses to 4 producer groups, and the union of columns is
dropped."""
        groups = ablation._collapse_to_groups(ablation.default_signals())
        names = [g.name for g in groups]
        assert names == ["shuttle", "gate_a", "person", "audio"]
        shuttle = next(g for g in groups if g.name == "shuttle")
        # the shuttle group unions duration + the velocity block
        assert set(shuttle.columns) == {"duration", "peak_velocity", "mean_velocity",
"active_ratio"}
        gate = next(g for g in groups if g.name == "gate_a")
        assert gate.gate_knob == "min_crossings"
        assert gate.structural is True                   # structural flag OR-ed up


    def test_logo_finds_separating_group():
        """On the learned fixture where PERSON cleanly separates, the LOGO person row must
be the
        load-bearing one (largest positive marginal ?F1) and the constant groups must be
~washes."""
        videos = _learned_videos(6)
        cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
                             granularity="group")
        report = run_ablation(videos, cfg)
        rows = {r.signal: r for r in report.rows}
        assert set(rows) == {"shuttle", "gate_a", "person", "audio"}
        top = report.sorted_rows()[0]
        assert top.signal == "person"
        assert rows["person"].delta_f1 > 0.0
        # the constant audio group carries no information ? marginal ? 0
        assert abs(rows["audio"].delta_f1) < 0.05


```

```
    def test_run_ablation_both_granularities():
        """LOSO yields one row per signal; LOGO yields one per group; both honest-LOVO on a
learned
        full set."""
        videos = _learned_videos(6)
        loso = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
                                                   signals=ablation.default_signals(),
                                                   granularity="signal"))
        logo = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
                                                   signals=ablation.default_signals(),
                                                   granularity="group"))
        assert len(loso.rows) == len(ablation.default_signals())     # 5 LOSO rows
        assert len(logo.rows) == 4                                   # 4 producer groups
        assert loso.honest_lovo is True and logo.honest_lovo is True


    def test_run_ablation_rejects_bad_granularity():
        videos = _learned_videos(2)
        cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
                             granularity="bogus")
        with pytest.raises(ValueError):
            run_ablation(videos, cfg)


    def test_run_ablation_needs_a_video():
        cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals())
        from backend.eval.experiment import ExperimentError
        with pytest.raises(ExperimentError):
            run_ablation([], cfg)


    # ------------------------------------------------------------------ structural-guard
exemption


    def test_structural_guard_never_demoted():
        """A structural guard with the SAME measured ? as a non-structural one that EARNS
its keep
        must report verdict ``structural_protected`` (never auto-demoted), but record the
honest ?."""
        videos = _gate_videos(6, separating=True)
        non = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
        struct = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=True)
        full = Variant(name="full", min_crossings=2)
        cfg = AblationConfig(full=full, signals=[non], granularity="signal")
        r_non = ablate_one(videos, cfg, non)
        r_struct = ablate_one(videos, cfg, struct)
        # identical honest measurement?
        assert pytest.approx(r_non.delta_f1) == r_struct.delta_f1
        assert (r_non.sign_wins, r_non.sign_losses) == (r_struct.sign_wins,
r_struct.sign_losses)
        # ?but the verdict differs: non-structural earns keep, structural is protected.
        assert r_non.verdict == ablation.EARNS_KEEP
        assert r_struct.verdict == ablation.STRUCTURAL_PROTECTED
        assert r_struct.structural_protected is True


    def test_structural_guard_protected_even_when_inert():
        """A structural guard that is a measured wash is STILL reported as
``structural_protected``
        (never flagged dead weight on 'no measured lift')."""
        videos = _gate_videos(6, separating=False)
        struct = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=True)
```

```
        cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[struct],
                             granularity="signal")
        row = ablate_one(videos, cfg, struct)
        assert abs(row.delta_f1) < 1e-9                  # honestly a wash
        assert row.verdict == ablation.STRUCTURAL_PROTECTED
        assert row.structural_protected is True


    def test_default_gate_a_is_structural():
        """The shipped Gate-A signal must carry ``structural=True`` (the held-shuttle FP
guard)."""
        sig = next(s for s in ablation.default_signals() if s.group == "gate_a")
        assert sig.structural is True
        assert sig.gate_knob == "min_crossings"


    # ---------------------------------------------------------------------- honest-
stats fields


    def test_row_carries_full_honest_evidence():
        """Every row ships the honest evidence: n, CI bounds, the sign test, and a verdict
in the
        closed vocabulary (no over-claim)."""
        videos = _learned_videos(6)
        cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
                             granularity="group")
        report = run_ablation(videos, cfg)
        valid = {ablation.EARNS_KEEP, ablation.SUGGESTIVE, ablation.INERT,
                 ablation.STRUCTURAL_PROTECTED}
        for r in report.rows:
            d = r.as_dict()
            assert {"delta_f1", "ci_low", "ci_high", "ci_excludes_zero", "sign_test", "n",
                    "verdict"} <= set(d)
            assert d["n"] == 6
            assert d["sign_test"]["wins"] + d["sign_test"]["losses"] <= 6
            # CI brackets the mean (±fp) ? never a bare mean detached from its interval.
            assert r.ci_low - 1e-9 <= r.delta_f1 <= r.ci_high + 1e-9
            assert r.verdict in valid


    def test_classify_verdicts_cover_all_branches():
        """``_classify`` maps evidence ? the closed verdict vocabulary, honestly:
        structural always protected; CI-clears-0 + sign-agrees ? earns_keep; leans-positive
but
        CI-spans-0 ? suggestive; a wash ? inert."""
        # structural always protected, regardless of evidence
        assert ablation._classify(True, True, 6, 0, 0.01, 0.05) ==
(ablation.STRUCTURAL_PROTECTED, True)
        assert ablation._classify(True, False, 0, 6, 1.0, 0.05) ==
(ablation.STRUCTURAL_PROTECTED, True)
        # earns keep: CI clears 0 AND sign agrees
        assert ablation._classify(False, True, 6, 0, 0.03, 0.05) == (ablation.EARNS_KEEP,
False)
        # suggestive: leans positive (more wins) but CI spans 0 / not significant
        assert ablation._classify(False, False, 4, 2, 0.30, 0.05) == (ablation.SUGGESTIVE,
False)
        # inert: not more wins than losses and CI spans 0
        assert ablation._classify(False, False, 2, 4, 0.69, 0.05) == (ablation.INERT, False)
        assert ablation._classify(False, False, 2, 2, 1.0, 0.05) == (ablation.INERT, False)


    def test_no_over_claim_vocabulary():
        """The verdict vocabulary must NOT contain 'irrelevant'/'useless' (attribution
```

```
contract)."""
274         forbidden = {"irrelevant", "useless", "dead", "remove"}
275         for v in ablation.VERDICT_LABEL.values():
276             assert not any(bad in v.lower() for bad in forbidden)
277 
278 
279     def test_report_round_trips_json():
280         """The structured report is JSON-able and sorted by ?F1 desc (contribution
ranking)."""
281         videos = _learned_videos(6)
282         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
283                                                      signals=ablation.default_signals(),
284                                                      granularity="group"))
285         blob = json.dumps(report.as_dict())              # must not raise
286         back = json.loads(blob)
287         deltas = [r["delta_f1"] for r in back["rows"]]
288         assert deltas == sorted(deltas, reverse=True)    # sorted desc
289         assert back["num_videos"] == 6
290         assert back["label_set_hash"]                    # provenance present
291 
292 
293     def test_seed_is_deterministic():
294         """Two runs with the same seed produce identical CIs (deterministic bootstrap)."""
295         videos = _learned_videos(6)
296         cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
297                              granularity="group", seed=7)
298         r1 = run_ablation(videos, cfg)
299         r2 = run_ablation(videos, cfg)
300         assert [r.ci_low for r in r1.rows] == [r.ci_low for r in r2.rows]
301         assert [r.ci_high for r in r1.rows] == [r.ci_high for r in r2.rows]
302 
303 
304     # ---------------------------------------------------------------------------- table +
PDF
305 
306 
307     def test_format_table_is_sorted_and_readable():
308         videos = _learned_videos(6)
309         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
310                                                      signals=ablation.default_signals(),
311                                                      granularity="group"))
312         out = ablation.format_table(report)
313         assert "Ablation report" in out
314         assert "?F1(full?ablated)" in out
315         assert "verdict" in out
316         # the highest-?F1 signal name appears before the lowest in the rendered order
317         top = report.sorted_rows()[0].signal
318         bottom = report.sorted_rows()[-1].signal
319         assert out.index(top) < out.index(bottom)
320 
321 
322     def test_export_pdf_is_valid_nontrivial(tmp_path):
323         """A valid PDF with at least one page is produced, and it contains the verdict
legend."""
324         pytest.importorskip("reportlab")
325         pypdf = pytest.importorskip("pypdf")
326         videos = _learned_videos(6)
327         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
328                                                      signals=ablation.default_signals(),
329                                                      granularity="group"))
330         out = str(tmp_path / "ablation.pdf")
331         ablation.export_pdf(report, out)
332         assert os.path.isfile(out)
333         assert os.path.getsize(out) > 1000               # non-trivial
```

```
        reader = pypdf.PdfReader(out)
        assert len(reader.pages) > 0
        text = "".join(p.extract_text() for p in reader.pages)
        assert "Ablation" in text
        assert "clears-bar" in text                     # the verdict key rendered


    # ------------------------------------------------------------------------ corpus loader
(no GPU)


    def test_load_videos_tolerates_missing_audio(tmp_path):
        """An older feature JSON without an ``audio`` block still loads ? audio columns
default to
        0.0 (loader never crashes on a pre-audio JSON)."""
        cand = {"start": 0.0, "end": 10.0,
                "shuttle": {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS},
                "person": {fn: 0.0 for fn in ablation.PERSON_COLUMNS},
                "label": 1}
        data = {"name": "noaudio", "fps": 30.0, "frame_width": 1920.0,
                "candidates": [cand], "gts": [[0.0, 10.0]]}
        with open(tmp_path / "noaudio_golden_features.json", "w", encoding="utf-8") as f:
            json.dump(data, f)
        videos = ablation.load_videos(str(tmp_path))
        assert len(videos) == 1
        assert videos[0].features["audio_hit_count"] == [0.0]        # defaulted, not
crashed


    # ------------------------------------------------------------------------- the
golden litmus

    GOLDEN_DIR = os.path.join(
        os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "output")


    def _golden_present() -> bool:
        return len(glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json"))) >= 6


    @pytest.mark.skipif(not _golden_present(),
                        reason="golden output/*_golden_features.json absent (run
fusion_golden first)")
    def test_litmus_reproduces_gate_a():
        """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-
measured Gate-A
        result ? marginal ?F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
tolerance.
        This validates the whole framework against the recorded ground truth. Skips cleanly
in CI
        where the golden substrate is absent; runs on the owner box where it is present."""
        videos = ablation.load_videos(GOLDEN_DIR)
        assert len(videos) == 6
        cfg = ablation.litmus_config()
        report = run_ablation(videos, cfg)
        assert len(report.rows) == 1
        row = report.rows[0]
        assert row.signal == "gate_a"
        # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
        assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
        assert row.sign_wins == 6 and row.sign_losses == 0
        assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
        assert row.ci_low == pytest.approx(0.042, abs=0.01)
        assert row.ci_high == pytest.approx(0.104, abs=0.01)
        assert row.ci_excludes_zero is True
```

```
391         # Gate-A is structural ? reported but never auto-promoted to earns_keep.
392         assert row.verdict == ablation.STRUCTURAL_PROTECTED
393         # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
394         assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
395         assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)
396
397
398     @pytest.mark.skipif(not _golden_present(),
399                         reason="golden output/*_golden_features.json absent")
400     def test_litmus_report_is_well_formed():
401         """The litmus report is a proper AblationReport (provenance + a sorted single
row)."""
402         report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
403         assert isinstance(report, AblationReport)
404         assert report.num_videos == 6
405         assert report.full_set_signals == ["gate_a"]
406         assert report.label_set_hash
407         out = ablation.format_table(report)
408         assert "gate_a" in out
409
410
411     # ------------------------------------------------------------------------------ edge
cases
412
413
414     def test_min_players_gate_knob_zeroed():
415         """The ``min_players`` gate knob is zeroed on ablation (the Gate-B analog)."""
416         full = Variant(name="full", min_players=2, feature_names=["a"])
417         ablated = ablation._ablated_variant(full, (), "min_players")
418         assert ablated.min_players == 0
419         assert ablated.feature_names == ["a"]            # columns untouched
420
421
422     def test_ablated_label_falls_back_to_gate_knob():
423         """With no columns, the ablated variant name carries the gate-knob label, not
'?'."""
424         full = Variant(name="full", min_crossings=2)
425         ablated = ablation._ablated_variant(full, (), "min_crossings")
426         assert "min_crossings" in ablated.name
427
428
429     # ------------------------------------------------------------------------------------
CLI
430
431
432     def _write_learned_corpus(d: str, n: int = 6) -> None:
433         for k in range(n):
434             cands = []
435             for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0),
((50.0, 51.0), 0)]:
436                 shuttle = {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS}
437                 person = {fn: 0.0 for fn in ablation.PERSON_COLUMNS}
438                 person["players_in_court"] = 4.0 if pos else 0.0
439                 person["in_court_ratio"] = 1.0 if pos else 0.0
440                 audio = {fn: 0.0 for fn in ablation.AUDIO_COLUMNS}
441                 cands.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
442                               "audio": audio, "label": pos})
443             data = {"name": f"v{k}", "fps": 30.0, "frame_width": 1920.0,
444                     "candidates": cands, "gts": [[0.0, 10.0], [20.0, 30.0]]}
445             with open(os.path.join(d, f"v{k}_golden_features.json"), "w", encoding="utf-8")
as f:
446                 json.dump(data, f)
447
448
449     def test_cli_main_writes_pdf_and_json(tmp_path, monkeypatch, capsys):
```

```
450         """``main()`` runs the LOGO ablation and writes both a PDF and the structured
JSON."""
451         pytest.importorskip("reportlab")
452         pytest.importorskip("pypdf")
453         d = str(tmp_path / "feat")
454         os.makedirs(d)
455         _write_learned_corpus(d)
456         pdf = str(tmp_path / "out.pdf")
457         js = str(tmp_path / "out.json")
458         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--granularity",
"group",
459                                          "--pdf", pdf, "--json", js])
460         ablation.main()
461         out = capsys.readouterr().out
462         assert "Ablation report" in out
463         assert os.path.isfile(pdf) and os.path.getsize(pdf) > 1000
464         assert os.path.isfile(js)
465         report = json.loads(open(js, encoding="utf-8").read())
466         assert {r["group"] for r in report["rows"]} == {"shuttle", "gate_a", "person",
"audio"}
467
468
469     def test_cli_main_litmus_flag(tmp_path, monkeypatch, capsys):
470         """``--litmus`` runs the static Gate-A config (single structural row)."""
471         d = str(tmp_path / "feat")
472         os.makedirs(d)
473         _write_learned_corpus(d)
474         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--litmus"])
475         ablation.main()
476         out = capsys.readouterr().out
477         assert "gate_a" in out
478         assert "static" in out                          # litmus full set is the static
gate variant
479
480
481     def test_cli_main_needs_two_videos(tmp_path, monkeypatch):
482         d = str(tmp_path / "feat")
483         os.makedirs(d)
484         _write_learned_corpus(d, n=1)
485         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d])
486         with pytest.raises(SystemExit):
487             ablation.main()
488
```

6. assistant (2026-06-18T00:53:47.030Z)

Now let me read the ablation module and the current-state memory (with offset since it's large).

7. user (2026-06-18T00:53:48.372Z)

```
1       """Leave-one-signal-out / leave-one-group-out ablation runner (Phase 0).
2
3       A *systematic, repeatable* version of the one-off ablation
(``scratch/gate_ab_ablation.py``):
4       run it and it EXCLUDES a single signal, measures whether quality dropped, and reports ?
per
5       signal ? the **marginal contribution** ?F1 (full set vs full-minus-that-signal) with a
bootstrap
6       95% CI + paired sign test + the C4 over-segmentation ratio. See
``scratch/ABLATION_FRAMEWORK_PLAN.md``
7       (the design) and ``docs/ABLATION_LAYER.md`` (the shipped note).
8
9       **Nothing here is new scoring/stats math.** It is a *pure driver* over the shipped
10      ``experiment.compare`` + ``eval_stats`` engine, exactly as ``compare`` was a driver over
11      ``metrics``/``calibration``. For each signal ``s`` the marginal contribution is:
```

```
12
13          ?(s) = score(full set S) ? score(S \\ {s})            # leave-one-out drop, ?F1
primary
14
15      operationally ``experiment.compare(videos, variant_a=without_s, variant_b=full)`` ? A is
the
16      ablated (smaller) variant, B is full, so the reported B?A delta IS ?(s), a per-video-
paired ?F1
17      with bootstrap CI + exact sign test, LOVO-honest for learned variants.
18
19      Two ablation granularities fall out of one inventory:
20
21        - **Leave-one-signal-out (LOSO)** ? remove ONE column at a time (fine-grained).
22        - **Leave-one-GROUP-out (LOGO)** ? remove a whole producer's columns at once (the
owner's
23          "is this old SIGNAL still relevant?" question at cue granularity).
24
25      A signal can be a **feature column** (person/audio/shuttle columns the learned scorer
weights) or
26      a **gate knob** (Gate-A's ``min_crossings`` threshold ? ablated by zeroing the knob, NOT
by
27      dropping a column). The runner handles both uniformly: ``without_s`` subtracts the
columns AND/OR
28      zeroes the gate knob from ``cfg.full``.
29
30      Honest small-N contract (inherited from ``eval_stats`` / ``per_user_gate``; §6 of the
plan):
31        - never a bare mean ? every row ships a CI + the exact sign test;
32        - a signal **earns its keep** only when removal drops F1 with **CI excluding 0 AND the
sign test
33          agreeing** (the ``per_user_gate`` bar, in reverse);
34        - **structural signals are exempt from auto-demotion** ? a guard whose value is in
failure modes
35          the golden corpus may lack (Gate-A vs held-shuttle FPs) is marked ``structural`` and
is NEVER
36          flagged dead weight on "no measured lift"; its row reports the (possibly ?0) golden
? honestly
37          but tags ``structural_protected``;
38        - the runner emits ``earns_keep`` / ``suggestive`` / ``inert`` /
``structural_protected`` ? it
39          MUST NOT print "irrelevant" or "proven useless" (one wash at n=6 is noise, not
proof).
40
41      Pure + offline + GPU-free. Reuses the persisted golden feature substrate
(``fusion_golden.py``
42      writes one ``<name>_golden_features.json`` per video). CLI::
43
44          python -m backend.eval.ablation --features-dir output [--pdf out.pdf]
45      """
46      from __future__ import annotations
47
48      import argparse
49      import glob
50      import json
51      import logging
52      import os
53      from dataclasses import dataclass, field
54      from typing import Any, Dict, List, Optional, Sequence, Tuple
55
56      from backend.eval import experiment
57      from backend.eval.experiment import Variant, VideoCandidates
58
59      logger = logging.getLogger(__name__)
60
61      # --- The signal inventory (the things the runner ablates), grouped by producer.
```

```
---------
62      # Source: distill_local.FEATURE_NAMES (shuttle, includes net_crossings), fusion_features
63      # (person), audio_features (audio). The shuttle group's velocity block + duration are
the
64      # learned-column shuttle signals; net_crossings is BOTH a column and the Gate-A gate
knob.
65      SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity", "mean_velocity",
66                                          "active_ratio", "net_crossings")
67      PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court", "two_sided_motion",
"mean_player_motion",
68                                         "in_court_ratio", "shuttle_player_colocation")
69      AUDIO_COLUMNS: Tuple[str, ...] = ("audio_hit_count", "audio_hit_rate",
"audio_hit_strength_mean")
70
71      # Verdict vocabulary (the no-over-claim contract ? never "irrelevant"/"proven useless").
72      EARNS_KEEP = "earns_keep"               # CI clears 0 AND sign test agrees ? removing it
hurts
73      SUGGESTIVE = "suggestive"               # point estimate leans positive but CI spans 0
74      INERT = "inert"                         # wash this run (?0, CI spans 0) ? candidate,
not proof
75      STRUCTURAL_PROTECTED = "structural_protected"   # a guard; never auto-demoted on "no
lift"
76
77      #: Human-readable one-liners for each verdict (used in the PDF + table legend).
78      VERDICT_LABEL = {
79          EARNS_KEEP: "clears-bar",
80          SUGGESTIVE: "suggestive",
81          INERT: "inert",
82          STRUCTURAL_PROTECTED: "structural-protected",
83      }
84
85
86      @dataclass(frozen=True)
87      class Signal:
88          """One unit of ablation ? a feature column (or a named group) and/or a gate knob.
89
90          - ``name`` ? table/row identity ("person.colocation" | "gate_a" | "audio").
91          - ``group`` ? producer ("person" | "gate_a" | "audio" | "shuttle"). LOGO ablates a
whole group.
92          - ``columns`` ? the persisted feature columns this signal contributes (may be empty
for a
93            pure gate signal like Gate-A whose column is shared with the shuttle group).
94          - ``gate_knob`` ? e.g. ``"min_crossings"`` for the Gate-A threshold; ablated by
zeroing it on
95            the full variant rather than dropping a column.
96          - ``structural`` ? a guard whose value is in failure modes the golden corpus may
lack; NEVER
97            flagged dead weight on "no measured lift" (mirrors ``per_user_gate``
``structural=True``).
98          """
99
100         name: str
101         group: str
102         columns: Tuple[str, ...] = ()
103         gate_knob: Optional[str] = None
104         structural: bool = False
105
106
107     @dataclass
108     class AblationConfig:
109         """How to run an ablation over a corpus.
110
111         - ``full`` ? the full-set reference variant (B). For the litmus Gate-A reproduction
this is
112           the STATIC ``Variant(min_crossings=2)`` (no classifier); for a learned ablation it
```

```
carries
113           ``feature_names`` = all columns and the gates ON.
114         - ``signals`` ? the inventory to ablate.
115         - ``granularity`` ? ``"signal"`` (LOSO, one column-set at a time) or ``"group"``
(LOGO).
116         - ``iou`` ? temporal-IoU match threshold.
117         - ``honest`` ? LOVO for learned variants (the number to trust).
118         - ``seed`` ? threaded into the ``eval_stats`` bootstrap (determinism).
119         - ``p_threshold`` ? sign-test significance bar for the ``earns_keep`` verdict.
120         """
121
122         full: Variant
123         signals: List[Signal]
124         granularity: str = "group"
125         iou: float = 0.5
126         honest: bool = True
127         seed: int = 0
128         p_threshold: float = 0.05
129
130
131     @dataclass
132     class AblationRow:
133         """One signal's marginal-contribution row (?(s) + CI + sign test + C4 + a
verdict)."""
134
135         signal: str
136         group: str
137         structural: bool
138         dropped_columns: List[str]
139         gate_knob: Optional[str]
140         n: int
141         delta_f1: float
142         ci_low: float
143         ci_high: float
144         ci_excludes_zero: bool
145         sign_wins: int
146         sign_losses: int
147         sign_p_value: float
148         f1_full: float
149         f1_ablated: float
150         seg_ratio_full: float
151         seg_ratio_ablated: float
152         verdict: str
153         structural_protected: bool
154
155         def as_dict(self) -> Dict[str, Any]:
156             return {
157                 "signal": self.signal,
158                 "group": self.group,
159                 "structural": self.structural,
160                 "dropped_columns": list(self.dropped_columns),
161                 "gate_knob": self.gate_knob,
162                 "n": self.n,
163                 "delta_f1": self.delta_f1,
164                 "ci_low": self.ci_low,
165                 "ci_high": self.ci_high,
166                 "ci_excludes_zero": self.ci_excludes_zero,
167                 "sign_test": {"wins": self.sign_wins, "losses": self.sign_losses,
168                               "p_value": self.sign_p_value},
169                 "f1_full": self.f1_full,
170                 "f1_ablated": self.f1_ablated,
171                 "segment_count_ratio_full": self.seg_ratio_full,
172                 "segment_count_ratio_ablated": self.seg_ratio_ablated,
173                 "verdict": self.verdict,
174                 "structural_protected": self.structural_protected,
```

```
            }


    @dataclass
    class AblationReport:
        """All signals' rows + the run's provenance ? a structured, JSON-able ablation
result."""

        granularity: str
        iou: float
        num_videos: int
        honest_lovo: bool
        full_set_signals: List[str]
        full_spec_hash: str
        label_set_hash: str
        rows: List[AblationRow] = field(default_factory=list)

        def sorted_rows(self) -> List[AblationRow]:
            """Rows by marginal ?F1 desc ? the contribution ranking (the plan's table
order)."""
            return sorted(self.rows, key=lambda r: r.delta_f1, reverse=True)

        def as_dict(self) -> Dict[str, Any]:
            return {
                "granularity": self.granularity,
                "iou": self.iou,
                "num_videos": self.num_videos,
                "honest_lovo": self.honest_lovo,
                "full_set_signals": list(self.full_set_signals),
                "full_spec_hash": self.full_spec_hash,
                "label_set_hash": self.label_set_hash,
                "rows": [r.as_dict() for r in self.sorted_rows()],
            }


    # ------------------------------------------------------------------------- the driver


    def _ablated_variant(full: Variant, columns: Sequence[str], gate_knob: Optional[str]) ->
Variant:
        """Build the leave-one-out (smaller) variant from ``full``: drop ``columns`` from
its
        ``feature_names`` and/or zero the named gate knob. Pure ? derives a new Variant,
never
        mutates ``full``."""
        cols = set(columns)
        fn = full.feature_names
        ablated_fn = [c for c in fn if c not in cols] if fn is not None else fn
        overrides: Dict[str, Any] = {"feature_names": ablated_fn}
        if gate_knob == "min_crossings":
            overrides["min_crossings"] = 0
        elif gate_knob == "min_players":
            overrides["min_players"] = 0
        elif gate_knob is not None:
            raise ValueError(f"unknown gate_knob {gate_knob!r} (expected
'min_crossings'/'min_players')")
        dropped_label = ",".join(sorted(cols)) or (gate_knob or "?")
        return experiment.Variant.from_dict({**full.to_dict(), **overrides,
                                             "name": f"?{dropped_label}"})


    def _classify(structural: bool, ci_excludes_zero: bool, sign_wins: int, sign_losses:
int,
                  sign_p: float, p_threshold: float) -> Tuple[str, bool]:
        """The honest verdict (no over-claim). Returns ``(verdict, structural_protected)``.
```

```
233
234         A structural guard is reported but never demoted on "no lift". Otherwise: removing
the
235         signal *earns its keep* when the ?F1 CI clears 0 AND the sign test agrees (majority
worse on
236         removal, ``p <= p_threshold``); a positive-leaning but CI-spanning row is
``suggestive``; a
237         wash this run is ``inert`` (a candidate, never "proven useless" at n=6)."""
238         if structural:
239             return STRUCTURAL_PROTECTED, True
240         sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
241         if ci_excludes_zero and sign_agrees:
242             return EARNS_KEEP, False
243         if sign_wins > sign_losses:           # leans toward "removal hurts" but evidence is
thin
244             return SUGGESTIVE, False
245         return INERT, False
246
247
248     def ablate_one(videos: Sequence[VideoCandidates], cfg: AblationConfig,
249                    signal: Signal) -> AblationRow:
250         """One ablation row: marginal ?(s) + CI + sign test + C4 + a verdict.
251
252         Drops ``signal``'s columns / zeroes its gate knob from ``cfg.full`` to form the
ablated
253         variant A, then calls ``experiment.compare(A, full)`` so the reported B?A ?F1 IS
?(s).
254         Reuses ``compare`` verbatim ? learned variants stay LOVO-honest, the C1+C4 honest
stats
255         come for free."""
256         ablated = _ablated_variant(cfg.full, signal.columns, signal.gate_knob)
257         res = experiment.compare(videos, ablated, cfg.full, iou=cfg.iou, honest=cfg.honest,
258                                  honest_stats=True)
259         # Re-pair the per-video ?F1 with the configured bootstrap seed for determinism (the
default
260         # honest block uses seed=0; thread cfg.seed explicitly so a run is reproducible).
261         from backend.eval.eval_stats import paired_delta_ci
262         by_a = {p["video"]: p["f1"] for p in res["variant_a"]["per_video"]}
263         by_b = {p["video"]: p["f1"] for p in res["variant_b"]["per_video"]}
264         vids = [p["video"] for p in res["variant_a"]["per_video"]]
265         pd = paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids],
seed=cfg.seed)
266         st = pd["sign_test"]
267         sign_wins, sign_losses = int(st["wins"]), int(st["losses"])
268         sign_p = float(st["p_value"])
269         verdict, protected = _classify(signal.structural, bool(pd["ci_excludes_zero"]),
270                                        sign_wins, sign_losses, sign_p, cfg.p_threshold)
271
272         seg_full = res["honest"]["segmentation"]["variant_b"]["segment_count_ratio"]
273         seg_abl = res["honest"]["segmentation"]["variant_a"]["segment_count_ratio"]
274
275         return AblationRow(
276             signal=signal.name,
277             group=signal.group,
278             structural=signal.structural,
279             dropped_columns=list(signal.columns),
280             gate_knob=signal.gate_knob,
281             n=int(pd["n"]),
282             delta_f1=float(pd["mean_delta"]),
283             ci_low=float(pd["ci_low"]),
284             ci_high=float(pd["ci_high"]),
285             ci_excludes_zero=bool(pd["ci_excludes_zero"]),
286             sign_wins=sign_wins,
287             sign_losses=sign_losses,
288             sign_p_value=sign_p,
```

```
            f1_full=float(res["variant_b"]["aggregate"]["f1"]),
            f1_ablated=float(res["variant_a"]["aggregate"]["f1"]),
            seg_ratio_full=float(seg_full),
            seg_ratio_ablated=float(seg_abl),
            verdict=verdict,
            structural_protected=protected,
        )


    def _collapse_to_groups(signals: Sequence[Signal]) -> List[Signal]:
        """Collapse a LOSO inventory into one Signal per producer GROUP (LOGO): union the
columns,
        OR the gate knob, OR the structural flag. First-seen group order is preserved."""
        order: List[str] = []
        by_group: Dict[str, List[Signal]] = {}
        for s in signals:
            if s.group not in by_group:
                order.append(s.group)
                by_group[s.group] = []
            by_group[s.group].append(s)
        out: List[Signal] = []
        for g in order:
            members = by_group[g]
            cols: Tuple[str, ...] = tuple(dict.fromkeys(c for m in members for c in
m.columns))
            knob = next((m.gate_knob for m in members if m.gate_knob), None)
            structural = any(m.structural for m in members)
            out.append(Signal(name=g, group=g, columns=cols, gate_knob=knob,
structural=structural))
        return out


    def run_ablation(videos: Sequence[VideoCandidates], cfg: AblationConfig) ->
AblationReport:
        """Loop the inventory at the configured granularity ? an ``AblationReport`` (sorted
table).

        A pure driver: each row is one ``experiment.compare(without_s, full)``.
``granularity="group"``
        (LOGO) ablates a whole producer's columns at once; ``"signal"`` (LOSO) one column-
set at a
        time. The full reference is scored once implicitly per row (cheap re-scoring;
detection is
        already persisted)."""
        if len(videos) < 1:
            raise experiment.ExperimentError("run_ablation needs at least one labeled
video")
        if cfg.granularity not in ("signal", "group"):
            raise ValueError(f"granularity must be 'signal' or 'group', got
{cfg.granularity!r}")

        inventory = cfg.signals if cfg.granularity == "signal" else
_collapse_to_groups(cfg.signals)
        rows = [ablate_one(videos, cfg, s) for s in inventory]

        # Provenance: a full-set evaluation gives the honest-LOVO flag + the label-set
fingerprint.
        full_eval = experiment.evaluate_variant(videos, cfg.full, iou=cfg.iou,
honest=cfg.honest)
        from backend.eval.regression import gt_hash
        all_gts = [iv for vc in videos for iv in (vc.gts or [])]
        return AblationReport(
            granularity=cfg.granularity,
            iou=cfg.iou,
            num_videos=len(videos),
```

```
341             honest_lovo=bool(full_eval["honest_lovo"]),
342             full_set_signals=[s.group for s in _collapse_to_groups(cfg.signals)],
343             full_spec_hash=cfg.full.spec_hash(),
344             label_set_hash=gt_hash(all_gts),
345             rows=rows,
346         )
347
348
349     # ------------------------------------------------------------------ default
inventories
350
351
352     def default_signals() -> List[Signal]:
353         """The current 4-group inventory (LOSO granularity): shuttle (velocity block +
duration),
354         gate_a (the structural net-crossings gate AND its column), person, audio.
355
356         Gate-A is marked ``structural`` (the held-shuttle FP guard) so it is reported but
never
357         flagged dead weight on "no measured lift". It ablates via the ``min_crossings`` knob
AND by
358         dropping the ``net_crossings`` column, so the leave-one-out genuinely removes the
whole cue."""
359         return [
360             Signal("shuttle.duration", "shuttle", ("duration",)),
361             Signal("shuttle.velocity", "shuttle", ("peak_velocity", "mean_velocity",
"active_ratio")),
362             Signal("gate_a", "gate_a", ("net_crossings",), gate_knob="min_crossings",
structural=True),
363             Signal("person", "person", PERSON_COLUMNS),
364             Signal("audio", "audio", AUDIO_COLUMNS),
365         ]
366
367
368     def static_gate_a_signal() -> Signal:
369         """The Gate-A-only structural signal (a pure gate knob over the recall-oriented
proposal
370         set). Ablating it against the static ``Variant(min_crossings=2)`` full reproduces
the
371         hand-measured Gate-A result (?F1 ? +0.074, 6/6, p ? 0.031) ? the framework's litmus
test."""
372         return Signal("gate_a", "gate_a", (), gate_knob="min_crossings", structural=True)
373
374
375     def litmus_config() -> AblationConfig:
376         """The Gate-A reproduction config: full = STATIC ``Variant(min_crossings=2)`` (no
learned
377         classifier), inventory = the single Gate-A gate signal. Ablating it = CONTROL+gate
vs CONTROL
378         ? exactly the historical measurement (``scratch/gate_ab_ablation.py``)."""
379         full = experiment.Variant(name="control_gate_a", min_crossings=2)
380         return AblationConfig(full=full, signals=[static_gate_a_signal()],
granularity="signal")
381
382
383     # ------------------------------------------------------------------------ corpus
I/O
384
385
386     def load_videos(features_dir: str) -> List[VideoCandidates]:
387         """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals +
shuttle +
388         person + audio columns + golden gts). Audio columns are optional (older JSONs
predate the
389         audio augment) ? a missing block defaults those columns to 0.0 so the loader never
```

```
crashes,
        mirroring ``experiment._feature_column``'s tolerance."""
        videos: List[VideoCandidates] = []
        for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
            with open(path, encoding="utf-8") as f:
                d = json.load(f)
            cands = d["candidates"]
            intervals = [(float(c["start"]), float(c["end"])) for c in cands]
            features: Dict[str, List[float]] = {}
            for fn in SHUTTLE_COLUMNS:
                features[fn] = [float(c["shuttle"][fn]) for c in cands]
            for fn in PERSON_COLUMNS:
                features[fn] = [float(c["person"][fn]) for c in cands]
            for fn in AUDIO_COLUMNS:
                features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
            gts = [(float(a), float(b)) for a, b in d["gts"]]
            videos.append(VideoCandidates(name=d["name"], intervals=intervals,
                                          features=features, gts=gts))
        return videos


    def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
        """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)
weighted
        by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the
learned path
        (distinct from the static Gate-A litmus)."""
        cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
        return experiment.Variant(name="full_learned", feature_names=cols,
                                  threshold=threshold, min_crossings=min_crossings)


    # ---------------------------------------------------------------------- table
rendering


    def format_table(report: AblationReport) -> str:
        """Sorted ASCII/markdown ablation table (by marginal ?F1 desc ? the contribution
ranking)."""
        rows = report.sorted_rows()
        header = (f"Ablation report ? full set = {{{', '.join(report.full_set_signals)}}}"
                  f"  ·  n={report.num_videos}  ·  IoU {report.iou}"
                  f"  ·  {'LOVO-honest' if report.honest_lovo else 'static'}"
                  f"  ·  {report.granularity.upper()}")
        cols = ["signal", "?F1(full?ablated)", "95% CI", "sign test", "CI?0", "verdict"]
        widths = [18, 17, 18, 14, 5, 22]

        def _fmt_row(cells: Sequence[str]) -> str:
            return "  ".join(c.ljust(w) for c, w in zip(cells, widths))

        lines = [header, "", _fmt_row(cols), _fmt_row(["-" * w for w in widths])]
        for r in rows:
            ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
            sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
            verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
            lines.append(_fmt_row([r.signal, f"{r.delta_f1:+.3f}", ci, sign,
                                   "yes" if r.ci_excludes_zero else "no", verdict]))
        return "\n".join(lines)


    # ---------------------------------------------------------------------- PDF
export


    def export_pdf(report: AblationReport, path: str) -> str:
```

```
449         """Render the ``AblationReport`` to a clean reportlab PDF: a table of each signal's
marginal
450         contribution + CI + sign test + a verdict column (clears-bar / suggestive / inert /
451         structural-protected). Reuses the muted booktabs aesthetic of
``scratch/quality_report.py``.
452
453         Returns the written path. ``reportlab`` is an offline, pure-Python dependency (no
GPU)."""
454         from reportlab.lib import colors
455         from reportlab.lib.enums import TA_CENTER, TA_LEFT
456         from reportlab.lib.pagesizes import A4
457         from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
458         from reportlab.lib.units import cm
459         from reportlab.platypus import (Paragraph, SimpleDocTemplate, Spacer, Table,
TableStyle)
460
461         navy = colors.HexColor("#1F3A5F")
462         black = colors.HexColor("#000000")
463         darkgrey = colors.HexColor("#333333")
464         light = colors.HexColor("#F0F0F0")
465         hairline = colors.HexColor("#BFBFBF")
466         row_alt = colors.HexColor("#F7F7F7")
467
468         base = getSampleStyleSheet()["Normal"]
469         title = ParagraphStyle("Title2", parent=base, fontName="Helvetica-Bold",
fontSize=16,
470                                leading=20, alignment=TA_CENTER, textColor=navy,
spaceAfter=4)
471         subtitle = ParagraphStyle("Sub", parent=base, fontName="Helvetica-Oblique",
fontSize=9.5,
472                                   leading=13, alignment=TA_CENTER, textColor=darkgrey,
spaceAfter=10)
473         h2 = ParagraphStyle("H2", parent=base, fontName="Helvetica-Bold", fontSize=11,
leading=15,
474                             textColor=navy, spaceBefore=10, spaceAfter=4)
475         body = ParagraphStyle("Body", parent=base, fontName="Helvetica", fontSize=9,
leading=12.5,
476                               alignment=TA_LEFT, textColor=black, spaceAfter=5)
477         cell = ParagraphStyle("Cell", parent=base, fontName="Helvetica", fontSize=8.2,
leading=10.5,
478                               textColor=black)
479         cell_b = ParagraphStyle("CellB", parent=base, fontName="Helvetica-Bold",
fontSize=8.2,
480                                 leading=10.5, textColor=black)
481         head = ParagraphStyle("Head", parent=base, fontName="Helvetica-Bold", fontSize=8.2,
482                               leading=10.5, textColor=black)
483
484         rows = report.sorted_rows()
485         story: List[Any] = [
486             Paragraph("Signal Ablation &amp;amp; Feature-Enablement Report", title),
487             Paragraph(
488                 f"Leave-one-{'group' if report.granularity == 'group' else 'signal'}-out
marginal "
489                 f"contribution &mdash; n={report.num_videos} golden matches, IoU
{report.iou}, "
490                 f"{'honest LOVO' if report.honest_lovo else 'static re-score'}. "
491                 "Honest measured results only.",
492                 subtitle),
493         ]
494
495         full_set = ", ".join(report.full_set_signals)
496         story.append(Paragraph("Run provenance", h2))
497         story.append(Paragraph(
498             f"Full signal set: <b>{{{full_set}}}</b>. Variant spec_hash "
499             f"<font face='Courier'>{report.full_spec_hash}</font>, label-set hash "
```

```
            f"<font face='Courier'>{report.label_set_hash}</font>. Each row is the held-out
"
            "&Delta;F1 of removing one signal from the full set (full &minus; ablated), with
a "
            "bootstrap 95% CI and an exact paired sign test &mdash; a signal earns its keep
only "
            "when removing it drops F1 with the CI clearing 0 <i>and</i> the sign test
agreeing.",
            body))

        story.append(Paragraph("Marginal contribution per signal", h2))
        table_data: List[List[Any]] = [[
            Paragraph("Signal", head), Paragraph("Group", head),
            Paragraph("&Delta;F1 (full&minus;abl.)", head), Paragraph("95% CI", head),
            Paragraph("Sign test", head), Paragraph("CI&ne;0", head),
            Paragraph("Over-seg", head), Paragraph("Verdict", head),
        ]]
        for r in rows:
            ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
            sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
            overseg = f"{r.seg_ratio_full:.2f}&rarr;{r.seg_ratio_ablated:.2f}"
            verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
            df1_style = cell_b if r.verdict == EARNS_KEEP else cell
            table_data.append([
                Paragraph(r.signal, cell), Paragraph(r.group, cell),
                Paragraph(f"{r.delta_f1:+.3f}", df1_style), Paragraph(ci, cell),
                Paragraph(sign, cell), Paragraph("yes" if r.ci_excludes_zero else "no",
cell),
                Paragraph(overseg, cell), Paragraph(verdict, cell),
            ])
        col_widths = [2.7 * cm, 1.7 * cm, 2.4 * cm, 2.7 * cm, 2.6 * cm, 1.1 * cm, 1.9 * cm,
2.6 * cm]
        tbl = Table(table_data, colWidths=col_widths, repeatRows=1)
        last = len(table_data) - 1
        tbl.setStyle(TableStyle([
            ("BACKGROUND", (0, 0), (-1, 0), light),
            ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
            ("TOPPADDING", (0, 0), (-1, -1), 4),
            ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
            ("LEFTPADDING", (0, 0), (-1, -1), 4),
            ("RIGHTPADDING", (0, 0), (-1, -1), 4),
            ("LINEABOVE", (0, 0), (-1, 0), 1.0, black),
            ("LINEBELOW", (0, 0), (-1, 0), 0.8, black),
            ("LINEBELOW", (0, 1), (-1, max(1, last - 1)), 0.3, hairline),
            ("LINEBELOW", (0, last), (-1, last), 1.0, black),
            ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, row_alt]),
        ]))
        story.append(tbl)
        story.append(Spacer(1, 0.3 * cm))

        story.append(Paragraph("Verdict key", h2))
        story.append(Paragraph(
            "<b>clears-bar</b> &mdash; removing the signal drops F1 with the &Delta;F1 CI
clearing 0 "
            "and the sign test agreeing (the signal earns its keep). "
            "<b>suggestive</b> &mdash; the point estimate leans toward 'removal hurts' but
the CI "
            "spans 0 (keep, watch as the corpus grows). "
            "<b>inert</b> &mdash; removal is a wash this run; a candidate for retirement
only after a "
            "trend across multiple runs, never on one wash at small n. "
            "<b>structural-protected</b> &mdash; a guard whose value is in failure modes the
golden "
            "corpus may lack (e.g. Gate-A vs held-shuttle false positives); reported
honestly but "
```

```
            "never auto-flagged dead weight on 'no measured lift'.",
            body))

        doc = SimpleDocTemplate(path, pagesize=A4, leftMargin=2 * cm, rightMargin=2 * cm,
                                topMargin=1.8 * cm, bottomMargin=1.8 * cm,
                                title="Signal Ablation & Feature-Enablement Report",
                                author="badminton-highlight-indexer")
        doc.build(story)
        logger.info("wrote ablation PDF -> %s", path)
        return path


    # ---------------------------------------------------------------------------
CLI


    def main() -> None:
        ap = argparse.ArgumentParser(
            description="LOSO/LOGO ablation runner over the golden feature substrate (pure
driver "
                        "over experiment.compare + eval_stats; no new scoring math).")
        ap.add_argument("--features-dir", default="output",
                        help="dir of *_golden_features.json (the persisted golden
substrate)")
        ap.add_argument("--granularity", choices=("signal", "group"), default="group",
                        help="leave-one-SIGNAL-out (column) or leave-one-GROUP-out
(producer)")
        ap.add_argument("--iou", type=float, default=0.5)
        ap.add_argument("--seed", type=int, default=0, help="bootstrap seed (determinism)")
        ap.add_argument("--litmus", action="store_true",
                        help="reproduce the static Gate-A measurement (CONTROL vs
min_crossings=2) "
                             "instead of the learned full-set ablation")
        ap.add_argument("--pdf", default=None, help="also write a PDF ablation report to
this path")
        ap.add_argument("--json", default=None, help="also write the structured report JSON
here")
        args = ap.parse_args()

        videos = load_videos(args.features_dir)
        if len(videos) < 2:
            raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} "
                             f"(found {len(videos)})")

        if args.litmus:
            cfg = litmus_config()
            cfg.iou, cfg.seed = args.iou, args.seed
        else:
            cfg = AblationConfig(full=learned_full_variant(), signals=default_signals(),
                                 granularity=args.granularity, iou=args.iou, seed=args.seed)

        report = run_ablation(videos, cfg)
        print(format_table(report))

        if args.json:
            with open(args.json, "w", encoding="utf-8") as f:
                json.dump(report.as_dict(), f, indent=2)
            print(f"\nwrote report JSON -> {args.json}")
        if args.pdf:
            export_pdf(report, args.pdf)
            print(f"wrote PDF report -> {args.pdf}")


    if __name__ == "__main__":
        main()
```

8. user (2026-06-18T00:53:48.563Z)

```
1       ---
2       name: current-state
3       description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not
only on request). What just landed, what's next, open decisions."
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
8       ---
9
10      # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12      **Checkpoint:** 2026-06-18 ~00:10 (? **REMAINING-3 4K REELS DONE ? ALL 6 BOXHILL-17
CLIPS NOW HAVE 4K** + cost-scaling doc + 10/10 visual QA ? all on G:). Owner asked for "just 4K
highlights of the remaining 3, all local no Gemini." Delivered:
13      - ? **GX010138/139/140 4K reels** (the larger 9/10/12.8-min clips): **45/47/68
rallies**, 1.77/1.89/2.72 GB, 3840×2160, watermarked. Published to G: as `? - Highlights
(4K).mp4`. Robust detached Scheduled Task `KhelSutraBoxhill4KRemaining` (keep-awake + self-
restart; unregistered on completion).
14      - ? **DOGFOODED `--detect-from` end-to-end on fresh videos** (the merged #205 feature):
`--video <4K original> --detect-from <1080p proxy>` ? detect on proxy, stitch 4K from original,
reel named after --video (NO `_1080p` naming bug this time). Log confirmed "detecting on proxy ?
stitching reel from ?". Validated beyond the unit tests.
15      - ? **Visual QA workflow** (`wf_2bd49aa1-361`, 10 vision agents): **10/10 delivered
reels clean** ? correct resolution, watermark VISIBLY burned in, real rally content.
(Adversarial verification of the shipped artifacts.)
16      - ? **`COST_SCALING.md` (all 6 clips)** on G:. **Cost model (RTX 2070 Super Max-Q):**
GPU detection (WASB) is the driver at **~3.4× real-time** (?3.4 GPU-min/footage-min, ~0.055
s/frame); NVENC proxy ~0.2× RT; 4K stitch (CPU libx264) ~16 s/rally; end-to-end ~5× clip length;
peak VRAM ~5.7 GB (fits 8 GB). ? the per-video GPU-busy sampler OVER-counts (background desktop
GPU during the CPU stitch counted as busy) ? use the WASB wall-clock as the clean GPU number.
6-clip totals: 52.2 min footage ? 176 GPU-min WASB + 68 CPU-min stitch, 250 rallies, ~11.6 GB of
4K reels.
17      - ? **OWNER (2026-06-18): will GOLDEN-LABEL the Boxhill-17 clips TODAY.** ? this SETTLES
the open precision question (is local's +50% rally over-detection vs Gemini real rallies Gemini
missed, or false positives?) and graduates these clips into the golden corpus. **When labels
arrive:** at-par each (reuse `scratch/at_par.py` + the proxy video_ids in `output/boxhill17.db`
? local rallies cached: GX010141=22, GX020140=29, GX010137=39, GX010138=45, GX010139=47,
GX010140=68; Gemini for the first 3 in `pr-stitch/output/boxhill17_gemini.db` = 16/16/27) ?
score local & Gemini vs the golden truth (real P/R/F1, not just agreement) ? ingest as golden
videos + update `docs/GOLDEN_VIDEOS.md`. The clips are 1080p proxies in
`output/boxhill17_proxies/` (calibration resolution) ? same timeline as the 4K, so golden labels
apply 1:1. Until then, the local-vs-Gemini numbers are AGREEMENT not accuracy. [[golden-set-
vendoring]] [[eval-dual-set-regression]] [[eval-boundary-tolerance]]
18      - **(superseded by the above)** data-driven triage of the precision question
(`peak_velocity`/`active_frame_count` cut) is now moot once golden labels land. [[gotcha-long-
runs-gpu-wsl]] [[paper-coauthor-aim]]
19
20      **Checkpoint:** 2026-06-17 ~20:55 (? **WATERMARK BUG FIXED #205 (`8736e2d`) + 4K reels +
GEMINI COMPARISON ? all published to G:**; Boxhill spawned-task follow-ups DONE). Owner reviewed
the Boxhill reels and flagged 3 things; all addressed:
21      - ? **#205 MERGED (`8736e2d`, owner-authorized merge):** `fix(runner): honor stitching
watermark/padding + add --detect-from`. **ROOT CAUSE of missing watermarks:**
`tools/generate_highlights.py` built `VideoCompiler(local_work_dir)` BARE ? `watermark=None`
(off) + ctor `end_padding=1.0` instead of `config.json stitching.{watermark,
begin/end_padding=0.5}`. Every other compile site threaded config; the runner didn't. Fix reads
`cfg.get("stitching")` dict-compatibly + passes watermark+paddings. **+ `--detect-from`**:
detect on a cheaper proxy, stitch the reel from the full-res `--video` (the "default-to-
original, flag-to-save-bandwidth" split owner described); 1:1 paired, reel named after
`--video`. 3 new tests; ruff+mypy+844-pass+cov80.65% green; adversarial-review workflow
(findings were intended-changes or pre-existing, none must-fix). ? **LESSON:** I'd told the
```

owner the reels were "watermarked" WITHOUT verifying ? they weren't; owner caught it. Verify visual/output claims before asserting ([[working-agreement-attribution]]). Verified the fix by extracting a frame + eyeballing the mark.
22   - ? **Reels REGENERATED + published** (G:\?\Boxhill\2026-06-17 Local no-AI (improved windowing)): **1080p WATERMARKED** (re-stitched from cached rallies, overwrote un-watermarked) **+ 4K WATERMARKED** (`? (4K).mp4`, ~1.6?1.9GB each, stitched from the 4K originals; rallies 1:1 from DB, NO WASB re-run). Direct `VideoCompiler` re-stitch (`scratch/boxhill17_restitch.py`) ? run_predict has no CSV short-circuit so re-running the runner risks re-inference; direct re-stitch is guaranteed-fast. NOTE end_pad 1.0?0.5 (config) makes reels ~0.5s×N shorter than the first cut.
23   - ? **GEMINI COMPARISON** (owner ask): ran Gemini on the 3 proxies (separate DB `pr-stitch/output/boxhill17_gemini.db`, preserves local), stitched watermarked **Gemini reels** (`? - Gemini.mp4`) next to local + **`LOCAL_vs_GEMINI.md`** eval (`scratch/boxhill17_eval.py`, reuses R2 `tolerance_metrics`). **KEY (agreement, NOT accuracy ? clips not golden):** local **90 rallies vs Gemini 59** (local out-detects ALL 3); boundary agreement TIGHT (|?start|/|?end| <1s) on GX020140/GX010137 (the R2/R3/R4 windowing paying off); **GX010141 = high-disagreement outlier** (local 58% vs Gemini 19% coverage ? continuous drilling, ambiguous rally definition). Open Q: is local's +50% real rallies Gemini missed, or false positives? ? golden labels on these clips would settle it.
24   - ? Lingering: worktrees `boxhill17` (WASB cache + `boxhill17.db` + reels), `pr-stitch` (gemini DB/reels + a copied `.env`) ? keep until owner done eyeballing; tasks KhelSutraBoxhill17/Gemini/4K unregistered. [[gotcha-long-runs-gpu-wsl]]
25
26   **Checkpoint:** 2026-06-17 ~18:40 (? **R1 GUARD #201 + R5 BLOB-FALLBACK #203 SHIPPED**, master `f5f5e52`; ? **R1 config-flip = DRAFT #204 awaiting OWNER PRODUCT-GO** ? spawned R1 task complete). R-series: R1? R2? R3? R4? R5? all on master; the only remaining R1 step is the owner's shipped-behaviour flip.
27   - ? **R1 net over-seg guard ? #201 merged** (`8c68143`): `tolerance_metrics.over_seg_guard` refines R2 §2.3.3's strict "no split/merge increase on ANY video" (which mis-fires on the structural mega?bounded change) to a **NET guard on normalized RATES** ? passes iff corpus-mean weighted (w_merge=2>w_split=1) merge_rate+split_rate cost doesn't rise AND no per-video merge_rate regresses. Wired into `rally_seg_eval.compare()`/`--vs` (+ self-describing header, `--vs-max-window-duration`). **R1 EVIDENCE (n=11, R2 tolF1):** baseline?milestone(cap6+R4) **?+0.232 [+0.153,+0.304], sign 10/0/1, p=0.002**; **cap12?cap6 ?+0.108, sign 10/0, p=0.002** (R3 confirmed via sign-test, was means-only); cap6?+R4 +0.047, 8/0, p=0.008. mean merge_rate 0.694?0.150 (all 11 ?); **net guard PASS, strict BLOCK** (the artifact). `scratch/r1_evidence.py`.
28   - ? **R5 blob-fallback ? #203 merged** (`f5f5e52`, default-OFF): `tracknet_runner._blob_fallback_chop` ? AFTER the gap-split + R4 trim, midpoint-chop any group still > **blob_factor(4×)** the cap (the gap-less blob), **log + flag `forced_cut`**. The factor gate keeps it surgical (vs `hard_cap` which chops before R4 + over-segments normals). **mahadevpura_1 blob tolF1 0.000?0.238** (2 windows max 65.8s ? 32 max 4.1s); do-no-harm elsewhere at factor 4 (stratum tool, agg +0.030 n.s.). Wired config/preset/served/CLI `--blob-fallback`.
29   - ? **R1 config-flip = DRAFT PR #204** (`feat/r1-milestone-config-flip`, NOT merged ? shipped-behaviour change): flips IndexingConfig defaults `max_window_duration 0?6`, `window_inactivity_end 0?1.5`, `inactivity_rally_thresh 0.08?0.20`, `inactivity_min_density 0.34?0.30`. Config-only; **merging entails** 3 default-OFF test updates + the **nightly coupling** (`nightly_regression.tracked_configs` "default"=SERVED ? post-flip default==milestone, all-OFF drift baseline collapses ? decide: drop redundant milestone cfg, or re-pin an explicit all-off control). Left for owner; I finish the test/nightly updates once the direction is chosen.
30   - ? **DISPATCH (owner): R1 PRODUCT-GO** ? flip the milestone default ON (review/merge draft #204 after deciding the nightly-control question). Evidence clears the net-guard bar (?+0.232, 10/0, p=0.002). Also still open: **IAA study** (2nd labeler re-labels 2?3 golden clips ? fix R2 ?). [[eval-dual-set-regression]] [[eval-boundary-tolerance]] [[decision-rigor-norm]]
31   - ? Lingering worktrees to clean later: r1-guard, r5-blob, r1-flip, nightly-reg (sibling) + the older ones listed below.
32
33   **Checkpoint:** 2026-06-17 ~17:50 (? **NIGHTLY RALLY-QUALITY REGRESSION SHIPPED ? PR #202 MERGED**, master `2eaccd5` ? spawned task #3). The owner's "process videos every night as nightly tests" ask, as a CPU **tripwire** (NOT a re-training run): `scripts/nightly_regression.py` re-scores the golden corpus (n=11) under **DEFAULT** (SERVED, knobs OFF) + **MILESTONE** (R3 cap=6 + R4 inactivity-trim) via `rally_seg_eval.evaluate` (cached trajectories ? seconds, no GPU/Gemini/WASB) and diffs headline metrics + the per-video over-seg

guard vs frozen `eval_baselines/nightly_baseline.json`. Frozen n=11: DEFAULT tolF1 0.091/f1@0.5 0.194; MILESTONE tolF1 0.323/f1@0.5 0.440 (matches QUALITY_ITERATIONS R3/R4 ? cross-checks the harness). Exit 1=regression, 4=stale(corpus/config changed?`--update-baseline`); writes `output/nightly_regression/<date>.{md,json}`. **DEFAULT-ADDITIVE ? does NOT change the promotion gate.** 27 offline unit tests; verified full suite+ruff+mypy green (cov 80.69%), e2e on real n=11 (both configs PASS) + tripwire fires on simulated regressions.
34   - **Scheduled task registered:** `KhelSutraNightlyRegression` (daily **03:00**, user-level, StartWhenAvailable) ? `scripts/nightly_regression.ps1` wrapper, pointed at the MAIN checkout. **Inline-guarded ? currently no-ops (SKIPPED) because the main checkout is on the sibling branch (95cd154), not master; it SELF-ACTIVATES once the main checkout is on master** (per the clean-slate convention). Re-point/remove via `scripts/register_nightly_task.ps1`. A cloud agent can't run it (cached trajectories live outside the repo).
35   - ? **DISPATCH (owner):** when you next put the main checkout on master, the nightly goes live automatically (no action needed); to run it NOW: `Start-ScheduledTask KhelSutraNightlyRegression` from a master checkout, or `python scripts/nightly_regression.py`. Re-snapshot the baseline (`--update-baseline`) after R1 flips the milestone ON or when the golden corpus grows.
36   - ? **Pre-existing, OUT-OF-SCOPE (spawned task `task_5878d224`):** with the full n=11 corpus in `output/`, 2 `test_ablation.py` litmus tests fail LOCALLY (`KeyError: 'shuttle'`) ? the 5 newer real clips' `*_golden_features.json` carry a leaner velocity-windowing schema than the n=6-era litmus expects. CI unaffected (empty `output/`?skipped); the nightly path is immune (reads only `gts`+trajectory). [[eval-dual-set-regression]] [[working-agreement-merging]]
37
38   **Checkpoint:** 2026-06-17 ~18:15 (? **BOXHILL-17 3-VIDEO HIGHLIGHTS ? DONE + PUBLISHED to G:** ? spawned task #2 COMPLETE). LOCAL no-AI reels for the **3 shortest** Boxhill-17-June clips: **GX010141 (6:25), GX020140 (6:26), GX010137 (7:31)** (the other 3 ? GX010138/139/140 ? are 8-11.5GB, skipped to keep GPU time reasonable). Improved milestone windowing in worktree `.claude/worktrees/boxhill17` config.json (`max_window_duration=6.0`, `window_inactivity_end=1.5`, `inactivity_rally_thresh=0.20`, `inactivity_min_density=0.30`, `wasb.timeout_sec=5400`; verified served preset reads them, `window_hard_cap` stays False ? soft cap-6 = R3 op-point). Branch `run/boxhill17-highlights`@`34b9caa` (origin/master, R4). Flow per video: NVENC 4K?1080p proxy (`output/boxhill17_proxies`) ? runner `wasb_hybrid --skip-ai-handoff --wasb-stream --db output/boxhill17.db` ? reel to `output/boxhill17_reels` ? **published to `G:\My Drive\Sharing\Badminton\Boxhill\2026-06-17 Local no-AI (improved windowing)\`** (additive). GX010141: proxy ~80s (~4.8× realtime), WSL env OK (torch+CUDA), WASB inference running @17:02. ROBUST LAUNCH: detached **Scheduled Task `KhelSutraBoxhill17`** (keep-awake `SetThreadExecutionState` + self-restart, mirrors `extract_val_until_done.ps1`); wrapper `scratch/boxhill17_highlights.ps1`; **5-min heartbeat Monitor `bop8jl2j4`**. COST: per-video `_COST_REPORT.md` (wall-clock/phase + GPU-busy-min via 10s nvidia-smi sampling + peak VRAM) in `output/boxhill17_cost`. To resume/observe: tail `scratch/boxhill17_wrapper.log`; `Get-ScheduledTask KhelSutraBoxhill17`. Reels are 1080p (proxy-based); 4K stitch-from-original is a cheap follow-up (rallies already in DB). ? **RESULT:** all 3 published ? **GX010141 22 rallies/4:18, GX020140 29/3:42, GX010137 39/4:07** (90 total; h264 1080p watermarked, 257/222/247s; 55-67% of each clip kept). **COST (n=3, ~20.4min 4K footage, RTX 2070 Super Max-Q):** GPU-busy **67.2 min** total, WASB **65.1 min** (~3.2× clip duration ? ~1 GPU-min per ~18s footage), proxy 4.2 min, peak VRAM **5966 MB**. Per-video `_COST_REPORT.md` + `README.md` on G:. ? **LESSON (filename bug):** the `_1080p` proxy suffix made the runner's reel-stem (`GX010141_1080p`) ? the wrapper's stem (`GX010141`) ? the wrapper's INLINE cost-report/publish/skip looked for the wrong filename (cost reports blank, no auto-publish, would re-loop). WASB output was fine; fixed POST-HOC via `scratch/boxhill17_finalize.ps1` (regen cost from run.log + gpu.csv.bak snapshots + atomic publish). **NEXT TIME: name proxies EXACTLY like the original** (e.g. `GX010141.MP4` in a proxies dir, as the kushagra June-12 run did) so runner-stem==wrapper-stem. Scheduled Task `KhelSutraBoxhill17` unregistered; heartbeat self-terminated; worktree config.json edit left uncommitted (isolated, intentional). [[gotcha-long-runs-gpu-wsl]] [[feedback-tail-long-job-logs]]
39
40   **Checkpoint:** 2026-06-17 ~16:00 (? **R4 SHIPPED #200 `34b9caa`**; ? **CONTEXT-PAUSE + SPUN OFF the rest** per owner's explicit "ensure context window constraints, pause + spin off others"). R-series so far: **R2?(#199) R3?(finding, cap6) R4?(#200)** ? all on master. This session got very long ? paused; remaining work moved to fresh-context spawned tasks.
41   - **NEW owner asks (2026-06-17, excited):** (a) **full highlights for 3 videos** from `F:\GoPro Hero Black 12\KhelSutra\Badminton - Boxhill - 17 June` (6 clips: GX010137/138/139/140/141, GX020140) ? AFTER all R-iterations, using the improved milestone windowing (cap=6 + R4). (b) **Computational-cost measurement** (CPU-min + GPU-min per video) ? he's still mulling ("let me process it more"); instrument per-phase wall-clock + GPU-busy in the

highlights run. (c) **An "educated quality LAUNCH"** = flip the milestone defaults ON (this IS R1; he's now leaning yes ? still his product-go). (d) **Nightly regression set** ? a scheduled task that re-scores the golden corpus nightly to catch regressions (his dual-eval-set discipline; note: cached-trajectory RE-SCORE is CPU+fast, no need to re-run WASB nightly).
42 - **SPUN OFF (3 fresh-context tasks, each reads this file + the docs):** (1) R1 launch evidence + net-over-seg guard + R5 blob-splitter; (2) Boxhill-17-June 3-video highlights + cost instrumentation; (3) nightly golden-regression scheduled task. Each branches from `origin/master` (now `34b9caa`).
43 - **? DISPATCH (owner):** (1) ? **R1 LAUNCH APPROVED** ? owner is happy to merge the launch PR GIVEN the documented impact table (below), which is now provided. Owner wants it (a) **documented separately for posterity** (last quality-bit flip ? Gate-A #146 ? noted a regression, hence the caution) and (b) **merged by Claude** (the R1 spawned task `task_eac3f171`: ? confirm the sign-test + net-over-seg guard; ? write a **LAUNCH REPORT** ? owner's new STANDING convention for EVERY default config change, see [[launch-report-convention]] ? with the total-quality-impact table below + the Gate-A-lesson rationale, e.g. `docs/launches/2026-06-17-rally-windowing.md` + a QUALITY_ITERATIONS pointer; ? make the config-flip PR by editing the tracked `config.json` indexing block ? NOT the IndexingConfig dataclass defaults, which stay OFF so the eval substrate + default-OFF tests are unchanged ? and **MERGE it on green CI**. ? The original chip prompt said "leave the PR for owner go / do NOT flip"; THIS MEMORY SUPERSEDES that ? the product-go is GRANTED conditioned on the impact table, so create + merge it). (2) **IAA study** ? owner WILL find a 2nd annotator (endorsed it as good practice) ? fix R2 ?.
44 - ? **LAUNCH CONFIG = W1 cap=6 + R4 (inactivity_end=1.5, rally_thresh=0.20, min_density=0.30); NO hardcap (it over-splits ? lowers tolF1 0.323?0.288), NO W2 (opt-in/retained).** Blob clip (mahadevpura_1) handled by R5, not by adding hardcap to the launch. **TOTAL LAUNCH IMPACT (golden n=11, baseline?launch):** tIoU-F1@0.5 **0.194?0.440 (+0.246)** · R2 tolF1 **0.091?0.323 (+0.232)** · mean |?end| **52.1s?3.8s** · merge-count **4.9?1.5** (over-merge crushed); split 0?5.1 (the honest over-seg tradeoff, net tolF1 hugely +). Far more rigorous than the Gate-A flip (measured on 2 rulers + n=11 + R4's 8/0 sign-test p=0.008).
45 - ? Lingering worktrees to clean later: w2-safeguard, hardchop, docs-*, r4-endrule, gpu-validation, + older ones. Heartbeat monitors/Scheduled Tasks (KhelSutraValExtract, KhelSutraNewClipsExtract) are done ? can be unregistered. [[eval-dual-set-regression]] [[eval-boundary-tolerance]]
46
47 **Checkpoint:** 2026-06-17 ~15:00 (? **MULTI-DAY AUTONOMY MANDATE**: owner greenlit executing ALL R* I deem possible while he's busy ~2 days adding 10-12 golden labels + talking to first customers. "Dispatch via app/here when you need something." Trusts repo sanctity to me. ? keep the discipline: measured, default-OFF, PRs clear the bar, dispatch when blocked.)
48 - ? **R2 SHIPPED ? #199 merged** (`7e8f88f`): `backend/eval/tolerance_metrics.py` (strict-1:1 Hungarian tolerance-match, scipy-or-greedy; unconditional best-overlap boundary-error split start/end; over-seg guard; ?-sweep) + wired into `rally_seg_eval` (R2 block in score_intervals/aggregate/format_report/compare ?tolF1 + `--tau-start/--tau-end`). Default-ADDITIVE (tIoU still the gate pending the ablation experiment). Provisional ?_start=2.0/?_end=1.5. n=9 R2 aggregate: tolF1 0.197, |?start| 1.25s vs |?end| 2.19s (end=gap confirmed). R2 DESIGN doc #198 merged (owner locked the 3 decisions: asymmetric ?; IAA-after-code; augment?ablation-gate?deprecate-with-docs).
49 - ? **R3 MEASURED (cap-sweep, n=9):** **12s cap was too coarse ? cap?6 is best** (tolF1 cap6 0.273 > cap12 0.171). The |?end|-vs-split tension IS the R4 spec: plain W1 = loose ends (|?end|~7s, few splits); hard-cap = tight ends (~1s?Gemini, heavy over-split). **R4 target: hard-cap's tight ends WITHOUT the over-split** = end at the real inactivity point. (cap=6 is a means-lead; confirm via paired sign-test on n=11 before locking ? feeds R1's config value 12?6.)
50 - ? **R1 BLOCKER (measured, key insight):** baseline?W1+hardcap is a SIGNIFICANT tolF1 win (?+0.095, sign 8/1, p=0.039) AND kills merges everywhere (?0) ? BUT trips the strict "no split regression on ANY video" guard (split 0?4-14 on all 9: it trades merges for over-splits). ? R1 unblock options: (1) do R3+R4 first (reduce over-prod), (2) **refine guard to NET over-seg** (the honest guard; I'll implement this for R1), (3) ship as floor-raiser. Recommended owner: (2)+ship. R1 final flip = owner product-go (dispatch).
51 - ? **R3 + R4 MEASURED & VALIDATED (n=11, under R2):** R3 cap **12?6** (tolF1 0.171?0.276). **R4 rally-END inactivity trim** (`_trim_inactive_tail` in tracknet_runner, default-OFF: trims post-rally slow-drift tail ? frames stay "active" while shuttle drifts >velocity_thresh but <`inactivity_rally_thresh`; trim to last frame whose trailing window is ?`min_density` fast). **Sweet spot T=1.5s, rally_thresh=0.20, min_density=0.3:** cap6?+R4 **?tolF1 +0.047 [+0.022,+0.072], sign 8/0, p=0.008**, |?end| 5.71?3.81s, splits NET-DOWN (only mp2 +1; GX010128 4?2, BXH2 5?4). Beats hard-cap's mechanism (hardcap got |?end| 1.07 but split 19.5 over-prod). Cumulative tolF1: baseline 0.096 ? cap12 0.171 ? cap6 0.276 ? +R4 0.323. R4

code in worktree `feat/r4-inactivity-end` (NOT yet wired to preset/config/CLI ? that's the in-progress PR).
52   - ?? **NEXT (executing autonomously):** finish **R4 PR** (wire `window_inactivity_end`/`inactivity_rally_thresh`/`inactivity_min_density` through WindowingPreset+IndexingConfig+trajectory_hybrid+rally_seg_eval CLI + tests + QUALITY_ITERATIONS) ? re-check **R1** (cap=6 + R4 + net-over-seg guard) ? **R5** blob-splitter. **DISPATCH LIST for owner:** (1) R1 product-go when it clears; (2) **IAA study** (2nd labeler re-labels 2-3 golden clips ? fix ? ? I can't generate IAA myself); (3) FYI net-over-seg guard tweak (revert=1 line).
53   - ? **n=11 dessert in flight:** BXH2 (Badminton BXH 2, 44 golden) WASB ~done via Scheduled Task `KhelSutraNewClipsExtract` (heartbeat `bvoc96z90`); GX030094 (41 golden, 30fps) already at-par'd (3rd venue confirms detection~80%/boundary~51%/Gemini-under-detects pattern + Gemini missed 14). When BXH2 lands ? ingest both as golden #10/#11 (n=11) + at-par + update GOLDEN_VIDEOS.md (#197). Gemini for both already in `output/sports_indexer.db` (job bbofujewg done). [[eval-boundary-tolerance]] [[eval-dual-set-regression]] [[decision-rigor-norm]]
54
55   **Checkpoint:** 2026-06-17 ~14:00 (R2 DAY: golden corpus **n=9**; golden-tracker + R2 design docs shipped; 2 new clips extracting). Owner back, wants R2 today + focus on LOCAL on-par-with-Gemini. Master now `c483346`.
56   - **3 new `[Done]` badminton videos labeled** (in `?/Golden Label Videos Request - June 15/`): Video 14=**mahadevpura-1** (10 rallies, the blob), Video 6=**GX030094** (41 rallies, **29.97fps**), Video 10=**Badminton BXH 2** (44 rallies). (Video 13=GX010128 + Video 3=mahadevpura-2 = already-ingested dupes.)
57   - ? **mahadevpura-1 ingested as golden #9** + at-par run: it's local's WORST clip ? baseline/W1/W1+W2 score **F1 0.000** (1-2 mega-windows match nothing); **hard-cap rescues to 0.294/0.588** (F1@.5/@.25) vs Gemini 0.842. Clearest proof hard-cap is non-optional for the milestone.
58   - ? **GX030094 + Badminton BXH 2: WASB extracting** via robust Scheduled Task `KhelSutraNewClipsExtract` (`scratch/extract_newclips_until_done.ps1`, keep-awake + self-restart; heartbeat monitor `bvoc96z90`). GX030094 video_id `27f1c707bbfb`. ~1hr. When done ? Gemini + at-par + ingest as #10/#11 + update GOLDEN_VIDEOS.md. (Task registered WITHOUT -RunLevel Highest ? Highest needs admin this session lacks.)
59   - ? **#197 MERGED** ? `docs/GOLDEN_VIDEOS.md` golden-labeled video registry (owner will isolate them). ? **#198 OPEN for OWNER REVIEW** ? `docs/R2_EVAL_METRIC_DESIGN.md` (do NOT auto-merge; owner wants a deeper look).
60   - ? **R2 PREVIEW FINDING (reshapes the design):** a symmetric ±2s tolerance under strict Hungarian 1:1 is NOT more lenient than tIoU ? it makes local look FURTHER behind (correctly charges the 1.5× over-production + loose ends tIoU's overlap-matching absorbed; GX010128 local/Gemini ratio 0.56@tIoU0.5 ? 0.41@?2.0). Even Gemini scores low at tight ? (mp2 tolF1@1.5=0.20) ? **golden END convention carries >1.5s noise ? ?_end MUST be IAA-calibrated** (2nd-labeler study). R2 design = **asymmetric ? (?_start?2 forgives service window, ?_end?1.5 exposes the end-deficit) + Hungarian 1:1 + decomposition (P/R/F1@? + start/end boundary-error split + over-seg guard + ?-sweep)**; tIoU?secondary. `tolF1@~3.0?tIoU@0.5`.

9. assistant (2026-06-18T00:54:59.279Z)

I have a clear picture of the bug and the preferred fix. Let me verify there are no other consumers of this schema before I implement, and check what's actually in `output/` here.

10. user (2026-06-18T00:55:01.789Z)

```
backend\eval\ablation.py:42:writes one ``<name>_golden_features.json`` per video). CLI::
backend\eval\ablation.py:386:def load_videos(features_dir: str) -> List[VideoCandidates]:
backend\eval\ablation.py:387:    """Load every ``*_golden_features.json`` into a
``VideoCandidates`` (intervals + shuttle +
backend\eval\ablation.py:392:    for path in sorted(glob.glob(os.path.join(features_dir,
"*_golden_features.json"))):
backend\eval\ablation.py:399:            features[fn] = [float(c["shuttle"][fn]) for c in cands]
backend\eval\ablation.py:401:            features[fn] = [float(c["person"][fn]) for c in cands]
backend\eval\ablation.py:574:                    help="dir of *_golden_features.json (the
persisted golden substrate)")
backend\eval\ablation.py:586:    videos = load_videos(args.features_dir)
backend\eval\ablation.py:588:        raise SystemExit(f"need >=2 *_golden_features.json in
{args.features_dir} "
backend\config\models.py:88:    shuttle net-crossing count (Gate A signal) and ? only when
```

```
``cue_flags['person']`` is
scripts\nightly_regression.py:24:outside the repo + ``output/<name>_golden_features.json``); the
corpus is NOT committed. So the
scripts\nightly_regression.py:498:            f"and do the
{features_dir}/<name>_golden_features.json files have 'gts'?")
scripts\nightly_regression.py:529:                    help="dir holding
<name>_golden_features.json")
tests\test_ablation.py:13:  - the golden litmus reproduction ? on the 6 real
``output/*_golden_features.json`` the runner
tests\test_ablation.py:156:    assert rows["person"].delta_f1 > 0.0
tests\test_ablation.py:344:def test_load_videos_tolerates_missing_audio(tmp_path):
tests\test_ablation.py:353:    with open(tmp_path / "noaudio_golden_features.json", "w",
encoding="utf-8") as f:
tests\test_ablation.py:355:    videos = ablation.load_videos(str(tmp_path))
tests\test_ablation.py:367:    return len(glob.glob(os.path.join(GOLDEN_DIR,
"*_golden_features.json"))) >= 6
tests\test_ablation.py:371:                    reason="golden output/*_golden_features.json
absent (run fusion_golden first)")
tests\test_ablation.py:377:    videos = ablation.load_videos(GOLDEN_DIR)
tests\test_ablation.py:399:                    reason="golden output/*_golden_features.json
absent")
tests\test_ablation.py:402:    report = run_ablation(ablation.load_videos(GOLDEN_DIR),
ablation.litmus_config())
tests\test_ablation.py:445:        with open(os.path.join(d, f"v{k}_golden_features.json"), "w",
encoding="utf-8") as f:
tests\test_audio_features.py:50:    with open(os.path.join(d, f"{name}_golden_features.json"),
"w", encoding="utf-8") as f:
tests\test_audio_features.py:58:    videos = fusion_audio.load_videos_with_audio(d)
tests\test_audio_features.py:66:def test_load_videos_requires_audio(tmp_path):
tests\test_audio_features.py:73:    with open(os.path.join(str(tmp_path),
"x_golden_features.json"), "w", encoding="utf-8") as f:
tests\test_audio_features.py:76:        fusion_audio.load_videos_with_audio(str(tmp_path))
backend\eval\fusion_audio.py:29:    """Add the 3 audio features to each existing
``<name>_golden_features.json`` window."""
backend\eval\fusion_audio.py:34:        jp = os.path.join(features_dir,
f"{spec['name']}_golden_features.json")
backend\eval\fusion_audio.py:44:            c["audio"] = af.compute_audio_features(hits,
c["start"], c["end"])
backend\eval\fusion_audio.py:52:def load_videos_with_audio(features_dir: str) ->
List[experiment.VideoCandidates]:
backend\eval\fusion_audio.py:56:    for path in sorted(glob.glob(os.path.join(features_dir,
"*_golden_features.json"))):
backend\eval\fusion_audio.py:65:            features[fn] = [float(c["shuttle"][fn]) for c in
cands]
backend\eval\fusion_audio.py:67:            features[fn] = [float(c["person"][fn]) for c in
cands]
backend\eval\fusion_audio.py:69:            features[fn] = [float(c["audio"][fn]) for c in
cands]
backend\eval\fusion_audio.py:121:        videos = load_videos_with_audio(args.features_dir)
backend\eval\fusion_compare.py:35:def load_videos(features_dir: str) ->
List[experiment.VideoCandidates]:
backend\eval\fusion_compare.py:36:    """Load every ``*_golden_features.json`` into a
VideoCandidates (intervals + the 10
backend\eval\fusion_compare.py:39:    for path in sorted(glob.glob(os.path.join(features_dir,
"*_golden_features.json"))):
backend\eval\fusion_compare.py:46:            features[fn] = [float(c["shuttle"][fn]) for c in
cands]
backend\eval\fusion_compare.py:48:            features[fn] = [float(c["person"][fn]) for c in
cands]
backend\eval\fusion_compare.py:91:    videos = load_videos(args.features_dir)
backend\eval\fusion_compare.py:93:        raise SystemExit(f"need >=2 *_golden_features.json in
{args.features_dir} (found {len(videos)})")
backend\eval\fusion_golden.py:14:Each output `output/<name>_golden_features.json` is the
replayable feature substrate ? re-run
backend\eval\fusion_golden.py:149:        out = os.path.join(out_dir,
```

```
f"{spec['name']}_golden_features.json")
backend\eval\golden_fixtures.py:352:    # fusion_compare.load_videos / ablation.load_videos ?
those read c['person'] unconditionally
backend\eval\rally_seg_eval.py:17:``output/<name>_golden_features.json`` (the ``gts`` key) so
this needs no extra label files.
backend\eval\rally_seg_eval.py:104:    intervals (``<name>_golden_features.json`` ``gts``).
Videos with no GTs are kept but flagged."""
backend\eval\rally_seg_eval.py:109:        feat = os.path.join(features_dir,
f"{v['name']}_golden_features.json")
backend\eval\serve_contrast.py:15:Unlike the persisted ``*_golden_features.json`` (which are
PROPOSAL-windowed only), this
tests\test_fusion_compare.py:29:    with open(os.path.join(d, f"{name}_golden_features.json"),
"w", encoding="utf-8") as f:
tests\test_fusion_compare.py:37:    videos = fusion_compare.load_videos(d)
tests\test_fusion_compare.py:55:    res = fusion_compare.compare(fusion_compare.load_videos(d),
iou=0.5)
tests\test_fusion_compare.py:67:    videos = fusion_compare.load_videos(str(tmp_path))
tests\test_fusion_compare.py:101:    assert set(c0["shuttle"]) == set(SHUTTLE_FEATURE_NAMES)
tests\test_fusion_compare.py:102:    assert set(c0["person"]) == set(PERSON_FEATURE_NAMES)
tests\test_fusion_compare.py:105:    assert all(isinstance(v, float) for v in
c0["person"].values())
tests\test_fusion_compare.py:121:    with open(os.path.join(out_dir,
"alpha_golden_features.json"), "w", encoding="utf-8") as f:
tests\test_fusion_golden_telemetry.py:145:    assert written == [os.path.join(out_dir,
"alpha_golden_features.json")]
tests\test_fusion_golden_telemetry.py:170:    assert written == [os.path.join(out_dir,
"alpha_golden_features.json")]
tests\test_golden_fixtures.py:65:        assert set(cand["shuttle"].keys()) ==
set(gf.SHUTTLE_FEATURES), (
```

11. user (2026-06-18T00:55:01.797Z)

```
output\adarsh_avi_singles_golden_features.json
output\kushagra_singles_golden_features.json
output\largetest_doubles_golden_features.json
output\gbaaddy_golden_features.json
output\testlarge_short_golden_features.json
output\mahadevpura_singles_golden_features.json
output\mahadevpura_2_golden_features.json
output\GX010128_golden_features.json
output\mahadevpura_1_golden_features.json
output\GX030094_golden_features.json
output\Badminton_BXH_2_golden_features.json
```

12. user (2026-06-18T00:55:02.755Z)

```
Found 27 files
scripts\nightly_regression.py
eval_baselines\fixtures\occlusion_break\expected.json
eval_baselines\fixtures\multi_rally_with_deadtime\expected.json
eval_baselines\fixtures\clean_single_rally\expected.json
eval_baselines\fixtures\README.md
docs\QUALITY_ITERATIONS.md
docs\GOLDEN_REGRESSION_FIXTURES.md
docs\GOLDEN_VIDEOS.md
docs\GOLDEN_DATA_SHARING.md
docs\CODE_MAP.md
backend\eval\rally_seg_eval.py
backend\eval\golden_fixtures.py
docs\README.md
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md
docs\ABLATION_LAYER.md
docs\NEXT_STEPS.md
backend\eval\serve_contrast.py
eval_baselines\experiment_leaderboard.json
```

```
tests\test_ablation.py
backend\eval\ablation.py
tests\test_fusion_golden_telemetry.py
backend\eval\fusion_golden.py
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md
tests\test_fusion_compare.py
backend\eval\fusion_compare.py
backend\eval\fusion_audio.py
tests\test_audio_features.py
```

13. assistant (2026-06-18T00:55:27.633Z)

```
`output/` here is fully populated with all 11 files ? including the 5 leaner ones. So I can
**reproduce and verify locally**. Let me confirm the two schemas and check whether the test
suite runs in this container.
```

14. user (2026-06-18T00:55:32.395Z)

```
=== adarsh_avi_singles (full fusion) ? first candidate keys ===
['audio', 'end', 'label', 'person', 'shuttle', 'start']
=== mahadevpura_1 (leaner) ? first candidate keys ===
['active_frame_count', 'chunk_index', 'end', 'mean_velocity', 'peak_velocity', 'start',
'track_id_count']
=== schema per file (has shuttle?) ===
LEANER Badminton_BXH_2_golden_features.json | ncands= 4
LEANER GX010128_golden_features.json | ncands= 15
LEANER GX030094_golden_features.json | ncands= 30
FUSION adarsh_avi_singles_golden_features.json | ncands= 172
FUSION gbaaddy_golden_features.json | ncands= 82
FUSION kushagra_singles_golden_features.json | ncands= 20
FUSION largetest_doubles_golden_features.json | ncands= 86
LEANER mahadevpura_1_golden_features.json | ncands= 1
LEANER mahadevpura_2_golden_features.json | ncands= 5
FUSION mahadevpura_singles_golden_features.json | ncands= 57
FUSION testlarge_short_golden_features.json | ncands= 14
```

15. user (2026-06-18T00:55:37.780Z)

```
numpy 2.4.6
scipy 1.17.1
pytest 9.0.3
ablation import OK
```

16. assistant (2026-06-18T00:55:44.393Z)

```
Confirmed: exactly 6 FUSION files + 5 LEANER files, and numpy/scipy/pytest are available ? so I
can fully reproduce and verify. Let me reproduce the failure first.
```

17. user (2026-06-18T00:55:49.134Z)

```
features_dir = 'C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\output'

    def load_videos(features_dir: str) -> List[VideoCandidates]:
        """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals + shuttle
+
        person + audio columns + golden gts). Audio columns are optional (older JSONs predate
the
        audio augment) ? a missing block defaults those columns to 0.0 so the loader never
crashes,
        mirroring ``experiment._feature_column``'s tolerance."""
        videos: List[VideoCandidates] = []
        for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
            with open(path, encoding="utf-8") as f:
                d = json.load(f)
```

```
            cands = d["candidates"]
            intervals = [(float(c["start"]), float(c["end"])) for c in cands]
            features: Dict[str, List[float]] = {}
            for fn in SHUTTLE_COLUMNS:
>               features[fn] = [float(c["shuttle"][fn]) for c in cands]
                                      ^^^^^^^^^^^^
E               KeyError: 'shuttle'

backend\eval\ablation.py:399: KeyError
=========================== short test summary info ===========================
FAILED tests/test_ablation.py::test_litmus_reproduces_gate_a - KeyError: 'shu...
!!!!!!!!!!!!!!!!!!!!!!!! stopping after 1 failures !!!!!!!!!!!!!!!!!!!!!!!!
1 failed in 2.29s
```

18. assistant (2026-06-18T00:55:56.291Z)

Reproduced exactly (`KeyError: 'shuttle'`). Now let me set up an isolated worktree from `origin/master` and implement the fix there.

19. user (2026-06-18T00:56:00.242Z)

```
=== origin/master HEAD ===
be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)
=== creating worktree ===
Preparing worktree (new branch 'fix/ablation-litmus-mixed-schema')
branch 'fix/ablation-litmus-mixed-schema' set up to track 'origin/master'.
HEAD is now at be39a70 docs: rally-detection quality report ? technical-paper checkpoint
(2026-06-17) (#206)
=== verify worktree HEAD ===
be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)
fix/ablation-litmus-mixed-schema
```

20. assistant (2026-06-18T00:56:05.883Z)

Worktree is on `origin/master` (be39a70). The main checkout was on a different branch (ahead 18), so let me confirm the worktree's versions of these files match what I analyzed before editing.

21. user (2026-06-18T00:56:08.616Z)

```
=== diff ablation.py (worktree-master vs my-checkout) ===
exit: 0  (empty == identical)
=== diff test_ablation.py ===
=== confirm load_videos region in worktree ===
def load_videos(features_dir: str) -> List[VideoCandidates]:
    """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals + shuttle +
    person + audio columns + golden gts). Audio columns are optional (older JSONs predate the
    audio augment) ? a missing block defaults those columns to 0.0 so the loader never crashes,
    mirroring ``experiment._feature_column``'s tolerance."""
    videos: List[VideoCandidates] = []
    for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
        with open(path, encoding="utf-8") as f:
            d = json.load(f)
        cands = d["candidates"]
        intervals = [(float(c["start"]), float(c["end"])) for c in cands]
        features: Dict[str, List[float]] = {}
        for fn in SHUTTLE_COLUMNS:
            features[fn] = [float(c["shuttle"][fn]) for c in cands]
        for fn in PERSON_COLUMNS:
            features[fn] = [float(c["person"][fn]) for c in cands]
        for fn in AUDIO_COLUMNS:
            features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
        gts = [(float(a), float(b)) for a, b in d["gts"]]
        videos.append(VideoCandidates(name=d["name"], intervals=intervals,
```

```
                                    features=features, gts=gts))
    return videos
```

22. assistant (2026-06-18T00:57:12.164Z)

Files match master exactly. Now I'll implement the fix in the worktree. Let me read the worktree copies (Edit requires an in-conversation Read of the exact path).

23. user (2026-06-18T00:57:13.272Z)

```
383     # ---------------------------------------------------------------------------- corpus
I/O
384
385
386     def load_videos(features_dir: str) -> List[VideoCandidates]:
387         """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals +
shuttle +
388         person + audio columns + golden gts). Audio columns are optional (older JSONs
predate the
389         audio augment) ? a missing block defaults those columns to 0.0 so the loader never
crashes,
390         mirroring ``experiment._feature_column``'s tolerance."""
391         videos: List[VideoCandidates] = []
392         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
393             with open(path, encoding="utf-8") as f:
394                 d = json.load(f)
395             cands = d["candidates"]
396             intervals = [(float(c["start"]), float(c["end"])) for c in cands]
397             features: Dict[str, List[float]] = {}
398             for fn in SHUTTLE_COLUMNS:
399                 features[fn] = [float(c["shuttle"][fn]) for c in cands]
400             for fn in PERSON_COLUMNS:
401                 features[fn] = [float(c["person"][fn]) for c in cands]
402             for fn in AUDIO_COLUMNS:
403                 features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
404             gts = [(float(a), float(b)) for a, b in d["gts"]]
405             videos.append(VideoCandidates(name=d["name"], intervals=intervals,
406                                           features=features, gts=gts))
407         return videos
```

24. assistant (2026-06-18T00:57:20.420Z)

Now I'll implement the fix. First, the loader in `ablation.py`:

25. user (2026-06-18T00:57:31.402Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-fix\backend\eval\ablation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

26. assistant (2026-06-18T00:57:35.585Z)

Now the test file. Let me read the worktree copy's relevant regions, then update `_golden_present()` and add the regression tests + helper.

27. user (2026-06-18T00:57:36.717Z)

```
340
341     # ---------------------------------------------------------------------- corpus loader
(no GPU)
342
343
344     def test_load_videos_tolerates_missing_audio(tmp_path):
345         """An older feature JSON without an ``audio`` block still loads ? audio columns
```

```
default to
346         0.0 (loader never crashes on a pre-audio JSON)."""
347         cand = {"start": 0.0, "end": 10.0,
348                 "shuttle": {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS},
349                 "person": {fn: 0.0 for fn in ablation.PERSON_COLUMNS},
350                 "label": 1}
351         data = {"name": "noaudio", "fps": 30.0, "frame_width": 1920.0,
352                 "candidates": [cand], "gts": [[0.0, 10.0]]}
353         with open(tmp_path / "noaudio_golden_features.json", "w", encoding="utf-8") as f:
354             json.dump(data, f)
355         videos = ablation.load_videos(str(tmp_path))
356         assert len(videos) == 1
357         assert videos[0].features["audio_hit_count"] == [0.0]        # defaulted, not
crashed
358
359
360     # ------------------------------------------------------------------------ the
golden litmus
361
362     GOLDEN_DIR = os.path.join(
363         os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "output")
364
365
366     def _golden_present() -> bool:
367         return len(glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json"))) >= 6
368
369
```

28. user (2026-06-18T00:57:55.247Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-
fix\tests\test_ablation.py has been updated successfully. (file state is current in your context
? no need to Read it back)
```

29. assistant (2026-06-18T00:57:59.747Z)

Now update `_golden_present()` so the litmus skip-gate counts only ablatable fusion files (keeping the gate consistent with the loader):

30. user (2026-06-18T00:58:06.977Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-
fix\tests\test_ablation.py has been updated successfully. (file state is current in your context
? no need to Read it back)
```

31. assistant (2026-06-18T00:58:21.579Z)

The worktree has no `output/` dir, so the litmus would skip there. To faithfully reproduce the owner's box (mixed corpus present), let me copy the 11 golden files into the worktree's `output/` and confirm `output/` is gitignored so they won't be staged.

32. user (2026-06-18T00:58:24.719Z)

```
=== is output/ gitignored? ===
output/
(output/ IS ignored)
=== copy the 11 golden_features.json into the worktree output/ ===
11
=== worktree git status (should show NO output files staged/untracked) ===
 M backend/eval/ablation.py
 M tests/test_ablation.py
```

33. assistant (2026-06-18T00:58:30.270Z)

`output/` is ignored (only my 2 source edits show as modified). Now run the targeted tests in the worktree:

34. user (2026-06-18T00:58:36.183Z)

```
=== the two formerly-failing litmus tests + the new regression tests ===
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-
fix
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 4 items

tests/test_ablation.py::test_litmus_reproduces_gate_a PASSED             [ 25%]
tests/test_ablation.py::test_litmus_report_is_well_formed PASSED         [ 50%]
tests/test_ablation.py::test_is_fusion_schema_distinguishes_lean_and_full PASSED [ 75%]
tests/test_ablation.py::test_load_videos_skips_windowing_only_schema PASSED [100%]

============================== 4 passed in 2.30s ==============================
```

35. assistant (2026-06-18T00:58:41.437Z)

All 4 green ? and the litmus reproduces the exact Gate-A numbers (?F1?0.074, 6/6, p?0.031), proving the filter picks exactly the right 6 fusion files. Now the full ablation file, then the whole suite + ruff + mypy.

36. user (2026-06-18T00:59:02.786Z)

```
.............................                                            [100%]
29 passed in 18.72s
```

37. user (2026-06-18T00:59:26.922Z)

```
=== ruff (changed files) ===
All checks passed!
=== mypy ablation.py ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 1 source file
```

38. assistant (2026-06-18T00:59:36.296Z)

29/29 in the ablation file, ruff clean, mypy clean. Now the full suite (owner's pre-LGTM gate). Let me check how `run_all_tests.py` runs and execute it ? the worktree's `output/` now mirrors the owner's box, so the litmus runs for real here.

39. user (2026-06-18T00:59:37.477Z)

```
#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.

A single, discoverable entry point so you (or a reviewer / CI) can verify the
whole suite before opening a PR:

    python run_all_tests.py            # full suite, quiet
    python run_all_tests.py -v         # verbose (passthrough to pytest)
    python run_all_tests.py -k config  # filter by keyword (passthrough)
```

```
    python run_all_tests.py --cov=backend --cov-report=term-missing   # coverage

Anything after the script name is passed straight through to pytest.

Notes
-----
- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional ``obsreport``
  package isn't installed (they use ``pytest.importorskip``). To exercise them too:
      pip install -e ../sports-obsreport --no-build-isolation
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
"""
from __future__ import annotations

import importlib.util
import subprocess
import sys


def _installed(module: str) -> bool:
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError):
        return False


def main() -> int:
    args = list(sys.argv[1:])
    # Default to quiet unless the caller chose a verbosity.
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
        args = ["-q", *args]
```

40. user (2026-06-18T01:00:53.327Z)

```
================================================================================
 Full test suite:  C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
 obsreport (optional reporting dep): available
================================================================================
........................................................................ [  8%]
........................................................................ [ 16%]
........................................................................ [ 25%]
........................................................................ [ 33%]
........................................................................ [ 42%]
........................................................................ [ 50%]
........................................................................ [ 59%]
........................................................................ [ 67%]
.........................................................................sss [ 75%]
ss...................................................................... [ 84%]
........................................................................ [ 92%]
........................................................               [100%]
848 passed, 5 skipped in 70.47s (0:01:10)
================================================================================
 RESULT: PASS - all tests green
================================================================================
```

41. assistant (2026-06-18T01:01:01.925Z)

Full suite: **848 passed, 5 skipped** (the litmus ran for real here and passed). Now let me match CI exactly for repo-wide ruff + mypy and the coverage floor. Let me find the CI invocations.

42. user (2026-06-18T01:01:04.178Z)

```
=== CI config files ===
ci.yml
--- grep CI commands (ruff/mypy/cov-fail-under) ---
.github/workflows/ci.yml:31:        run: ruff check . --output-format github
.github/workflows/ci.yml:35:  # up. Typed core deps are installed so mypy sees their real
signatures
.github/workflows/ci.yml:45:      - name: Install mypy + typed core deps
.github/workflows/ci.yml:46:        run: python -m pip install mypy pydantic fastapi uvicorn
python-dotenv google-genai numpy
.github/workflows/ci.yml:48:        run: mypy backend training
.github/workflows/ci.yml:97:        run: python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70
```

43. assistant (2026-06-18T01:01:08.922Z)

CI runs `ruff check .`, `mypy backend training`, and the suite with `--cov-fail-under=70`. Let me run all three to match.

44. user (2026-06-18T01:01:48.939Z)

```
=== ruff check . ===
All checks passed!
=== mypy backend training ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 104 source files
```

45. assistant (2026-06-18T01:01:52.786Z)

ruff clean, mypy clean (104 files). Now the coverage gate (`--cov-fail-under=70`):

46. user (2026-06-18T01:03:44.603Z)

```
backend\utils\run_telemetry.py                       126     12    90%
backend\utils\video.py                                25      3    88%
training\__init__.py                                   0      0   100%
training\gen0\__init__.py                              0      0   100%
training\gen0\harness.py                             108     11    90%
----------------------------------------------------------------------
TOTAL                                               7985   1544    81%
Required test coverage of 70% reached. Total coverage: 80.66%
848 passed, 5 skipped, 7 warnings in 107.27s (0:01:47)
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

47. assistant (2026-06-18T01:03:52.237Z)

Coverage **80.66%** (floor 70, matches the ~80.6% baseline ? no regression). Let me confirm `ablation.py`'s own coverage didn't slip:

48. user (2026-06-18T01:04:25.738Z)

```
backend\eval\ablation.py     257      1    99%   632
TOTAL                        257      1    99%
```

49. assistant (2026-06-18T01:04:30.538Z)

ablation.py at **99%** (only the `__main__` guard line is uncovered) ? new code fully exercised.
Let me review the complete diff before committing.

50. user (2026-06-18T01:04:31.722Z)

```
diff --git a/backend/eval/ablation.py b/backend/eval/ablation.py
index fec3e7d..968eefd 100644
--- a/backend/eval/ablation.py
+++ b/backend/eval/ablation.py
@@ -383,16 +383,37 @@ def litmus_config() -> AblationConfig:
 # ---------------------------------------------------------------------------- corpus I/O


+def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
+    """True iff every candidate row carries the per-cue feature dicts the ablation scorer reads
+    (``shuttle`` + ``person``; ``audio`` stays optional ? older JSONs predate it).
+
+    The velocity-windowing / distill path persists a LEANER ``*_golden_features.json`` row
+    (``start``/``end`` + velocity stats, no cue dicts) for clips that never went through fusion
+    feature extraction. ``load_videos`` SKIPS those rather than crash on ``c["shuttle"]`` ? so
a
+    mixed corpus (some fusion files, some windowing-only) loads the ablatable subset cleanly.
+    An empty candidate list is not ablatable either."""
+    return bool(candidates) and all("shuttle" in c and "person" in c for c in candidates)
+
+
 def load_videos(features_dir: str) -> List[VideoCandidates]:
-    """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals + shuttle +
-    person + audio columns + golden gts). Audio columns are optional (older JSONs predate the
-    audio augment) ? a missing block defaults those columns to 0.0 so the loader never crashes,
-    mirroring ``experiment._feature_column``'s tolerance."""
+    """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a ``VideoCandidates``
+    (intervals + shuttle + person + audio columns + golden gts). Audio columns are optional
+    (older JSONs predate the audio augment) ? a missing block defaults those columns to 0.0 so
+    the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
+
+    Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the
+    R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus loads the
+    ablatable subset cleanly (see ``_is_fusion_schema``)."""
     videos: List[VideoCandidates] = []
     for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
         with open(path, encoding="utf-8") as f:
             d = json.load(f)
         cands = d["candidates"]
+        if not _is_fusion_schema(cands):
+            logger.info("ablation.load_videos: skipping %s ? windowing-only schema "
+                        "(no shuttle/person cue dicts), not an ablatable fusion-feature file",
+                        os.path.basename(path))
+            continue
         intervals = [(float(c["start"]), float(c["end"])) for c in cands]
         features: Dict[str, List[float]] = {}
         for fn in SHUTTLE_COLUMNS:
diff --git a/tests/test_ablation.py b/tests/test_ablation.py
index b29ecbb..224d497 100644
--- a/tests/test_ablation.py
+++ b/tests/test_ablation.py
@@ -357,6 +357,48 @@ def test_load_videos_tolerates_missing_audio(tmp_path):
     assert videos[0].features["audio_hit_count"] == [0.0]        # defaulted, not crashed


+def _write_windowing_only(d: str, name: str) -> None:
+    """Write a LEANER velocity-windowing/distill ``*_golden_features.json`` ? the schema the
+    R3/R4 corpus-growth path persists for clips that never went through fusion feature
+    extraction (``start``/``end`` + velocity stats, NO shuttle/person/audio cue dicts). The
+    ablation loader must SKIP these, never crash on ``c["shuttle"]``."""
```

```
+    cands = [{"start": 0.0, "end": 5.0, "active_frame_count": 90, "chunk_index": 0,
+              "mean_velocity": 2.0, "peak_velocity": 9.0, "track_id_count": 1}]
+    data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
+            "candidates": cands, "gts": [[0.0, 5.0]]}
+    with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as f:
+        json.dump(data, f)
+
+
+def test_is_fusion_schema_distinguishes_lean_and_full():
+    """The schema predicate keeps full-fusion rows (shuttle+person, audio optional) and rejects
+    the leaner velocity-windowing rows (no cue dicts); an empty candidate list is not
ablatable."""
+    full = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}, "audio": {}}]
+    no_audio = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}}]          # audio
optional
+    lean = [{"start": 0.0, "end": 1.0, "mean_velocity": 1.0, "track_id_count": 1}]
+    assert ablation._is_fusion_schema(full) is True
+    assert ablation._is_fusion_schema(no_audio) is True
+    assert ablation._is_fusion_schema(lean) is False
+    assert ablation._is_fusion_schema([]) is False
+
+
+def test_load_videos_skips_windowing_only_schema(tmp_path):
+    """Mixed-corpus regression (the n=11 ``KeyError: 'shuttle'``): full-fusion files load; the
+    leaner velocity-windowing files (the 5 newer real clips) are SKIPPED rather than crashed
on,
+    and the litmus path runs end-to-end over the filtered ablatable subset."""
+    d = str(tmp_path)
+    _write_learned_corpus(d, n=6)                                  # 6 full-fusion files
(v0..v5)
+    for name in ("mahadevpura_1", "GX010128", "Badminton_BXH_2"):  # 3 leaner real-clip names
+        _write_windowing_only(d, name)
+    # only the fusion-schema files load ? the leaner ones never crash the loader
+    videos = ablation.load_videos(d)
+    assert {v.name for v in videos} == {f"v{k}" for k in range(6)}
+    # ?and the litmus path (which reads the cue columns) runs cleanly on the mixed corpus
+    report = run_ablation(videos, ablation.litmus_config())
+    assert [r.signal for r in report.rows] == ["gate_a"]
+    assert report.num_videos == 6
+
+
 # ------------------------------------------------------------------------------ the golden
litmus

 GOLDEN_DIR = os.path.join(
@@ -364,7 +406,20 @@ GOLDEN_DIR = os.path.join(


 def _golden_present() -> bool:
-    return len(glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json"))) >= 6
+    """True iff ?6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the shuttle/person
cue
+    columns). Counts only ablatable files ? leaner velocity-windowing files are skipped by
+    ``load_videos``, so counting them would trip the litmus on an un-ablatable corpus (the n=11
+    mixed-schema bug)."""
+    n = 0
+    for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
+        try:
+            with open(p, encoding="utf-8") as f:
+                cands = json.load(f).get("candidates") or []
+        except (OSError, ValueError):
+            continue
+        if ablation._is_fusion_schema(cands):
+            n += 1
+    return n >= 6
```

```
 @pytest.mark.skipif(not _golden_present(),
```

51. assistant (2026-06-18T01:04:57.086Z)

```
Diff is clean and fully covered. Now I'll commit per the shared-checkout protocol: re-verify
HEAD, stage explicit paths only, commit.
```

52. user (2026-06-18T01:05:00.211Z)

```
=== re-verify branch + HEAD before staging ===
fix/ablation-litmus-mixed-schema
be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)
=== stage explicit paths only ===
=== staged (should be exactly 2 source files, NO output/) ===
M  backend/eval/ablation.py
M  tests/test_ablation.py
```

53. assistant (2026-06-18T01:05:03.333Z)

```
Exactly 2 files staged on my branch atop `origin/master`. Committing:
```

54. user (2026-06-18T01:05:13.990Z)

```
[fix/ablation-litmus-mixed-schema 59ae347] fix(eval): ablation litmus tolerates a mixed-schema
golden corpus
 2 files changed, 81 insertions(+), 5 deletions(-)
=== verify only my commit sits atop origin/master ===
59ae347 fix(eval): ablation litmus tolerates a mixed-schema golden corpus
=== diffstat ===
 backend/eval/ablation.py | 29 ++++++++++++++++++++----
 tests/test_ablation.py   | 57 ++++++++++++++++++++++++++++++++++++++++++++++++-
 2 files changed, 81 insertions(+), 5 deletions(-)
```

55. assistant (2026-06-18T01:05:36.889Z)

```
Committed cleanly ? exactly one commit atop `origin/master`, 2 files.

While reading, I found the **identical latent bug** in two sibling loaders that aren't currently
failing any test (their tests use synthetic tmp data, not the real `output/`), so they're out of
scope for this focused PR but would crash the same way on the n=11 corpus. Let me flag that as a
follow-up, then update the live checkpoint.
```

56. user (2026-06-18T01:05:48.305Z)

```
Noted (position 1, task_id: task_8dd4efde). A chip is showing for the user ? they can start it
in a fresh worktree with one click, or dismiss it. If this suggestion becomes stale or
superseded, call dismiss_task with this task_id. Currently pending: task_8dd4efde "Harden
fusion_compare/fusion_audio loaders for mixed schema". Continue your current work.
```

57. assistant (2026-06-18T01:05:50.991Z)

```
Now let me update the live checkpoint to record that this previously out-of-scope item is fixed.
```

58. user (2026-06-18T01:05:52.053Z)

```
10      # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12      **Checkpoint:** 2026-06-18 ~00:10 (? **REMAINING-3 4K REELS DONE ? ALL 6 BOXHILL-17
CLIPS NOW HAVE 4K** + cost-scaling doc + 10/10 visual QA ? all on G:). Owner asked for "just 4K
highlights of the remaining 3, all local no Gemini." Delivered:
```

59. user (2026-06-18T01:06:07.429Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

60. assistant (2026-06-18T01:06:22.439Z)

Done. The two failing litmus tests are fixed, committed, and fully verified locally.

What was wrong

`ablation.load_videos()` read `c["shuttle"]`/`c["person"]` on every candidate. The 5 newer R3/R4 clips (`Badminton_BXH_2`, `GX010128`, `GX030094`, `mahadevpura_1`, `mahadevpura_2`) are persisted via the velocity-windowing/distill path with a leaner schema (no cue dicts), so the loader threw `KeyError: 'shuttle'` whenever the full n=11 corpus was in `output/`. I reproduced it here exactly (this box had all 11 files).

The fix (option a ? most robust)

backend/eval/ablation.py:
- **`_is_fusion_schema()`** ? a file is ablatable iff every candidate carries `shuttle`+`person` (audio stays optional, preserving the existing pre-audio tolerance).
- **`load_videos()`** skips leaner windowing-only files with an info log instead of crashing ? a mixed corpus loads only the ablatable subset.
- **`_golden_present()`** (test gate) now counts only fusion-schema files, so the litmus skip-gate matches the loader (it won't fire the litmus on an un-ablatable corpus).

The 6 fusion-schema files are exactly the original n=6, so the litmus still reproduces the recorded Gate-A result (?F1?0.074, 6/6, p?0.031) ? those assertions pass against real data, not just "no crash."

Regression coverage

`test_load_videos_skips_windowing_only_schema` (mixed corpus ? only fusion files load + litmus runs end-to-end) and `test_is_fusion_schema_distinguishes_lean_and_full`. Offline/mocked.

Verification (with `output/` mirroring the owner box, so the litmus ran for real)

- Full suite: **848 passed, 5 skipped**
- `ruff check .`: clean · `mypy backend training`: clean (104 files)
- Coverage **80.66%** (floor 70, no regression); `ablation.py` itself at 99%

State

- Branch `fix/ablation-litmus-mixed-schema` @ `59ae347`, **one commit** atop `origin/master` (`be39a70`), in worktree `.claude/worktrees/ablation-schema-fix`. Staged explicit paths only; the golden JSONs I copied into the worktree's `output/` are gitignored and not committed.
- **Not pushed / no PR** ? per CLAUDE.md "don't open a PR unless asked." Say the word and I'll push, open the PR, and merge it (you've durably authorized merging good-engineering fixes on green CI).
- Did **not** touch `nightly_regression.py`.

One follow-up flagged (chip `task_8dd4efde`)

`fusion_compare.load_videos` and `fusion_audio.load_videos_with_audio` have the **identical** unconditional `c["shuttle"]` access. They aren't red today (their tests only use synthetic tmp corpora), but the same CLIs would crash on the real mixed corpus. Kept out of this focused PR per the one-PR-per-item convention.

61. user (2026-06-18T01:08:34.339Z)

please report this as a github issue, attach the PR and merge it ensuring the due diligence in the repo.

62. assistant (2026-06-18T01:09:00.963Z)

The user has now explicitly authorized opening a PR and merging ? with due diligence. I'll: push the branch, file a GitHub issue, open a PR linking it, run an adversarial review + let CI run, and merge only on green. Let me start with environment checks.

63. user (2026-06-18T01:09:03.526Z)

```
=== gh auth ===
github.com
  ? Logged in to github.com account avidullu (keyring)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_**********************************
  - Token scopes: 'gist', 'read:org', 'repo', 'workflow'
=== remote ===
origin  https://github.com/avidullu/badminton-highlight-indexer.git (fetch)
origin  https://github.com/avidullu/badminton-highlight-indexer.git (push)
=== ci.yml triggers ===
name: CI

on:
  push:
    branches: [master]
  pull_request:

permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
  # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
  # tracked .py, including scripts/ and training/, which the backend-only
  # py_compile smoke sweep does not reach.
  lint:
```

64. assistant (2026-06-18T01:09:07.727Z)

All set ? `gh` authed with `repo` scope, CI runs on `pull_request`. Let me push the branch.

65. user (2026-06-18T01:09:10.778Z)

```
remote:
remote: Create a pull request for 'fix/ablation-litmus-mixed-schema' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/fix/ablation-
litmus-mixed-schema
remote:
branch 'fix/ablation-litmus-mixed-schema' set up to track 'origin/fix/ablation-litmus-mixed-
schema'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      fix/ablation-litmus-mixed-schema -> fix/ablation-litmus-mixed-schema
```

66. assistant (2026-06-18T01:09:15.218Z)

Branch pushed. Now I'll file the GitHub issue describing the bug.

67. user (2026-06-18T01:09:29.014Z)

https://github.com/avidullu/badminton-highlight-indexer/issues/208

68. assistant (2026-06-18T01:09:35.878Z)

Issue [#208](https://github.com/avidullu/badminton-highlight-indexer/issues/208) filed. Now the PR, linking the issue with `Closes #208`.

69. user (2026-06-18T01:09:49.118Z)

https://github.com/avidullu/badminton-highlight-indexer/pull/209

70. assistant (2026-06-18T01:10:18.794Z)

PR [#209](https://github.com/avidullu/badminton-highlight-indexer/pull/209) is open and linked to the issue. Now the due-diligence layer: I'll launch an adversarial multi-lens review of the committed diff (read-only, won't touch the repo) while CI runs on GitHub. Let me kick off both.

71. user (2026-06-18T01:10:52.178Z)

Workflow launched in background. Task ID: w35v6q4cl
Summary: Adversarial multi-lens review of the ablation mixed-schema fix before merge
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_34138e24-10c
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer--claude-worktrees-ablation-schema-fix\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\ablation-fix-due-diligence-wf_34138e24-10c.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer--claude-worktrees-ablation-schema-fix\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\ablation-fix-due-diligence-wf_34138e24-10c.js"} to iterate without resending the script.)
Run ID: wf_34138e24-10c
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer--claude-worktrees-ablation-schema-fix\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\ablation-fix-due-diligence-wf_34138e24-10c.js", resumeFromRunId: "wf_34138e24-10c"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

72. user (2026-06-18T01:11:01.216Z)

=== PR #209 checks ===
Golden fixtures cross-OS (windows-latest) pass 54s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985388
Mypy (lenient baseline) pass 29s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985329
Ruff lint pass 9s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985319
Syntax sweep + import smoke (no GPU stack) pass 22s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985373
Full test suite (offline, mocked) pending 0 https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985339
Golden fixtures cross-OS (macos-latest) pass 21s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985351
Golden fixtures cross-OS (ubuntu-latest) pass 27s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985346
=== PR mergeable state ===
{"mergeStateStatus":"UNSTABLE","mergeable":"MERGEABLE","reviewDecision":"","state":"OPEN"}

73. assistant (2026-06-18T01:11:11.479Z)

CI is already mostly green (ruff, mypy, syntax, golden-fixtures on all 3 OSes pass); only the full offline suite is still running. The review workflow is running in the background. Let me watch CI to completion.

74. user (2026-06-18T01:12:16.650Z)

Golden fixtures cross-OS (ubuntu-latest) pass 27s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985346
Full test suite (offline, mocked) pass 2m13s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985339
Golden fixtures cross-OS (macos-latest) pass 21s https://github.com/avidullu/badminton-

highlight-indexer/actions/runs/27730077676/job/82034985351
Golden fixtures cross-OS (ubuntu-latest)	pass	27s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985346
Golden fixtures cross-OS (windows-latest)	pass	54s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985388
Mypy (lenient baseline) pass	29s	https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985329
Ruff lint	pass	9s	https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985319
Syntax sweep + import smoke (no GPU stack)	pass	22s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985373
Full test suite (offline, mocked)	pass	2m13s	https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985339
Golden fixtures cross-OS (macos-latest) pass	21s	https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985351
Golden fixtures cross-OS (ubuntu-latest)	pass	27s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985346
Golden fixtures cross-OS (windows-latest)	pass	54s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985388
Mypy (lenient baseline) pass	29s	https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985329
Ruff lint	pass	9s	https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985319
Syntax sweep + import smoke (no GPU stack)	pass	22s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985373
=== exit: 0 (0 == all checks passed) ===

75. assistant (2026-06-18T01:12:27.274Z)

All 8 CI checks pass. The last gate is the adversarial review workflow ? I won't merge until it returns a clean verdict. Let me check its status.

76. assistant (2026-06-18T01:12:31.322Z)

I'll wait for the review workflow to finish before merging (it's the last gate).

77. user (2026-06-18T01:17:31.767Z)

<retrieval_status>timeout</retrieval_status>

<task_id>w35v6q4cl</task_id>

<task_type>local_workflow</task_type>

<status>running</status>

78. assistant (2026-06-18T01:17:38.047Z)

Still running. Let me give it more time ? I committed to gating the merge on its verdict.

79. user (2026-06-18T01:21:03.782Z)

<retrieval_status>success</retrieval_status>

<task_id>w35v6q4cl</task_id>

<task_type>local_workflow</task_type>

<status>completed</status>

<output>

```
{
  "summary": "Adversarial multi-lens review of the ablation mixed-schema fix before merge",
  "agentCount": 5,
```

```
  "logs": [
    "reviews: 4/4 returned · findings: 13 · must-fix: 0"
  ],
  "result": {
    "synthesis": {
      "verdict": "merge",
      "must_fixes": [],
      "should_fixes": [
        "Litmus count-growth fragility (pre-existing, not a blocker): _golden_present() gates on
>=6 fusion files (tests/test_ablation.py:422) while test_litmus_reproduces_gate_a asserts
len(videos) == 6 exactly (line 433). Today this is consistent ? I verified exactly 6 fusion + 5
lean files in output/, so load_videos returns exactly 6. But if a 7th FULL-FUSION golden clip is
ever added, the gate stays True and the == 6 assertion turns the litmus red on more data. This
PR actually fixes the opposite drift (lean files no longer inflate the count). A follow-up could
assert len(videos) >= 6 and slice/use a recorded 6, or make the litmus corpus selection
explicit. Out of scope for this bug-fix.",
        "Sibling loaders fusion_compare.load_videos (backend/eval/fusion_compare.py:46-48) and
fusion_audio.load_videos_with_audio still read c['shuttle']/c['person'] unconditionally and
would KeyError on the same mixed corpus (golden_fixtures.py:352 documents this hazard). Pre-
existing and explicitly outside this commit's scope (ablation litmus only); not run against the
mixed corpus today. A follow-up could factor _is_fusion_schema into a shared filter."
      ],
      "nits": [
        "_is_fusion_schema (ablation.py:395) only guards TOP-LEVEL presence of shuttle/person,
not the inner cue columns load_videos reads (e.g. c['shuttle'][fn] at line 420). A file that
passes the predicate but has a cue dict missing an individual column would still KeyError. Not a
concern for the current corpus ? I confirmed all 6 fusion files carry every shuttle/person
column. Theoretical only; could switch inner reads to .get(fn, 0.0) if malformed-but-present cue
dicts ever appear.",
        "_golden_present() (tests/test_ablation.py:415-419) catches (OSError, ValueError) around
json.load(...).get('candidates'); a valid-but-non-dict top-level JSON (e.g. a bare list) would
raise AttributeError on .get(), which is uncaught. Test-only skip-gate code; realistic failure
modes (missing file, malformed JSON ? JSONDecodeError is a ValueError subclass) are handled.
Very low priority; could guard isinstance(obj, dict).",
        "No explicit test for the asymmetric partial case (a candidate with 'person' present but
'shuttle' absent, or vice versa). The all('shuttle' in c and 'person' in c ...) predicate
handles it correctly (returns False -> skipped), but only the symmetric lean case is pinned.
Could add a one-line assert _is_fusion_schema([{'shuttle':{}}]) is False.",
        "Docstring wording imprecision (tests/test_ablation.py:387): says the synthetic mixed
corpus contains 'the 5 newer real clips' but the test body writes only 3 windowing-only files.
The '5' refers to the real owner-box corpus (which I confirmed has 5 lean files); the synthetic
uses 3 as a representative sample. Test logic is correct; cosmetic only."
      ],
      "rationale": "Merge. I verified every load-bearing claim locally against the real golden
substrate, not just the PR description. (1) The diff is correctly scoped to exactly 2 files
(backend/eval/ablation.py +29, tests/test_ablation.py +57) and touches nothing else. (2) The
real corpus output/ has 11 golden files; _is_fusion_schema partitions them into exactly 6 fusion
(adarsh_avi_singles, gbaaddy, kushagra_singles, largetest_doubles, mahadevpura_singles,
testlarge_short) + 5 lean (Badminton_BXH_2, GX010128, GX030094, mahadevpura_1, mahadevpura_2),
and load_videos('output') returns exactly the 6 fusion videos ? matching all four reviews. (3)
The bug is real: pre-fix load_videos (git show HEAD~1) accesses c['shuttle'][fn] unconditionally
with no guard, and _is_fusion_schema did not exist ? so the 5 lean files (which lack a 'shuttle'
key) would raise KeyError: 'shuttle', exactly the n=11 crash. (4) The litmus is preserved:
test_litmus_reproduces_gate_a RAN (not skipped, golden substrate present here) and PASSED,
reproducing the recorded Gate-A numbers within tolerance (?F1?0.074, 6/0 wins, p?0.031,
CI?[0.042,0.104], seg 1.01/1.68). num_videos is computed as len(videos), so == 6 is derived, not
a magic constant. (5) Pre-LGTM gate clean: full suite 848 passed / 5 skipped (the 5 skips are
unrelated owner-box golden-trajectory regression tests), all 29 ablation tests pass, ruff clean,
mypy clean on the changed module. No must-fix exists: all four lenses independently returned
blocks_merge=false, and the only non-trivial finding (the >=6 vs ==6 litmus mismatch) is a pre-
existing latent fragility that does not fire today because the corpus has exactly 6 fusion files
and the litmus is intentionally pinned to n=6 ? acceptable per the gatekeeping criteria. The
should-fixes and nits are all pre-existing or theoretical hardening, none of which break the
litmus, cause a regression, or ship a real bug in this change."
    },
```

```
    "must_fix_count": 0,
    "review_count": 4
  }
}
</output>
```

80. assistant (2026-06-18T01:21:26.852Z)

Both gates are clear: **all 8 CI checks pass** and the **adversarial review verdict is `merge` with 0 must-fixes** (all 4 lenses independently confirmed the bug is real, the litmus reproduces the Gate-A numbers, and the diff is correctly scoped). The two "should-fixes" it raised are both pre-existing/out-of-scope ? and one is the sibling-loader bug I already filed as a follow-up chip.

Merging now (squash, matching the repo's `?(#NNN)` convention):

81. user (2026-06-18T01:21:33.133Z)

```
=== final pre-merge state ===
{"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE","state":"OPEN"}
=== squash-merge + delete remote branch ===
failed to run git: fatal: 'master' is already used by worktree at
'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing'
```

82. assistant (2026-06-18T01:21:38.080Z)

The error is from `gh`'s local post-merge step (it tried to switch to `master`, which is checked out in another worktree). The server-side merge itself likely went through. Let me verify the PR state and master.

83. user (2026-06-18T01:21:43.410Z)

```
=== PR #209 state ===
Unknown JSON field: "merged"
Available fields:
  additions
  assignees
  author
  autoMergeRequest
  baseRefName
  baseRefOid
=== issue #208 state (should auto-close via 'Closes #208') ===
{"closed":true,"state":"CLOSED"}
=== remote branch still there? ===
(empty above == remote branch deleted)
```

84. assistant (2026-06-18T01:21:47.086Z)

Issue #208 auto-closed and the remote branch is gone ? both confirm the merge landed. Let me confirm the merge commit and that `origin/master` advanced.

85. user (2026-06-18T01:21:51.634Z)

```
=== PR state + merge commit ===
{"mergeCommit":{"oid":"4bd9d2dbe540a4903a654fe69acc96077d1e31ec"},"state":"MERGED","title":"fix(
eval): ablation litmus tolerates a mixed-schema golden corpus"}
=== origin/master after fetch ===
 * branch            master     -> FETCH_HEAD
   be39a70..4bd9d2d  master     -> origin/master
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)
4bbd987 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17)
```

86. assistant (2026-06-18T01:21:59.998Z)

PR **MERGED** ? `origin/master` is now at `4bd9d2d ?(#209)`. Now cleanup: remove the worktree (work is merged; the gitignored `output/` copies are disposable) and delete the local feature branch that `gh` couldn't remove.

87. user (2026-06-18T01:22:02.580Z)

```
=== remove the merged worktree (force: drops gitignored output/ copies) ===
error: failed to delete 'C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/ablation-schema-fix': Permission denied
=== delete the local feature branch ===
Deleted branch fix/ablation-litmus-mixed-schema (was 59ae347).
=== worktrees now ===
C:/Users/avidu/Projects/badminton-highlight-indexer
f5f5e52 [docs/golden-regression-fixtures-plan]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed
7e0881a [perf/wasb-fast-frames]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy
5cb7bbf [bhi-execpolicy]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17
8736e2d [run/boxhill17-highlights]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/docs-golden
c33cf1d [docs/golden-videos-tracker]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/docs-rally-quality
877ab1f [docs/rally-quality-research]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/docs-today
ce7eb47 [docs/quality-iterations-realfootage]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hardchop
614325d [feat/w1-hard-cap-fallback]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/local-noai
3ea7ad4 [feat/local-no-ai-segmenter]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/pr-stitch
a6adcc5 [fix/runner-stitch-watermark]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/r2-impl
a450c53 [feat/r2-tolerance-metric]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/r4-endrule
e248d2d [feat/r4-inactivity-end]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier0-eval
9e03e5a [feat/tier0-rally-seg-eval]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing
a3b209a [master]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-netcross
effd920 [feat/w2-netcross-gate]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-safeguard
5ba6955 [feat/w2-never-empty-safeguard]
=== main checkout untouched? ===
docs/golden-regression-fixtures-plan
```

88. assistant (2026-06-18T01:22:14.361Z)

The local branch is deleted and git no longer lists the worktree (its metadata was removed). Only the physical directory deletion hit a Windows "Permission denied" (a lingering file lock). Let me try to clear the leftover directory.

89. user (2026-06-18T01:22:17.548Z)

```
CLEAN: worktree directory removed
=== confirm not in worktree list ===
0
0 (absent from worktree list ? good)
```

90. assistant (2026-06-18T01:22:23.596Z)

Clean. Let me update the live checkpoint to reflect the merge.

91. user (2026-06-18T01:22:42.573Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

92. assistant (2026-06-18T01:22:58.298Z)

All done. The fix is reported, reviewed, and merged with full due diligence.

What landed

Step	Result
Issue	[#208](https://github.com/avidullu/badminton-highlight-indexer/issues/208) ? bug report (root cause, repro, scope) ? **auto-closed** by the merge
PR	[#209](https://github.com/avidullu/badminton-highlight-indexer/pull/209) `fix(eval): ablation litmus tolerates a mixed-schema golden corpus` ? **MERGED** (squash)
master	advanced `be39a70 ? 4bd9d2d ?(#209)`

Due diligence (three independent gates, all green)

1. **Local, mirroring the owner box** (golden corpus in `output/` so the litmus ran for real): full suite **848 passed / 5 skipped**, `ruff check .` clean, `mypy backend training` clean (104 files), coverage **80.66%** (floor 70), `ablation.py` at 99%.
2. **GitHub CI** ? all **8 checks pass**: Ruff, Mypy, full offline suite, syntax/import smoke, and golden-fixtures cross-OS (ubuntu/macos/windows).
3. **Adversarial 4-lens review workflow** (correctness · edge-cases · test-quality · convention/scope ? gatekeeper synthesis): **verdict = merge, 0 must-fixes**. The reviewers independently re-verified the bug against the pre-fix code, confirmed the litmus reproduces the recorded Gate-A numbers (?F1?0.074, 6/0, p?0.031), and confirmed the diff is scoped to exactly 2 files.

Cleanup

Remote and local `fix/ablation-litmus-mixed-schema` branches deleted; worktree removed; the live checkpoint memory is updated.

Two follow-ups the review surfaced (both correctly left out of this focused PR)

- **Sibling loaders** `fusion_compare.load_videos` / `fusion_audio.load_videos_with_audio` have the identical unconditional `c["shuttle"]` access ? latent (no test exercises the real corpus) but would crash the same way. Already tracked as a follow-up chip (`task_8dd4efde`).
- **Litmus count fragility** (pre-existing, non-blocking): `_golden_present()` gates on `>=6` fusion files while the litmus asserts `len == 6`. Fine today (exactly 6 fusion files) ? and this PR fixed the *opposite* drift ? but a 7th fusion clip would turn it red. Worth a one-line `>= 6` relaxation when the golden corpus next grows.

Want me to spin up the sibling-loader fix now, or leave it on the chip for later?

93. user (2026-06-18T01:24:23.308Z)

I approved a chip but if there are *any* other fixes then plesae go ahead and send and merge their PRs

94. assistant (2026-06-18T01:25:48.950Z)

The chip is being handled by its own spawned session ? so I'll steer clear of the sibling loaders to avoid colliding with it, and instead hunt for *other* real fixes to ship. Let me first sync to latest master (the chip session may have merged) and map the code surface.

95. user (2026-06-18T01:25:54.456Z)

=== origin/master HEAD now ===

```
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)
4bbd987 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17)
=== any open PRs (chip session may have one in flight)? ===
210     fix(eval): fusion loaders skip windowing-only files on a mixed golden corpus
fix/fusion-loaders-mixed-schema OPEN    2026-06-18T01:15:28Z
207     docs: capture external review inputs (research, pick-up-later)  claude/indexer-doc-
review-XVPTh OPEN    2026-06-17T18:10:06Z
204     [DRAFT · product-go needed] R1 milestone ? flip windowing defaults to cap=6 + R4
feat/r1-milestone-config-flip   DRAFT   2026-06-17T12:04:01Z
=== backend tree ===
backend
backend/annotations
backend/config
backend/eval
backend/eval/audio
backend/features
backend/infrastructure
backend/pipeline/audio
backend/pipeline/detectors
backend/pipeline/personalization
backend/pipeline/segmenters
backend/pipeline/stitcher
backend/pipeline/validators
backend/providers
backend/reporting
backend/sports
backend/tools
backend/utils
scripts
training
training/gen0
```

96. assistant (2026-06-18T01:26:39.427Z)

PR **#210** is already open ? that's the chip session's sibling-loader fix, actively in flight. I'll leave it to that session and **not** touch the fusion loaders, to avoid racing on the shared checkout. The other open PRs (#207 docs, #204 product-gated draft) aren't mine to merge.

Now to hunt for *other* real fixes. Let me set up a clean `origin/master` worktree as the read base so any finding maps cleanly to a master-based PR.

97. user (2026-06-18T01:26:42.351Z)

```
Preparing worktree (detached HEAD 4bd9d2d)
HEAD is now at 4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
=== source file count on master ===
108
```

98. assistant (2026-06-18T01:27:02.520Z)

Clean `origin/master` base ready (108 source files). Now I'll run a high-precision adversarial bug-hunt: parallel finders across the codebase (excluding the just-fixed `ablation.py` and the in-flight fusion loaders) ? diverse refutation of each candidate ? synthesis into shippable vs owner-gated. Token cost isn't a constraint here (ultracode), but I'm tuning for precision ? only real, reachable defects.

99. user (2026-06-18T01:27:48.587Z)

```
Workflow launched in background. Task ID: wip5pkgi0
Summary: Adversarial hunt for real, reachable defects to ship as individual PRs
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_b7f012f8-5c3
```

Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bug-hunt-shippable-fixes-wf_b7f012f8-5c3.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bug-hunt-shippable-fixes-wf_b7f012f8-5c3.js"} to iterate without resending the script.)
Run ID: wf_b7f012f8-5c3
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bug-hunt-shippable-fixes-wf_b7f012f8-5c3.js", resumeFromRunId: "wf_b7f012f8-5c3"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

100. assistant (2026-06-18T01:28:02.630Z)

The hunt is running across the codebase. Let me wait for its results before acting.

101. user (2026-06-18T01:32:03.884Z)

<retrieval_status>success</retrieval_status>

<task_id>wip5pkgi0</task_id>

<task_type>local_workflow</task_type>

<status>completed</status>

<output>

```
{
  "summary": "Adversarial hunt for real, reachable defects to ship as individual PRs",
  "agentCount": 6,
  "logs": [
    "Find: 0 candidate(s) across 6 areas"
  ],
  "result": {
    "shippable": [],
    "needs_owner": [],
    "rejected": [],
    "note": "No candidate defects surfaced."
  }
}
```

</output>

102. assistant (2026-06-18T01:32:35.683Z)

Zero candidates across all 6 areas ? and these finders did real work (763k tokens, 201 tool reads), and were explicitly told an empty list is a respected answer and not to manufacture findings. For a codebase with 848 tests and multiple hardening sweeps, a clean result is plausible.

But a single precision-tuned pass can suppress subtler issues, and you asked me to be thorough. Let me run one complementary pass decomposed by **bug-class** (cross-cutting all files, the proven "unguarded persisted-data" vein included) with a completeness critic, before I conclude there's nothing else to ship.

103. user (2026-06-18T01:33:12.198Z)

Workflow launched in background. Task ID: w9knuyga2
Summary: Second-pass defect hunt decomposed by bug-class, with a completeness critic
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_49c1018a-e67

Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bug-hunt-pass2-by-class-wf_49c1018a-e67.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bug-hunt-pass2-by-class-wf_49c1018a-e67.js"} to iterate without resending the script.)
Run ID: wf_49c1018a-e67
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bug-hunt-pass2-by-class-wf_49c1018a-e67.js", resumeFromRunId: "wf_49c1018a-e67"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

104. assistant (2026-06-18T01:33:15.397Z)

Second pass running. Let me wait for it.

105. user (2026-06-18T01:43:15.954Z)

<retrieval_status>timeout</retrieval_status>

<task_id>w9knuyga2</task_id>

<task_type>local_workflow</task_type>

<status>running</status>

106. assistant (2026-06-18T01:43:20.817Z)

Still running (it found candidates and is verifying them, or finders are still reading). Let me continue waiting.

107. user (2026-06-18T01:44:08.556Z)

<retrieval_status>success</retrieval_status>

<task_id>w9knuyga2</task_id>

<task_type>local_workflow</task_type>

<status>completed</status>

<output>

```
{
  "summary": "Second-pass defect hunt decomposed by bug-class, with a completeness critic",
  "agentCount": 15,
  "logs": [
    "Find(pass2): 4 candidate(s)",
    "Verify(pass2): 3/4 survived"
  ],
  "result": {
    "synthesis": {
      "shippable": [
        {
          "title": "Make v3/v4 migration ALTERs idempotent so an interrupted one-time upgrade can't permanently brick the DB",
          "file": "backend/infrastructure/database.py",
          "line": "156-159 (v3) and 168-178 (v4)",
          "bug": "The v3 block (`ALTER TABLE jobs ADD COLUMN policy` / `next_poll_at`) and v4 block (`ALTER TABLE videos ADD COLUMN user_id` / `ALTER TABLE candidate_segments ADD COLUMN user_id`) use bare, non-idempotent `ALTER TABLE ... ADD COLUMN` and only bump `PRAGMA
```

</output>

```
user_version` at the END of the block. In CPython sqlite3 each DDL ALTER auto-commits independently of the surrounding `with conn:` scope in _connect, so if the process is interrupted (SIGKILL/power-loss, disk I/O error, or a later-statement OperationalError such as busy_timeout exhaustion) AFTER an ALTER commits but BEFORE the version bump, the column persists while user_version stays at the pre-block value. On the next open, _init_db re-runs the same bare ALTER and raises `sqlite3.OperationalError: duplicate column name: ...`. Since _init_db runs in __init__, every subsequent Database(db_path) construction throws and the app can never open that DB again. v1/v2/v5 blocks are deliberately re-runnable (CREATE ... IF NOT EXISTS); only v3/v4 break the crash-safe-ladder invariant.",
          "fix_summary": "Guard each bare ALTER with a column-existence check before running it, e.g. a small helper `def _has_col(cur, table, col): return col in {r[1] for r in cur.execute(f'PRAGMA table_info({table})')}` and only ALTER when the column is absent (equivalently, catch and ignore the 'duplicate column name' OperationalError per-ALTER). This makes an interrupted block safely re-applicable, matching the already-idempotent v1/v2/v5 blocks. The IF-NOT-EXISTS index statements in those blocks are already fine and unchanged. No schema/behavior change on the happy path.",
          "risk": "low",
          "test_plan": "Add a test in tests/test_database.py simulating a partial migration: build a DB at user_version=3 whose `videos` table already has a `user_id` column (and a user_version=2 DB whose `jobs` table already has `policy`), then construct Database(db_path) and assert it opens cleanly and reaches user_version=5 (instead of raising 'duplicate column name'). Keep the existing test_init_db_idempotent_and_upgrades_old_db and test_migration_v3_db_upgrades_to_v5_preserving_rows green. Run full suite: python run_all_tests.py.",
          "owner_gated": false
        },
        {
          "title": "Unwrap the segmenter Result before len() in run_phase3_eval.py (motion is the default + documented segmenter)",
          "file": "scripts/run_phase3_eval.py",
          "line": "145-146",
          "bug": "Line 145 does `segs = segmenter.process_video(...)` and line 146 does `len(segs)`. Per the base.py contract, process_video returns a Result (Success|Failure). The motion segmenter ? which is the argparse default (`--segmenter` default=\"motion\", line 53) and the documented 'Free CPU baseline' example (docstring lines 10-13) ? returns `Success(value=segments_data)` (motion.py:224) or `Failure(...)` (motion.py:31). Success/Failure are plain dataclasses with no __len__, so `len(segs)` raises `TypeError: object of type 'Success' has no len()`, crashing the documented first-run command before any scoring/report. Other segmenters (gemini/yolo_hybrid/trajectory_hybrid) return bare lists and mask the bug. The driver never unwraps, unlike backend/main.py.",
          "fix_summary": "Mirror the backend/main.py pattern: after line 145, `result = segmenter.process_video(...)`; `if isinstance(result, Failure): print(f'        -> FAILED: {result.message}'); continue`; `segs = result.value if hasattr(result, 'value') else result`; then `len(segs)`. Import Failure from backend.results.",
          "risk": "low",
          "test_plan": "This is a dev/eval CLI (no GPU/cv2 in this container, so motion can't run E2E here). Add a focused unit check or run on the owner's WSL/GPU machine: `python run_phase3_eval.py --manifest <manifest> --segmenter motion --only <fresh_vid>` and confirm it prints the segment count and proceeds to scoring instead of TypeError. Confirm the bare-list segmenters (yolo_hybrid) still work unchanged. Run python run_all_tests.py for regressions.",
          "owner_gated": false
        },
        {
          "title": "Unwrap the segmenter Result in run_process_and_stitch.py (len() + empty-guard both wrong under motion)",
          "file": "scripts/run_process_and_stitch.py",
          "line": "81-86",
          "bug": "Line 81 `segments = segmenter.process_video(video_id, video_path)` is used without unwrapping. When `indexing.default_segmenter` is 'motion' (a selectable, registered segmenter), process_video returns a Success/Failure dataclass. Line 82 `len(segments)` then raises `TypeError: object of type 'Success' has no len()`, and line 84 `if not segments:` is always False for any dataclass instance, so the 'No play segments detected' guard is dead code (an empty Success or a Failure would silently fall through to stitching). Latent for the shipped default (gemini returns a bare list) but a real defect once motion is selected. Unlike backend/main.py, the script never does an isinstance(Failure)/.value unwrap.",
```

```
          "fix_summary": "Unwrap before use, mirroring backend/main.py: `result =
segmenter.process_video(video_id, video_path)`; `if isinstance(result, Failure): print(f'[FAIL]
{result.message}'); return`; `segments = result.value if hasattr(result, 'value') else result`.
Then lines 82 (len) and 84 (truthiness empty-guard) operate on the actual list. Import Failure
from backend.results.",
          "risk": "low",
          "test_plan": "Dev/manual-QA entrypoint with hardcoded paths; not on the production API
path and can't run E2E here (no cv2). On the owner's machine: set
indexing.default_segmenter='motion', run the script on a non-cached video, confirm it reports
the segment count (or [FAIL]/empty-guard) cleanly instead of TypeError; confirm
gemini/yolo_hybrid paths still work. Run python run_all_tests.py.",
          "owner_gated": false
        }
      ],
      "rejected": [],
      "notes": "All three survivors confirmed real against the live working-tree code (not just
the hunt-base worktree): Success/Failure in backend/results.py have no __len__ (verified),
motion.py returns Success/Failure at lines 31/224, and run_phase3_eval defaults --segmenter to
'motion' (line 53) which is also its documented baseline command. None of the three fixes change
product behavior, defaults, scoring, or licensing, so owner_gated=false for all. Recommended
landing order: the two scripts/ Result-unwrap fixes are tiny, self-contained, and share the
identical backend/main.py pattern ? could ship as one PR or two. The DB migration-ladder fix is
the higher-value one (permanent unrecoverable DB lockout) but touches the production migration
ladder, so it warrants its own PR with the new interrupted-migration test. Honest caveat carried
from both verifiers: the DB-brick trigger window is narrow in the wild today (no legacy sub-v4
on-disk DBs exist yet per repo state; 'no platform implementation has started'), so it's medium
severity / low current frequency ? but the fix is pure crash-safety hardening that simply brings
v3/v4 in line with the already-idempotent v1/v2/v5 blocks. Environment limit: this dev container
has no GPU/cv2/torch, so neither script nor the motion segmenter can be exercised E2E here; the
motion-path crashes were confirmed by code+contract inspection, and E2E validation of the script
fixes should be done on the owner's WSL/GPU machine. The pure-logic DB migration test CAN and
should run in CI here."
    },
    "confirmed_count": 3,
    "candidate_count": 4
  }
}
</output>
```


108. assistant (2026-06-18T01:44:51.285Z)

The second pass found **3 confirmed real defects** (4 candidates, 3 survived diverse refutation), all `owner_gated=false`. Before trusting any of it, I'll verify each against the actual master code myself (my pre-LGTM gate ? reproduce, don't trust the description). Let me read the relevant code.

109. user (2026-06-18T01:44:52.094Z)

```
100                             video_id TEXT NOT NULL,
101                             detector TEXT NOT NULL,
102                             chunk_index INTEGER,
103                             start_time REAL NOT NULL,
104                             end_time REAL NOT NULL,
105                             duration REAL NOT NULL,
106                             confidence REAL,
107                             stats TEXT,
108                             detection_params TEXT,
109                             confirmed_segment_id INTEGER,
110                             human_label TEXT,
111                             notes TEXT,
112                             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
113                             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
114                             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
115                         )
```

```
                    """)
                    cursor.execute("""
                        CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                            ON candidate_segments(video_id, detector)
                    """)
                    cursor.execute("PRAGMA user_version = 1")

                if version < 2:
                    # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
                    # /api/process request. `status` is the EXECUTION state of the job
                    # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
                    # that failed validation) lives inside `result` JSON as
result["status"].
                    # `result` is the full sync-era response dict (incl. report paths), so
the
                    # observability report is the job's artifact, per the plan.
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS jobs (
                            id TEXT PRIMARY KEY,
                            video_path TEXT NOT NULL,
                            video_id TEXT,
                            segmenter TEXT,
                            player_pool TEXT,
                            max_frames INTEGER,
                            status TEXT NOT NULL DEFAULT 'queued',
                            progress TEXT,
                            error TEXT,
                            result TEXT,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                            started_at TIMESTAMP,
                            finished_at TIMESTAMP
                        )
                    """)
                    cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
                    cursor.execute("PRAGMA user_version = 2")

                if version < 3:
                    # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
                    # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
                    # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
                    # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
                    # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
                    cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
                    cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
                    cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
                    cursor.execute("PRAGMA user_version = 3")

                if version < 4:
                    # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
NULLABLE
                    # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
                    # path can scope rows to an owner. **NULL = local install = byte-
identical to
                    # today** ? every existing row stays NULL and every existing query
(which never
                    # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive.
```

```
167                     # Additive ONLY: no served-behavior change rides on this slice.
168                     cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
169                     cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
170                     # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
path free
171                     # of index churn while making the eventual per-user lookups cheap.
172                     cursor.execute(
173                         "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
174                         "WHERE user_id IS NOT NULL")
175                     cursor.execute(
176                         "CREATE INDEX IF NOT EXISTS idx_candidates_user "
177                         "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
178                     cursor.execute("PRAGMA user_version = 4")
179
180                 if version < 5:
181                     # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
(user_id,
182                     # version): the resolved per-user `fusion_config` overlay + an INLINE
183                     # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
184                     # and provenance that earned it. `is_active` pins the one row the
serving bridge
185                     # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
This slice
186                     # only persists/loads it ? NOTHING reads it on the served path yet.
187                     #
188                     #   user_id              ? owner of the model (NULL allowed = the local
install)
189                     #   version              ? monotonic per-user model version
190                     #   baseline_version     ? the shared baseline this was fit against
(upgrade switch)
191                     #   feature_schema_hash  ? fingerprint of the feature set; MINOR-vs-
MAJOR re-fit gate
192                     #   fusion_config        ? JSON per-user FusionConfig overlay (cue_flags
/ thresholds)
193                     #   classifier_json      ? JSON LogisticRegression.to_dict() (None =
config-only model)
194                     #   promotion_evidence   ? JSON per_user_gate PromotionDecision (why it
was promoted)
195                     #   n_videos_at_fit      ? held-out-user corpus size when fit (min_n
trust floor input)
196                     #   is_active            ? the one resolved row for this user (partial-
unique below)
197                     cursor.execute("""
198                         CREATE TABLE IF NOT EXISTS user_models (
199                             id INTEGER PRIMARY KEY AUTOINCREMENT,
200                             user_id TEXT,
201                             version INTEGER NOT NULL DEFAULT 1,
202                             baseline_version TEXT,
203                             feature_schema_hash TEXT,
204                             fusion_config TEXT,
205                             classifier_json TEXT,
206                             promotion_evidence TEXT,
207                             n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
208                             is_active INTEGER NOT NULL DEFAULT 0,
209                             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

110. user (2026-06-18T01:44:53.016Z)

```
1       """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3       This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5       The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
```

```
6       (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8       We use simple dataclasses so callers can use isinstance checks or structural pattern
matching.
9       This is intentionally small and incremental ? we start by applying it to the segmenter
10      and provider layers (the biggest sources of silent [] returns today).
11
12      The types are generic where it makes sense. Callers can still access .value on Success
13      or .message on Failure.
14
15      This module has no dependencies on the rest of the app.
16      """
17
18      from __future__ import annotations
19
20      from dataclasses import dataclass, field
21      from typing import Generic, TypeVar, List, Optional, Any, Union
22
23      T = TypeVar("T")
24
25
26      @dataclass
27      class Success(Generic[T]):
28          """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30          value: T
31          warnings: List[str] = field(default_factory=list)
32
33          def is_success(self) -> bool:
34              return True
35
36          def unwrap(self) -> T:
37              """Return the value. Only safe to call if you already know it's a Success."""
38              return self.value
39
40
41      @dataclass
42      class Failure:
43          """Represents a failure with a human-readable message and optional details."""
44
45          message: str
46          details: Optional[dict[str, Any]] = None
47          is_recoverable: bool = False  # e.g. missing API key (recoverable) vs internal crash
48
49          def is_success(self) -> bool:
50              return False
51
52          def unwrap(self) -> Any:
53              """Raise to make misuse obvious in code that assumes success."""
54              raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57      # Convenient alias for use in type hints: Result[list[dict], Failure]
58      # Result[ValueType] means Success[ValueType] | Failure
59      # (the Failure "error" type is fixed for this contract)
60      Result = Union[Success[T], Failure]
61
```

111. user (2026-06-18T01:44:53.949Z)

```
130                 skipped.append((vid, "no GT csv"))
131                 print(f"[skip] {vid}: GT csv missing ({csv_path})")
132                 continue
133
134             video_id = get_file_md5(video_path)
```

```
135
136             # Segment unless score-only or already present (and not --reprocess).
137             if not args.score_only:
138                 already = db.query_play_segments(video_id, {}) or
db.query_candidate_segments(video_id)
139                 if already and not args.reprocess:
140                     print(f"[have] {vid}: detections already in DB (use --reprocess to
redo)")
141                 else:
142                     print(f"[proc] {vid}: segmenting with {args.segmenter}...")
143                     db.add_video(video_id, video_path, "processed", [])
144                     t0 = time.time()
145                     segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
146                     print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")
147
148             gt = load_annotations(csv_path)
149             report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
detector=args.detector)
150             print(format_report_text(report, show_failures=args.show_failures))
151             print()
152             d = report_to_dict(report)
153             d["collector_video_id"] = vid
154             per_video.append(d)
155
156         agg = batch.aggregate(per_video, stages)
157         print(batch.format_aggregate(agg))
158         if skipped:
159             print(f"\nSkipped {len(skipped)}: " + ", ".join(f"{v} ({why})" for v, why in
skipped))
160
161         if args.json_out:
162             with open(args.json_out, "w") as f:
163                 json.dump({"segmenter": args.segmenter, "iou": args.iou, "stages": stages,
164                            "aggregate": agg, "per_video": per_video,
```

112. user (2026-06-18T01:44:54.805Z)

```
70              print("Bypassing OpenCV validations for performance on massive files...",
flush=True)
71              db.add_video(video_id, video_path, "processed", [])
72
73              print("Running segmenter & indexing...", flush=True)
74              seg_name = cfg.indexing.default_segmenter
75              segmenter_cls = segmenter_registry.get(seg_name)
76              if not segmenter_cls:
77                  print(f"[FAIL] Segmenter '{seg_name}' not found.")
78                  return
79              print(f"Using segmenter: {seg_name}", flush=True)
80              segmenter = segmenter_cls(db, cfg)
81              segments = segmenter.process_video(video_id, video_path)
82              print(f"Successfully indexed {len(segments)} play segments.", flush=True)
83
84          if not segments:
85              print("No play segments detected. Cannot compile highlights.")
86              return
87
88
89          end_padding = cfg.stitching.end_padding
90          begin_padding = cfg.stitching.begin_padding
91          min_shot_count = cfg.stitching.min_shot_count
92
93          if cache_hit:
94              print("\n--- Stitching Customizations ---")
95              try:
```

```
96                  bpad_input = input(f"Enter begin_padding in seconds (default
{begin_padding}): ").strip()
97                  if bpad_input:
98                      begin_padding = float(bpad_input)
99
```

113. assistant (2026-06-18T01:45:31.348Z)

Verified the two core facts: `Success`/`Failure` (backend/results.py) have **no `__len__`**, and the v3/v4 migration blocks use **bare non-idempotent `ALTER`** while v1/v2/v5 are `IF NOT EXISTS`. Let me confirm the segmenter-return facts and the canonical unwrap pattern before deciding how to ship.

114. user (2026-06-18T01:45:36.637Z)

```
=== motion.process_video return sites ===
20:    def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None,
max_frames: int = None) -> List[Dict[str, Any]]:
31:            return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
224:        return Success(value=segments_data)

=== run_phase3_eval argparse --segmenter default ===
22:import os
23:import sys
24:import json
25:import time
26:import argparse
27:import logging
29:from dotenv import load_dotenv
32:from backend.infrastructure.database import Database
33:from backend.config import load_config
34:from backend.utils.hashing import compute_video_id as get_file_md5  # canonical video_id
36:from backend.pipeline.segmenters import segmenter_registry
37:from backend.eval.annotations import load_annotations
38:from backend.eval.harness import evaluate, format_report_text, report_to_dict
39:from backend.eval import batch
53:    p.add_argument("--segmenter", default="motion",

=== backend/main.py unwrap pattern (isinstance Failure / .value) ===
35:from backend.results import Failure  # standardized error/result contract
192:            result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
195:            # state (same destroy-after-confirm contract as the Failure branch), then let
201:        if isinstance(result, Failure):
213:        segments = result.value if hasattr(result, "value") else result
220:            "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
347:def process_video(req: ProcessRequest):
482:            received += len(chunk)
607:    if len(kept) < len(matched_segments):
609:                    f"{len(matched_segments) - len(kept)}/{len(matched_segments)} segments
below the floor.")
612:                            detail=f"All {len(matched_segments)} matching segments fall
below "
683:        avg_motion = sum(motions) / len(motions)
689:        print(f"  Sample Frame Count: {len(motions)}")
835:            result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
838:            if isinstance(result, Failure):
843:                segments = result.value if hasattr(result, "value") else result
844:                print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments
inside database: {db_path}")

=== how do OTHER segmenters return? (gemini/yolo_hybrid/trajectory_hybrid) ===
```

```
backend/pipeline/segmenters/base.py:24:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
backend/pipeline/segmenters/fusion_hybrid.py:143:                            idx_cfg: Dict[str,
Any]) -> List[int]:
backend/pipeline/segmenters/gemini.py:35:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend/pipeline/segmenters/motion.py:20:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend/pipeline/segmenters/motion.py:31:            return Failure(message=f"Could not open
video: {video_path}", is_recoverable=False)
backend/pipeline/segmenters/motion.py:224:        return Success(value=segments_data)
backend/pipeline/segmenters/trajectory_hybrid.py:98:                            idx_cfg:
Dict[str, Any]) -> List[int]:
backend/pipeline/segmenters/trajectory_hybrid.py:106:    def process_video(self, video_id: str,
video_path: str,  # type: ignore[override]
backend/pipeline/segmenters/trajectory_hybrid.py:108:                        max_frames:
Optional[int] = None) -> List[Dict[str, Any]]:
backend/pipeline/segmenters/trajectory_hybrid.py:277:            def
process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
backend/pipeline/segmenters/yolo_hybrid.py:57:    def process_video(self, video_id: str,
video_path: str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend/pipeline/segmenters/yolo_hybrid.py:279:        def process_candidate_window(wi: int,
window: Dict[str, Any]) -> List[dict]:
```

115. assistant (2026-06-18T01:46:08.882Z)

All three confirmed real on master:
- **motion** returns `Success`/`Failure` (motion.py:31,224); `run_phase3_eval.py` defaults `--segmenter` to **`motion`** (line 53) ? the documented default command hits `len(Success)` ? `TypeError`.
- **`backend/main.py`** has the canonical unwrap (`isinstance(result, Failure)` ? handle; else `result.value if hasattr(result,"value") else result`) at lines 201-213 and 838-843 ? the scripts simply never adopted it.
- v3/v4 bare `ALTER` vs idempotent v1/v2/v5 confirmed.

I'll ship **two PRs** (separate subsystems, no stacking): (1) the DB migration idempotency fix ? highest value, fully testable in CI here; (2) the two script unwraps grouped (same bug-class). Let me start with PR #1. First, the migration function structure:

116. user (2026-06-18T01:46:10.341Z)

```
1       import sqlite3
2       import json
3       import logging
4       from contextlib import contextmanager
5       from typing import List, Dict, Any, Optional, Tuple
6
7       logger = logging.getLogger(__name__)
8
9       class Database:
10          def __init__(self, db_path: str):
11              self.db_path = db_path
12              self._init_db()
13
14          @contextmanager
15          def _connect(self):
16              """Transaction scope that also CLOSES the connection.
17
18              ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19              never closes, so every call leaked its connection to GC (lingering file
20              handles on the WAL sidecars under Windows). Internal methods use this.
21              """
22              conn = self._get_connection()
23              try:
24                  with conn:
```

```
                    yield conn
            finally:
                conn.close()

        def _get_connection(self):
            # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
            # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
            conn = sqlite3.connect(self.db_path, timeout=30.0)
            conn.row_factory = sqlite3.Row
            # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
            conn.execute("PRAGMA foreign_keys = ON")
            # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
            conn.execute("PRAGMA busy_timeout = 30000")
            try:
                conn.execute("PRAGMA journal_mode = WAL")
            except sqlite3.OperationalError:
                pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
            return conn

        def _init_db(self):
            """Initializes tables for videos, play_segments, and events."""
            with self._connect() as conn:
                cursor = conn.cursor()

                # Videos Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS videos (
                        id TEXT PRIMARY KEY,
                        filepath TEXT NOT NULL,
                        processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                        status TEXT NOT NULL,
                        validation_results TEXT
                    )
                """)

                # Play Segments Table (Extensible metadata JSON)
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS play_segments (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        video_id TEXT NOT NULL,
                        start_time REAL NOT NULL,
                        end_time REAL NOT NULL,
                        duration REAL NOT NULL,
                        server TEXT,
                        receiver TEXT,
                        metadata TEXT,
                        FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
                    )
                """)

                # Events Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS events (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        segment_id INTEGER NOT NULL,
                        timestamp REAL NOT NULL,
                        type TEXT NOT NULL,
                        player TEXT,
                        details TEXT,
                        FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
                    )
                """)

```

```
88                  # Versioned migrations live here. PRAGMA user_version is the schema
89                  # contract ? bump it for every change and add an ordered `if version < N`
90                  # block below. Tests assert the expected version, so accidental drift fails
CI.
```

117. user (2026-06-18T01:46:15.599Z)

```
88                  # Versioned migrations live here. PRAGMA user_version is the schema
89                  # contract ? bump it for every change and add an ordered `if version < N`
90                  # block below. Tests assert the expected version, so accidental drift fails
CI.
91                  version = cursor.execute("PRAGMA user_version").fetchone()[0]
92
93                  if version < 1:
94                      # v1: raw candidate detections (pre-confirmation), detector-agnostic.
95                      # Common fields are columns; detector-specific metrics go in `stats`
JSON,
96                      # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
97                      cursor.execute("""
98                          CREATE TABLE IF NOT EXISTS candidate_segments (
99                              id INTEGER PRIMARY KEY AUTOINCREMENT,
```

118. assistant (2026-06-18T01:47:11.093Z)

Everything checks out. The cleanest idempotent fix keeps the `ALTER` statements as literals (no f-string SQL) and tolerates re-application. Let me set up PR #1's worktree from a fresh `origin/master` and read the existing DB tests to match conventions.

119. user (2026-06-18T01:47:17.324Z)

```
=== origin/master HEAD (did chip #210 merge yet?) ===
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
{"state":"OPEN","title":"fix(eval): fusion loaders skip windowing-only files on a mixed golden
corpus"}
=== create PR#1 worktree ===
branch 'fix/db-migration-idempotent-alters' set up to track 'origin/master'.
HEAD is now at 4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
fix/db-migration-idempotent-alters
```

120. user (2026-06-18T01:47:25.325Z)

```
=== migration-related tests ===
4:def test_database_init(tmp_path):
16:def test_database_video(tmp_path):
31:def test_database_play_segment(tmp_path):
60:    "user_id",  # v4: personalization groundwork (nullable; NULL = local install)
64:def test_candidate_schema_contract(tmp_path):
80:def test_schema_version_is_current(tmp_path):
84:    version = conn.execute("PRAGMA user_version").fetchone()[0]
86:    # v5 = per-user model store (PERSONALIZATION_PLAN.md §2). Bump with each migration.
90:def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
106:    version = conn.execute("PRAGMA user_version").fetchone()[0]
114:def test_candidate_round_trip(tmp_path):
141:def test_candidate_link_and_unlabeled_filter(tmp_path):
160:def test_candidate_cascade_on_video_delete(tmp_path):
169:# --- v4: personalization groundwork (nullable user_id; NULL = local = unchanged) -----
178:def test_v4_user_id_columns_present_and_nullable(tmp_path):
181:    assert "user_id" in _table_cols(db_path, "videos")
182:    assert "user_id" in _table_cols(db_path, "candidate_segments")
188:        assert by_name["user_id"][3] == 0, f"{table}.user_id must be nullable"
192:def test_v4_existing_writers_leave_user_id_null(tmp_path):
193:    """NULL user_id back-compat: add_video / add_candidate_segment never set user_id, so
```

```
every
201:    vu = conn.execute("SELECT user_id FROM videos WHERE id = 'vid1'").fetchone()[0]
202:    cu = conn.execute("SELECT user_id FROM candidate_segments WHERE video_id =
'vid1'").fetchone()[0]
206:    # Existing readers are unaffected (no user_id filter in the query path).
210:def test_migration_v3_db_upgrades_to_v5_preserving_rows(tmp_path):
212:    columns/tables appear, user_id back-fills to NULL, and prior rows survive the ALTERs."""
222:    # Recreate v3-shaped tables WITHOUT user_id by rebuilding (simplest: just rewind version
224:    conn.execute("ALTER TABLE videos DROP COLUMN user_id")
225:    conn.execute("ALTER TABLE candidate_segments DROP COLUMN user_id")
226:    conn.execute("PRAGMA user_version = 3")
229:    assert "user_id" not in _table_cols(db_path, "videos")
231:    # Re-open: the migration ladder must carry the v3 DB up to v5 without data loss.
234:    assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
238:    assert "user_id" in _table_cols(db_path, "videos")
239:    assert "user_id" in _table_cols(db_path, "candidate_segments")
249:def test_user_models_schema_contract(tmp_path):
254:        "id", "user_id", "version", "baseline_version", "feature_schema_hash",
260:def test_upsert_and_get_active_user_model_round_trip(tmp_path):
274:    assert m["user_id"] == "user-1"
285:def test_get_active_user_model_none_when_absent_or_inactive(tmp_path):
294:def test_upsert_is_idempotent_on_user_version(tmp_path):
=== total lines ===
351 tests/test_database.py
```

121. assistant (2026-06-18T01:47:23.530Z)

Chip PR #210 still open (its session's call). My PR#1 worktree is on `origin/master`. Let me read the existing migration tests to match conventions.

122. assistant (2026-06-18T01:47:33.889Z)

There's a `test_migration_v3_db_upgrades_to_v5_preserving_rows` for the happy re-migration. Mine tests the *partial/interrupted* path (columns present, version behind). Let me read it + the helper to match style.

123. user (2026-06-18T01:47:35.817Z)

```
1       import sqlite3
2       from backend.infrastructure.database import Database
3
4       def test_database_init(tmp_path):
5           db_path = tmp_path / "test.db"
6           Database(str(db_path))  # instantiation creates the schema
7
8           conn = sqlite3.connect(str(db_path))
9           cursor = conn.cursor()
10          cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
11          tables = [r[0] for r in cursor.fetchall()]
12          assert "videos" in tables
13          assert "play_segments" in tables
14          conn.close()
```

124. user (2026-06-18T01:47:36.443Z)

```
169     # --- v4: personalization groundwork (nullable user_id; NULL = local = unchanged) -----
170
171     def _table_cols(db_path, table):
172         conn = sqlite3.connect(str(db_path))
173         cols = {c[1] for c in conn.execute(f"PRAGMA table_info({table})").fetchall()}
174         conn.close()
175         return cols
176
177
```

```
    def test_v4_user_id_columns_present_and_nullable(tmp_path):
        db_path = tmp_path / "test.db"
        Database(str(db_path))
        assert "user_id" in _table_cols(db_path, "videos")
        assert "user_id" in _table_cols(db_path, "candidate_segments")

        # The column is NULLABLE: PRAGMA table_info notnull flag must be 0.
        conn = sqlite3.connect(str(db_path))
        for table in ("videos", "candidate_segments"):
            by_name = {c[1]: c for c in conn.execute(f"PRAGMA
table_info({table})").fetchall()}
            assert by_name["user_id"][3] == 0, f"{table}.user_id must be nullable"
        conn.close()


    def test_v4_existing_writers_leave_user_id_null(tmp_path):
        """NULL user_id back-compat: add_video / add_candidate_segment never set user_id, so
every
        row a current writer produces is local (NULL) ? byte-identical to pre-v4
behaviour."""
        db_path = tmp_path / "test.db"
        db = Database(str(db_path))
        db.add_video("vid1", "vid1.mp4", "processed", [])
        db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)

        conn = sqlite3.connect(str(db_path))
        vu = conn.execute("SELECT user_id FROM videos WHERE id = 'vid1'").fetchone()[0]
        cu = conn.execute("SELECT user_id FROM candidate_segments WHERE video_id =
'vid1'").fetchone()[0]
        conn.close()
        assert vu is None
        assert cu is None
        # Existing readers are unaffected (no user_id filter in the query path).
        assert len(db.query_candidate_segments("vid1")) == 1


    def test_migration_v3_db_upgrades_to_v5_preserving_rows(tmp_path):
        """A real v3 DB (jobs + candidates already populated) upgrades cleanly to v5: the
new
        columns/tables appear, user_id back-fills to NULL, and prior rows survive the
ALTERs."""
        db_path = tmp_path / "legacy_v3.db"
        db = Database(str(db_path))  # fresh DB is born at the current version; simulate a
v3 state
        # Seed pre-upgrade data, then force the version back to 3 and re-open to run v4+v5.
        db.add_video("vidA", "a.mp4", "processed", [{"v": 1}])
        db.add_candidate_segment("vidA", "wasb_hybrid", 2.0, 9.0, stats={"net_crossings":
3})
        conn = sqlite3.connect(str(db_path))
        conn.execute("DROP INDEX IF EXISTS idx_videos_user")
        conn.execute("DROP INDEX IF EXISTS idx_candidates_user")
        conn.execute("DROP TABLE IF EXISTS user_models")
        # Recreate v3-shaped tables WITHOUT user_id by rebuilding (simplest: just rewind
version
        # and drop the v4/v5 artifacts that a true v3 DB wouldn't have).
        conn.execute("ALTER TABLE videos DROP COLUMN user_id")
        conn.execute("ALTER TABLE candidate_segments DROP COLUMN user_id")
        conn.execute("PRAGMA user_version = 3")
        conn.commit()
        conn.close()
        assert "user_id" not in _table_cols(db_path, "videos")

        # Re-open: the migration ladder must carry the v3 DB up to v5 without data loss.
        db2 = Database(str(db_path))
        conn = sqlite3.connect(str(db_path))
```

```
234         assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
235         assert "user_models" in {r[0] for r in conn.execute(
236             "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
237         conn.close()
238         assert "user_id" in _table_cols(db_path, "videos")
239         assert "user_id" in _table_cols(db_path, "candidate_segments")
240
241         v = db2.get_video("vidA")
242         assert v is not None and v["validation_results"] == [{"v": 1}]
243         cands = db2.query_candidate_segments("vidA")
```

125. assistant (2026-06-18T01:47:55.094Z)

Conventions clear. My test covers the *partial-apply* path (columns present, version behind) ? complementary to the existing happy-path test. Let me read the worktree's exact copy and implement.

126. user (2026-06-18T01:47:56.529Z)

```
42              return conn
43
44          def _init_db(self):
45              """Initializes tables for videos, play_segments, and events."""
46              with self._connect() as conn:
47                  cursor = conn.cursor()
```

127. assistant (2026-06-18T01:47:59.465Z)

Now implement the fix ? the idempotent-ALTER helper and the v3/v4 call sites:

128. user (2026-06-18T01:48:03.836Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\backend\infrastructure\database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

129. user (2026-06-18T01:48:11.029Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\backend\infrastructure\database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

130. user (2026-06-18T01:48:17.401Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\backend\infrastructure\database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

131. assistant (2026-06-18T01:48:19.651Z)

Now the regression test for the partial/interrupted-migration path:

132. user (2026-06-18T01:48:20.715Z)

```
243         cands = db2.query_candidate_segments("vidA")
244         assert len(cands) == 1 and cands[0]["stats"]["net_crossings"] == 3
245
246
247     # --- v5: per-user model store CRUD ------------------------------------------------
248
249     def test_user_models_schema_contract(tmp_path):
250         db_path = tmp_path / "test.db"
```

133. user (2026-06-18T01:48:37.626Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-
fix\tests\test_database.py has been updated successfully. (file state is current in your context
? no need to Read it back)
```

134. assistant (2026-06-18T01:48:41.220Z)

I left a nonsensical dead line in that test ? let me clean it up.

135. user (2026-06-18T01:48:44.195Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-
fix\tests\test_database.py has been updated successfully. (file state is current in your context
? no need to Read it back)
```

136. assistant (2026-06-18T01:48:51.539Z)

Now verify the test pins the bug: run it against the fix (expect pass), then temporarily revert the fix to confirm it fails without it.

137. user (2026-06-18T01:48:57.715Z)

```
=== 1) new test WITH the fix (expect PASS) ===
.                                                                        [100%]
1 passed in 0.18s

=== 2) revert ONLY database.py, re-run (expect FAIL: duplicate column name) ===
Saved working directory and index state WIP on fix/db-migration-idempotent-alters: 4bd9d2d
fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
                # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
>               cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
E               sqlite3.OperationalError: duplicate column name: policy

backend\infrastructure\database.py:156: OperationalError
=========================== short test summary info ===========================
FAILED tests/test_database.py::test_migration_idempotent_on_interrupted_upgrade
1 failed in 0.32s
=== restore the fix ===
no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (3f83b3951c42cdc5d90d0bf9702de77b637a4824)
```

138. assistant (2026-06-18T01:49:02.416Z)

The test pins the bug precisely ? passes with the fix, fails with `duplicate column name: policy` without it. Now the full local gate: DB tests, full suite, ruff, mypy, coverage.

139. user (2026-06-18T01:49:07.758Z)

```
=== test_database.py (full) ===
....................                                                     [100%]
20 passed in 0.83s
=== ruff + mypy on changed files ===
All checks passed!
Success: no issues found in 1 source file
```

140. assistant (2026-06-18T01:49:10.840Z)

DB tests 20/20, ruff + mypy clean. Now the full suite + repo-wide lint/type + coverage gate:

141. user (2026-06-18T01:51:15.539Z)

```
=== ruff check . ===
```

```
All checks passed!
=== mypy backend training ===
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 104 source files
=== full suite + coverage ===
TOTAL                                                7992   1545    81%
Required test coverage of 70% reached. Total coverage: 80.67%
847 passed, 7 skipped, 7 warnings in 86.46s (0:01:26)
================================================================================
 RESULT: PASS - all tests green
================================================================================
```

142. assistant (2026-06-18T01:51:23.088Z)

```
All green: ruff clean, mypy clean (104 files), suite 847 passed/7 skipped (the 2 litmus tests
skip here since this worktree's `output/` is empty ? expected), coverage 80.67%. Let me review
the diff, then commit.
```

143. user (2026-06-18T01:51:24.019Z)

```
diff --git a/backend/infrastructure/database.py b/backend/infrastructure/database.py
index 59473ac..58153a9 100644
--- a/backend/infrastructure/database.py
+++ b/backend/infrastructure/database.py
@@ -41,6 +41,24 @@ class Database:
             pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
         return conn

+    @staticmethod
+    def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
+        """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.
+
+        A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn:``
+        transaction, so an interrupted v3/v4 block can leave the column added while
+        ``user_version`` stays un-bumped ? and the next open would re-run the same
+        ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
+        future ``Database(...)`` construction (``_init_db`` runs in ``__init__``). Swallow
+        ONLY that specific error so the ladder stays crash-safe, matching the re-runnable
+        ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
+        """
+        try:
+            cursor.execute(ddl)
+        except sqlite3.OperationalError as exc:
+            if "duplicate column name" not in str(exc).lower():
+                raise
+
     def _init_db(self):
         """Initializes tables for videos, play_segments, and events."""
         with self._connect() as conn:
@@ -152,9 +170,10 @@ class Database:
                 # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
due
                 # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any
worker
                 # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
-                # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
-                cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
-                cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
+                # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
+                # ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
+                self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
+                self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
```

```
                cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
                cursor.execute("PRAGMA user_version = 3")

@@ -163,10 +182,11 @@ class Database:
                # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
                # path can scope rows to an owner. **NULL = local install = byte-identical to
                # today** ? every existing row stays NULL and every existing query (which never
-               # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive.
+               # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive;
+               # the ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
                # Additive ONLY: no served-behavior change rides on this slice.
-               cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
-               cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
+               self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
TEXT")
+               self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
                # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path
free
                # of index churn while making the eventual per-user lookups cheap.
                cursor.execute(
diff --git a/tests/test_database.py b/tests/test_database.py
index 5ccdb49..8909208 100644
--- a/tests/test_database.py
+++ b/tests/test_database.py
@@ -244,6 +244,39 @@ def test_migration_v3_db_upgrades_to_v5_preserving_rows(tmp_path):
     assert len(cands) == 1 and cands[0]["stats"]["net_crossings"] == 3


+def test_migration_idempotent_on_interrupted_upgrade(tmp_path):
+    """An INTERRUPTED v3/v4 upgrade must not brick the DB. A DDL ``ALTER`` auto-commits
+    on its own, so a crash after the ADD COLUMNs but before the ``PRAGMA user_version``
+    bump leaves the columns present while the version is stale ? re-opening re-runs the
+    bare ALTERs. The ladder must tolerate that (re-runnable, like the v1/v2/v5 blocks)
+    instead of raising ``OperationalError: duplicate column name``."""
+    db_path = tmp_path / "interrupted.db"
+    Database(str(db_path))  # fully migrate to the current version (v3/v4 columns all present)
+
+    # Simulate the interruption: the v3/v4 ADD COLUMNs already landed and auto-committed,
+    # but the process died before the version bump, so user_version is stuck at 2.
+    conn = sqlite3.connect(str(db_path))
+    conn.execute("DROP TABLE IF EXISTS user_models")          # a true v2 DB lacks the v5 table
+    conn.execute("PRAGMA user_version = 2")
+    conn.commit()
+    conn.close()
+
+    # Re-opening re-runs v3 (jobs.policy/next_poll_at) and v4 (videos/candidate_segments
+    # .user_id) over columns that ALREADY exist ? must NOT raise, and must reach v5.
+    Database(str(db_path))
+    conn = sqlite3.connect(str(db_path))
+    assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
+    conn.close()
+    # Columns/tables are all present after the clean re-apply.
+    assert "policy" in _table_cols(db_path, "jobs")
+    assert "user_id" in _table_cols(db_path, "videos")
+    assert "user_id" in _table_cols(db_path, "candidate_segments")
+    conn = sqlite3.connect(str(db_path))
+    assert "user_models" in {r[0] for r in conn.execute(
+        "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
+    conn.close()
+
```

```
+
 # --- v5: per-user model store CRUD ---------------------------------------------

 def test_user_models_schema_contract(tmp_path):
```

144. assistant (2026-06-18T01:51:27.581Z)

```
Clean diff. Let me commit (re-verify HEAD, stage explicit paths):
```

145. user (2026-06-18T01:51:37.996Z)

```
fix/db-migration-idempotent-alters
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
M  backend/infrastructure/database.py
M  tests/test_database.py
[fix/db-migration-idempotent-alters d140722] fix(db): make v3/v4 migration ALTERs idempotent so
an interrupted upgrade can't brick the DB
 2 files changed, 59 insertions(+), 6 deletions(-)
=== only my commit atop origin/master? ===
d140722 fix(db): make v3/v4 migration ALTERs idempotent so an interrupted upgrade can't brick
the DB
 backend/infrastructure/database.py | 32 ++++++++++++++++++++++++------
 tests/test_database.py             | 33 +++++++++++++++++++++++++++++++++
 2 files changed, 59 insertions(+), 6 deletions(-)
```

146. assistant (2026-06-18T01:51:41.357Z)

```
Clean single commit. Push and open PR #1:
```

147. user (2026-06-18T01:51:59.108Z)

```
remote:
branch 'fix/db-migration-idempotent-alters' set up to track 'origin/fix/db-migration-idempotent-
alters'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      fix/db-migration-idempotent-alters -> fix/db-migration-idempotent-alters
https://github.com/avidullu/badminton-highlight-indexer/pull/212
```

148. assistant (2026-06-18T01:52:13.682Z)

```
PR **[#212](https://github.com/avidullu/badminton-highlight-indexer/pull/212)** open. PR #1 is
committed+pushed (no uncommitted work), so I can safely set up PR #2 in its own worktree while
#1's CI runs. Let me create it and read the two scripts.
```

149. user (2026-06-18T01:52:17.754Z)

```
branch 'fix/scripts-unwrap-segmenter-result' set up to track 'origin/master'.
HEAD is now at 4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
=== run_phase3_eval.py imports (1-45) ===

from dotenv import load_dotenv
load_dotenv()  # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters

from backend.infrastructure.database import Database
from backend.config import load_config
from backend.utils.hashing import compute_video_id as get_file_md5  # canonical video_id
# Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
from backend.pipeline.segmenters import segmenter_registry
from backend.eval.annotations import load_annotations
from backend.eval.harness import evaluate, format_report_text, report_to_dict
from backend.eval import batch

logger = logging.getLogger(__name__)
```

```
def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(
=== run_phase3_eval.py call-site context (141-147) ===
            else:
                print(f"[proc] {vid}: segmenting with {args.segmenter}...")
                db.add_video(video_id, video_path, "processed", [])
                t0 = time.time()
                segs = segmenter.process_video(video_id, video_path, max_frames=args.max_frames)
                print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")

=== run_process_and_stitch.py imports (1-40) ===
import os
import json
import logging
from dotenv import load_dotenv

load_dotenv()
```

Central root-logger setup ? all backend modules use logging.getLogger(__name__). Quiet third-party HTTP chatter by default; re-honoured against the config knob (logging.verbose_http) once the config is loaded in main().

```
from backend.utils.logging_setup import setup_logging
setup_logging()

from backend.infrastructure.database import Database
from backend.config import load_config
from backend.utils.hashing import compute_video_id
from backend.pipeline.segmenters import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.tools.export import format_rally_summary

logger = logging.getLogger(__name__)

def main():
    video_path = r"C:\Users\avidu\Videos\Testing\KushagraYashAviFirstVideo.mp4"
    output_dir = r"C:\Users\avidu\Videos\Testing"
    output_filename = "Final_2_OutputOfKushagraYashAviFirstVideo.mp4"

    print("Loading config...")
    cfg = load_config()  # typed AppConfig ? single source of defaults
    # Re-apply now that the verbose_http knob is known (default keeps HTTP chatter quiet).
    setup_logging(verbose_http=cfg.logging.verbose_http)

    print("Verifying files exist...")
    if not os.path.exists(video_path):
        print(f"Error: Video file not found at {video_path}")
        return

    os.makedirs(cfg.output.output_dir, exist_ok=True)
    db_path = os.path.join(cfg.output.output_dir, cfg.output.db_name)
    db = Database(db_path)
```

150. user (2026-06-18T01:52:43.303Z)

```
40          db = Database(db_path)
41
42          # Calculate MD5 of the video to uniquely identify it
43          import time
44          print("Calculating video MD5 (this might take a few seconds on large files)...",
flush=True)
45          t0 = time.time()
46          video_id = compute_video_id(video_path)
47          print(f"Calculated MD5: {video_id} (took {time.time()-t0:.1f}s)", flush=True)
```

```
48
49          segments = []
50          cached_video = db.get_video(video_id)
51          cache_hit = False
52
53          if cached_video and cached_video.get("status") == "processed":
54              segments = db.query_play_segments(video_id, {})
55              if len(segments) > 0:
56                  print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
with {len(segments)} parsed play segments.", flush=True)
57                  choice = input("Would you like to (1) Run stitching on the cached segments,
or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
58                  if choice == '1':
59                      print("[CACHE] Using cached play segments. Skipping segmenter API
call.", flush=True)
60                      cache_hit = True
61                  else:
62                      print("[CACHE] Ignoring cache...", flush=True)
63              else:
64                  print("[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.", flush=True)
65
66          if not cache_hit:
67              print("[CACHE] Initializing clean indexing...", flush=True)
68              db.clear_video_data(video_id)
69
70              print("Bypassing OpenCV validations for performance on massive files...",
flush=True)
71              db.add_video(video_id, video_path, "processed", [])
72
73              print("Running segmenter & indexing...", flush=True)
74              seg_name = cfg.indexing.default_segmenter
75              segmenter_cls = segmenter_registry.get(seg_name)
76              if not segmenter_cls:
77                  print(f"[FAIL] Segmenter '{seg_name}' not found.")
78                  return
79              print(f"Using segmenter: {seg_name}", flush=True)
80              segmenter = segmenter_cls(db, cfg)
81              segments = segmenter.process_video(video_id, video_path)
82              print(f"Successfully indexed {len(segments)} play segments.", flush=True)
83
84          if not segments:
85              print("No play segments detected. Cannot compile highlights.")
86              return
87
88
89          end_padding = cfg.stitching.end_padding
90          begin_padding = cfg.stitching.begin_padding
91          min_shot_count = cfg.stitching.min_shot_count
92
93          if cache_hit:
94              print("\n--- Stitching Customizations ---")
95              try:
96                  bpad_input = input(f"Enter begin_padding in seconds (default
{begin_padding}): ").strip()
97                  if bpad_input:
98                      begin_padding = float(bpad_input)
99
100                 pad_input = input(f"Enter end_padding in seconds (default {end_padding}):
").strip()
101                 if pad_input:
102                     end_padding = float(pad_input)
103
104                 shot_input = input(f"Enter min_shot_count to filter by (default
{min_shot_count}): ").strip()
```

```
105                 if shot_input:
106                     min_shot_count = int(shot_input)
107             except ValueError:
108                 print("[WARNING] Invalid input detected, falling back to defaults from
config.json.")
109
110         print(f"\nCompiling highlights (with begin_padding={begin_padding}s,
end_padding={end_padding}s, min_shot_count={min_shot_count})...", flush=True)
111
112         # Filter segments where shot_count >= min_shot_count
113         time_pairs = []
114         skipped_count = 0
115         for s in segments:
116             meta = s.get("metadata", {})
117             if isinstance(meta, str):
118                 try:
119                     meta = json.loads(meta)
120                 except Exception:
121                     meta = {}
122             shot_count = meta.get("shot_count", 0)
123             if shot_count >= min_shot_count:
124                 time_pairs.append((s["start_time"], s["end_time"]))
125             else:
126                 skipped_count += 1
127
128         if skipped_count > 0:
129             print(f"[FILTER] Excluded {skipped_count} play segments with fewer than
{min_shot_count} shots.", flush=True)
```

151. assistant (2026-06-18T01:52:42.258Z)

Let me read the full `run_process_and_stitch.py` flow to fix it without breaking the cache path (`segments` is used after the branch):

152. assistant (2026-06-18T01:53:01.052Z)

`segments` lifecycle is clear (init `[]`, cache path = real list, non-cache path = the Result to unwrap). Let me check what's importable here (cv2/torch) to gauge how far I can verify the script fixes locally.

153. user (2026-06-18T01:53:14.730Z)

```
cv2 4.13.0
torch 2.6.0+cu124
numpy 2.4.6
=== can we import the segmenter registry (pulls motion?cv2)? ===
registry OK, segmenters: n/a
=== confirm skipped/loop var in run_phase3_eval ===
117:    skipped = []
126:            skipped.append((vid, "no video file"))
130:            skipped.append((vid, "no GT csv"))
```

154. assistant (2026-06-18T01:54:21.222Z)

cv2/torch/numpy are all available here (the CLAUDE.md note is stale) and `skipped`/`vid` are in scope. The scripts themselves aren't unit-testable (hardcoded paths, `input()` prompts, argparse + real-video deps), so to get a *real* regression test I'll add a tiny generic helper to `backend/results.py` ? `unwrap_or_failure()` ? DRYing the unwrap pattern (duplicated 4× across main.py + these scripts) and making it unit-testable. Let me check for an existing results test and read the worktree's results.py.

155. user (2026-06-18T01:54:24.197Z)

```
ls: cannot access 'tests/test_results.py': No such file or directory
```

```
=== existing results tests (if any) ===
tests/test_base.py
=== py_compile both scripts (baseline syntax OK) ===
compile OK
```

156. user (2026-06-18T01:54:29.819Z)

```
18      from __future__ import annotations
19
20      from dataclasses import dataclass, field
21      from typing import Generic, TypeVar, List, Optional, Any, Union
22
23      T = TypeVar("T")
24
25
26      @dataclass
27      class Success(Generic[T]):
28          """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30          value: T
31          warnings: List[str] = field(default_factory=list)
32
33          def is_success(self) -> bool:
34              return True
35
36          def unwrap(self) -> T:
37              """Return the value. Only safe to call if you already know it's a Success."""
38              return self.value
39
40
41      @dataclass
42      class Failure:
43          """Represents a failure with a human-readable message and optional details."""
44
45          message: str
46          details: Optional[dict[str, Any]] = None
47          is_recoverable: bool = False  # e.g. missing API key (recoverable) vs internal crash
48
49          def is_success(self) -> bool:
50              return False
51
52          def unwrap(self) -> Any:
53              """Raise to make misuse obvious in code that assumes success."""
54              raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57      # Convenient alias for use in type hints: Result[list[dict], Failure]
58      # Result[ValueType] means Success[ValueType] | Failure
59      # (the Failure "error" type is fixed for this contract)
60      Result = Union[Success[T], Failure]
```

157. assistant (2026-06-18T01:54:32.315Z)

Now add the `unwrap_or_failure` helper to `backend/results.py`:

158. user (2026-06-18T01:54:34.375Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-
unwrap\backend\results.py has been updated successfully. (file state is current in your context
? no need to Read it back)
```

159. user (2026-06-18T01:54:42.700Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-
```

```
unwrap\backend\results.py has been updated successfully. (file state is current in your context
? no need to Read it back)
```

160. assistant (2026-06-18T01:54:45.143Z)

Now apply the unwrap in both scripts. First `run_phase3_eval.py`:

161. user (2026-06-18T01:54:47.552Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

162. user (2026-06-18T01:54:51.449Z)

```
34      from backend.utils.hashing import compute_video_id as get_file_md5  # canonical video_id
35      # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
36      from backend.pipeline.segmenters import segmenter_registry
37      from backend.eval.annotations import load_annotations
38      from backend.eval.harness import evaluate, format_report_text, report_to_dict
39      from backend.eval import batch
```

163. user (2026-06-18T01:54:56.525Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_phase3_eval.py has been updated successfully. (file state is current in your context ? no need to Read it back)

164. user (2026-06-18T01:54:59.364Z)

```
142                 else:
143                     print(f"[proc] {vid}: segmenting with {args.segmenter}...")
144                     db.add_video(video_id, video_path, "processed", [])
145                     t0 = time.time()
146                     segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
147                     print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")
148
```

165. user (2026-06-18T01:55:05.493Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_phase3_eval.py has been updated successfully. (file state is current in your context ? no need to Read it back)

166. assistant (2026-06-18T01:55:10.744Z)

Now `run_process_and_stitch.py` ? import and call site:

167. user (2026-06-18T01:55:11.716Z)

```
15      from backend.config import load_config
16      from backend.utils.hashing import compute_video_id
17      from backend.pipeline.segmenters import segmenter_registry
18      from backend.pipeline.stitcher.compiler import VideoCompiler
19      from backend.tools.export import format_rally_summary
20
```

168. user (2026-06-18T01:55:17.210Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_process_and_stitch.py has been updated successfully. (file state is current in your context ? no need to Read it back)

169. user (2026-06-18T01:55:20.960Z)

```
79                 return
80             print(f"Using segmenter: {seg_name}", flush=True)
81             segmenter = segmenter_cls(db, cfg)
82             segments = segmenter.process_video(video_id, video_path)
83             print(f"Successfully indexed {len(segments)} play segments.", flush=True)
84
85         if not segments:
86             print("No play segments detected. Cannot compile highlights.")
87             return
```

170. user (2026-06-18T01:55:27.853Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-
unwrap\scripts\run_process_and_stitch.py has been updated successfully. (file state is current
in your context ? no need to Read it back)
```

171. assistant (2026-06-18T01:55:41.803Z)

Now a focused unit test for the helper that also documents the original crash:

172. assistant (2026-06-18T01:55:47.731Z)

```
"""Unit tests for ``backend.results`` ? the Success/Failure contract + the ``unwrap_or_failure``
helper that standalone scripts use to avoid ``len()``-ing a raw ``Success`` dataclass."""
import pytest

from backend.results import Failure, Success, unwrap_or_failure


def test_unwrap_success_returns_value_and_no_failure():
    value, failure = unwrap_or_failure(Success(value=[{"start": 1.0}]))
    assert value == [{"start": 1.0}]
    assert failure is None


def test_unwrap_failure_returns_failure_and_no_value():
    f = Failure(message="Could not open video: x.mp4", is_recoverable=False)
    value, failure = unwrap_or_failure(f)
    assert value is None
    assert failure is f


def test_unwrap_bare_value_passes_through():
    """Legacy segmenters that predate the Result contract still return a bare list ? the helper
    passes those through unchanged as ``(value, None)`` so a mixed segmenter set all works."""
    bare = [{"start": 1.0}, {"start": 2.0}]
    value, failure = unwrap_or_failure(bare)
    assert value == bare
    assert failure is None


def test_unwrap_fixes_the_len_on_success_crash():
    """Regression for the scripts bug: ``len()`` on a raw ``Success`` raises ``TypeError``;
    unwrapping first yields the underlying list so callers (run_phase3_eval /
    run_process_and_stitch, which default to the ``motion`` segmenter) can ``len()`` it."""
    result = Success(value=[1, 2, 3])
    with pytest.raises(TypeError):
        len(result)                      # the original bug (Success has no __len__)
    segs, failure = unwrap_or_failure(result)
    assert failure is None
    assert len(segs) == 3                # fixed
```

173. user (2026-06-18T01:55:47.754Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\scripts-unwrap\tests\test_results.py (file state is current in your
context ? no need to Read it back)
```

174. assistant (2026-06-18T01:56:00.280Z)

Now verify PR #2: py_compile, the new test, confirm imports resolve, then the full gate.

175. user (2026-06-18T01:56:10.670Z)

```
=== __main__ guards present (safe to import)? ===
scripts/run_phase3_eval.py:180:if __name__ == "__main__":
scripts/run_process_and_stitch.py:160:if __name__ == "__main__":
=== py_compile both scripts ===
compile OK
=== imports resolve (new unwrap_or_failure import) ===
script imports OK
=== new results test ===
....                                                                     [100%]
4 passed in 0.07s
```

176. assistant (2026-06-18T01:56:19.816Z)

Both scripts compile and import cleanly; 4 new tests pass. Since cv2 is available, let me also do a quick real-segmenter sanity check (motion on a bogus path ? Failure flows through the helper), then run the full gate.

177. user (2026-06-18T01:57:05.063Z)

```
=== integration sanity: real motion segmenter Failure flows through unwrap_or_failure ===
Segmenter could not open video: /no/such/video.mp4
raw result type: Failure
unwrapped -> segs: None | failure is Failure: True
OK: motion Failure handled cleanly (no TypeError, guard fires)
=== ruff + mypy ===
All checks passed!
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 104 source files
```

178. assistant (2026-06-18T01:57:09.012Z)

Integration check confirms the fix works against the **real** motion segmenter (Failure flows through cleanly, no TypeError). ruff + mypy clean. Now the full suite + coverage:

179. user (2026-06-18T01:58:39.870Z)

```
TOTAL                                                7990   1544    81%
Required test coverage of 70% reached. Total coverage: 80.68%
850 passed, 7 skipped, 7 warnings in 86.14s (0:01:26)
================================================================================
 RESULT: PASS - all tests green
================================================================================
```

180. assistant (2026-06-18T01:58:43.045Z)

All green: 850 passed, 7 skipped, coverage 80.68%. Let me review the PR #2 diff and commit.

181. user (2026-06-18T01:58:44.615Z)

```
 backend/results.py                | 20 ++++++++++++++++++-
 scripts/run_phase3_eval.py        | 11 ++++++++++-
 scripts/run_process_and_stitch.py |  9 ++++++++-
```

```
 3 files changed, 37 insertions(+), 3 deletions(-)
============ full diff ============
diff --git a/backend/results.py b/backend/results.py
index d6b1ea4..3c354c7 100644
--- a/backend/results.py
+++ b/backend/results.py
@@ -18,7 +18,7 @@ This module has no dependencies on the rest of the app.
 from __future__ import annotations

 from dataclasses import dataclass, field
-from typing import Generic, TypeVar, List, Optional, Any, Union
+from typing import Generic, TypeVar, List, Optional, Any, Tuple, Union

 T = TypeVar("T")

@@ -58,3 +58,21 @@ class Failure:
 # Result[ValueType] means Success[ValueType] | Failure
 # (the Failure "error" type is fixed for this contract)
 Result = Union[Success[T], Failure]
+
+
+def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
+    """Split a segmenter/provider return into ``(value, failure)`` so a caller branches once.
+
+    - ``Failure``            -> ``(None, failure)``
+    - ``Success(value=v)``   -> ``(v, None)``
+    - a bare value (legacy code that predates this contract and still returns a raw list)
+      -> ``(value, None)``
+
+    This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline; callers
+    that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
+    on a ``Success`` dataclass. ``input`` is typed ``Any`` because the contract is
heterogeneous
+    (Success | Failure | legacy bare value)."""
+    if isinstance(result, Failure):
+        return None, result
+    value = result.value if hasattr(result, "value") else result
+    return value, None
diff --git a/scripts/run_phase3_eval.py b/scripts/run_phase3_eval.py
index 56e21ca..dd1263e 100644
--- a/scripts/run_phase3_eval.py
+++ b/scripts/run_phase3_eval.py
@@ -34,6 +34,7 @@ from backend.config import load_config
 from backend.utils.hashing import compute_video_id as get_file_md5  # canonical video_id
 # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
 from backend.pipeline.segmenters import segmenter_registry
+from backend.results import unwrap_or_failure
 from backend.eval.annotations import load_annotations
 from backend.eval.harness import evaluate, format_report_text, report_to_dict
 from backend.eval import batch
@@ -142,7 +143,15 @@ def main(argv=None) -> int:
                 print(f"[proc] {vid}: segmenting with {args.segmenter}...")
                 db.add_video(video_id, video_path, "processed", [])
                 t0 = time.time()
-                segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
+                # process_video returns a Result (Success|Failure) for some segmenters
+                # (e.g. the default `motion`); unwrap before len() so this doesn't crash with
+                # TypeError on a Success dataclass ? mirrors the unwrap in backend/main.py.
+                segs, failure = unwrap_or_failure(
+                    segmenter.process_video(video_id, video_path, max_frames=args.max_frames))
+                if failure is not None:
+                    print(f"        -> FAILED: {failure.message}")
+                    skipped.append((vid, f"segment failed: {failure.message}"))
```

```
+                    continue
                 print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")

         gt = load_annotations(csv_path)
diff --git a/scripts/run_process_and_stitch.py b/scripts/run_process_and_stitch.py
index b8ffef6..ea7c2de 100644
--- a/scripts/run_process_and_stitch.py
+++ b/scripts/run_process_and_stitch.py
@@ -16,6 +16,7 @@ from backend.config import load_config
 from backend.utils.hashing import compute_video_id
 from backend.pipeline.segmenters import segmenter_registry
 from backend.pipeline.stitcher.compiler import VideoCompiler
+from backend.results import unwrap_or_failure
 from backend.tools.export import format_rally_summary

 logger = logging.getLogger(__name__)
@@ -78,7 +79,13 @@ def main():
             return
         print(f"Using segmenter: {seg_name}", flush=True)
         segmenter = segmenter_cls(db, cfg)
-        segments = segmenter.process_video(video_id, video_path)
+        # process_video returns a Result (Success|Failure) for some segmenters (e.g. `motion`);
+        # unwrap before len()/truthiness so this doesn't crash with TypeError on a Success
+        # dataclass (and so the empty-guard below actually sees the list) ? mirrors
backend/main.py.
+        segments, failure = unwrap_or_failure(segmenter.process_video(video_id, video_path))
+        if failure is not None:
+            print(f"[FAIL] Segmentation failed: {failure.message}")
+            return
         print(f"Successfully indexed {len(segments)} play segments.", flush=True)

     if not segments:
```

182. assistant (2026-06-18T01:58:49.712Z)

Small docstring typo ? the param is `result`, not `input`. Let me fix it.

183. user (2026-06-18T01:58:51.947Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-
unwrap\backend\results.py has been updated successfully. (file state is current in your context
? no need to Read it back)
```

184. assistant (2026-06-18T01:58:54.309Z)

Now commit, push, and open PR #2:

185. user (2026-06-18T01:59:06.757Z)

```
fix/scripts-unwrap-segmenter-result
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
warning: in the working copy of 'tests/test_results.py', LF will be replaced by CRLF the next
time Git touches it
M  backend/results.py
M  scripts/run_phase3_eval.py
M  scripts/run_process_and_stitch.py
A  tests/test_results.py
[fix/scripts-unwrap-segmenter-result 03d11d9] fix(scripts): unwrap the segmenter Result before
len()/truthiness (motion crashes the default path)
 4 files changed, 76 insertions(+), 3 deletions(-)
 create mode 100644 tests/test_results.py
03d11d9 fix(scripts): unwrap the segmenter Result before len()/truthiness (motion crashes the
default path)
 backend/results.py               | 20 ++++++++++++++++++-
```

```
 scripts/run_phase3_eval.py        | 11 ++++++++++-
 scripts/run_process_and_stitch.py |  9 ++++++++-
 tests/test_results.py             | 39 +++++++++++++++++++++++++++++++++++++++
 4 files changed, 76 insertions(+), 3 deletions(-)
```

186. assistant (2026-06-18T01:59:10.242Z)

Clean single commit. Push and open PR #2:

187. user (2026-06-18T01:59:27.051Z)

```
branch 'fix/scripts-unwrap-segmenter-result' set up to track 'origin/fix/scripts-unwrap-
segmenter-result'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      fix/scripts-unwrap-segmenter-result -> fix/scripts-unwrap-segmenter-result
https://github.com/avidullu/badminton-highlight-indexer/pull/213
```

188. assistant (2026-06-18T01:59:38.254Z)

Both PRs open: **#212** (DB migration) and **#213** (scripts unwrap). Now the same due-diligence as before ? adversarial review of each diff plus CI. Let me launch a combined review workflow and check both PRs' CI in parallel.

189. user (2026-06-18T02:00:03.232Z)

```
Workflow launched in background. Task ID: wz4qtf4k3
Summary: Adversarial pre-merge review of the DB-migration and scripts-unwrap fix PRs
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e3b53aed-f4e
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer
--claude-worktrees-scripts-unwrap\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\review-
two-fix-prs-wf_e3b53aed-f4e.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:
"C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer--claude-
worktrees-scripts-unwrap\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\review-two-fix-
prs-wf_e3b53aed-f4e.js"} to iterate without resending the script.)
Run ID: wf_e3b53aed-f4e
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--
Users-avidu-Projects-badminton-highlight-indexer--claude-worktrees-scripts-
unwrap\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\review-two-fix-prs-
wf_e3b53aed-f4e.js", resumeFromRunId: "wf_e3b53aed-f4e"}) ? completed agents return cached
results.

You will be notified when it completes. Use /workflows to watch live progress.
```

190. user (2026-06-18T02:00:09.107Z)

```
=== PR #212 checks ===
Full test suite (offline, mocked)       pass    2m9s    https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27731509959/job/82039380259
Golden fixtures cross-OS (macos-latest) pass    16s     https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27731509959/job/82039380287
Golden fixtures cross-OS (ubuntu-latest)        pass    18s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27731509959/job/82039380330
Golden fixtures cross-OS (windows-latest)       pass    51s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27731509959/job/82039380303
Mypy (lenient baseline) pass    34s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27731509959/job/82039380268
Ruff lint       pass    12s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27731509959/job/82039380297
Syntax sweep + import smoke (no GPU stack)      pass    18s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27731509959/job/82039380313
--- mergeable ---
{"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE"}
```

```
=== PR #213 checks ===
Full test suite (offline, mocked)       pending 0       https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27731763134/job/82040168742
Golden fixtures cross-OS (windows-latest)       pending 0
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27731763134/job/82040168669
Golden fixtures cross-OS (macos-latest) pass    19s     https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27731763134/job/82040168719
Golden fixtures cross-OS (ubuntu-latest)        pass    22s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27731763134/job/82040168701
Mypy (lenient baseline) pass    30s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27731763134/job/82040168694
Ruff lint       pass    9s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27731763134/job/82040168654
Syntax sweep + import smoke (no GPU stack)      pass    18s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27731763134/job/82040168703
--- mergeable ---
{"mergeStateStatus":"UNSTABLE","mergeable":"MERGEABLE"}
```

191. assistant (2026-06-18T02:00:14.002Z)

PR #212 CI is fully green (CLEAN); PR #213 has the full suite still running. The review workflow is running. Let me wait for the review verdicts.

192. user (2026-06-18T02:07:14.587Z)

<retrieval_status>timeout</retrieval_status>

<task_id>wz4qtf4k3</task_id>

<task_type>local_workflow</task_type>

<status>running</status>

193. assistant (2026-06-18T02:07:24.134Z)

Still running. Let me continue waiting for the review.

194. user (2026-06-18T02:08:45.322Z)

<retrieval_status>success</retrieval_status>

<task_id>wz4qtf4k3</task_id>

<task_type>local_workflow</task_type>

<status>completed</status>

<output>
```
{
  "summary": "Adversarial pre-merge review of the DB-migration and scripts-unwrap fix PRs",
  "agentCount": 10,
  "logs": [
    "gatekeeper verdicts: pr212-db-migration=merge · pr213-scripts-unwrap=merge"
  ],
  "result": {
    "verdicts": [
      {
        "pr": "pr212-db-migration",
        "verdict": "merge",
        "must_fixes": [],
        "should_fixes": [
          "Optional (non-blocking) test coverage: the regression test only exercises the all-
columns-present interruption. Consider parametrizing/adding a case for a partial interruption
(only one of v4's two ALTERs landed) to pin the per-statement granularity. I independently
```
</output>

```
verified this partial case already re-applies cleanly to v5, so it is a coverage nicety, not a
masked bug. tests/test_database.py:247-279.",
          "Optional: backend/infrastructure/database.py:60 (the `raise` re-raise branch in
_add_column_idempotent) is the one new line not directly asserted by a test. I confirmed it
works (a 'no such table' OperationalError correctly propagates), but a tiny test asserting non-
duplicate errors re-raise would close the last gap."
        ],
        "rationale": "Independently verified, not rubber-stamped. The bug is real and the fix is
correct, surgical, and well-tested. (1) Scope: single commit d140722, only database.py (new
_add_column_idempotent static helper wired at the four v3/v4 ALTER sites) + test_database.py; it
is the only commit ahead of origin/master. The #213/results.py/main.py/scripts-unwrap clause in
one lens is a template carry-over and genuinely does not apply. (2) Bug confirmed: reverting
database.py to origin/master makes the new test fail with exactly `sqlite3.OperationalError:
duplicate column name: policy`; the fix makes it pass. The ladder bumps PRAGMA user_version at
the END of each `if version < N` block, so a crash after an auto-committed ALTER but before the
bump bricks every future Database() open ? the documented failure mode. (3) Swallow is correctly
narrow: on SQLite 3.50.4 I reproduced `duplicate column name: b` (matched, swallowed) vs `no
such table: nope` (not matched, re-raised), so unrelated/corrupt-DB errors still surface. (4)
Partial mid-v4 interruption (only one of two ALTERs landed) re-applies cleanly to v5 ? verified,
confirming the repeated reviewer nit is non-blocking. (5) Pre-LGTM gate: full suite 847 passed /
7 skipped, ruff clean and mypy clean on both touched files; PR's 80.67% coverage vs 70 floor is
consistent with the green full suite. Happy path is byte-identical (NULL=local behavior
unchanged, version contract tests green). The only findings across all four lenses are optional
test-coverage nits, no genuine must-fix."
      },
      {
        "pr": "pr213-scripts-unwrap",
        "verdict": "merge",
        "must_fixes": [],
        "should_fixes": [
          "Prose accuracy (non-blocking, no code change): the commit subject \"motion crashes
the default path\" and tests/test_results.py:32-33 docstring imply both scripts default to
motion. Only scripts/run_phase3_eval.py:54 defaults to motion (a real default-path crash);
scripts/run_process_and_stitch.py uses cfg.indexing.default_segmenter which defaults to
yolo_hybrid (bare list, no crash) ? its crash is latent, only under motion config. The inline
comment at run_process_and_stitch.py:82 ('for some segmenters (e.g. motion)') is already
correct.",
          "Optional follow-up (out of scope): fold the now-four hand-rolled Success/Failure
unwraps (backend/main.py:201/213 and :838/843, tools/generate_highlights.py:168) onto
unwrap_or_failure so there is a single unwrap implementation. tools/generate_highlights.py is
correct as-is (handles Failure, reads segments from the DB, never len()s the raw result).",
          "Optional test addition: pin unwrap_or_failure(Success(value=[])) -> ([], None)
explicitly, since run_process_and_stitch.py's empty-guard depends on that empty-success path
(verified correct at runtime here, but not directly asserted in tests/test_results.py)."
        ],
        "rationale": "Verified the fix independently end-to-end, not trusting the review claims.
Bug is real: motion.py returns Failure(31)/Success(value=...)(224), and the two scripts
len()/truthiness'd the raw dataclass -> TypeError; run_phase3_eval.py genuinely defaults to
motion. unwrap_or_failure faithfully mirrors backend/main.py's inline isinstance/.value unwrap
(both untouched, still pass); failure branching (continue/return) is correct. Verified at
runtime: Success(value=[])->([],None) feeds the empty-guard, Failure->(None,f), bare list passes
through, Success(value=None)->(None,None) caught by empty-guard. No other caller has the latent
bug (generate_highlights.py handles Failure and reads from DB). Gate: new tests 4/4 pass,
results.py 100% covered, ruff clean, mypy clean on CI scope (backend training; scripts/ not
type-gated), full suite 855 passed at 80.96% coverage (>=70% floor). The 2 local failures
(test_served_gate_a_regression.py) reproduce identically on origin/master and live in
rally_gate/windowing code this PR does not touch -> pre-existing and unrelated. All findings
across the four lenses dedup to three minor prose/DRY/test-coverage nits, none blocking."
      }
    ]
  }
}
</output>
```


195. assistant (2026-06-18T02:09:13.413Z)

Both PRs: **verdict=merge, 0 must-fixes** ? each independently re-verified (reverting database.py reproduces the exact `duplicate column name`; SQLite error-matching is correctly narrow; the scripts' default-segmenter analysis).

The reviews surfaced no blockers, but two cheap, legitimate improvements worth addressing before merge (my gate: resolve review findings):
1. **PR #213 accuracy fix (important per my no-overclaim rule):** the test docstring implies *both* scripts default to `motion`, but only `run_phase3_eval` does ? `run_process_and_stitch` defaults to `yolo_hybrid` (its crash is latent under a motion config). I'll correct that and add the `Success(value=[])` ? `([], None)` assertion the empty-guard depends on.
2. **PR #212:** add a tiny test for the re-raise branch (the one uncovered new line).

Let me make those small strengthening edits. First PR #213:

196. user (2026-06-18T02:09:21.307Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\tests\test_results.py has been updated successfully. (file state is current in your context ? no need to Read it back)

197. assistant (2026-06-18T02:09:36.131Z)

Now PR #212 ? add the re-raise branch test. Let me check its test imports and add the test:

198. user (2026-06-18T02:09:36.864Z)

```
1       import sqlite3
2       from backend.infrastructure.database import Database
3
```

199. user (2026-06-18T02:09:40.805Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\tests\test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

200. user (2026-06-18T02:09:51.190Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\tests\test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

201. assistant (2026-06-18T02:09:58.474Z)

Run the targeted tests + lint on both worktrees:

202. user (2026-06-18T02:10:02.878Z)

```
=== PR#212: test_database.py + ruff ===
.....................                                                    [100%]
21 passed in 0.92s
All checks passed!
```

203. user (2026-06-18T02:10:04.139Z)

```
=== PR#213: test_results.py + ruff ===
.....                                                                    [100%]
5 passed in 0.08s
All checks passed!
```

204. assistant (2026-06-18T02:10:13.147Z)

Both targeted suites green (+1 test each). Now the full gate on both (the follow-ups are test-

only additions, but I'll match CI):

205. user (2026-06-18T02:12:11.620Z)

```
########## PR#212 full gate ##########
All checks passed!
Success: no issues found in 104 source files
848 passed, 7 skipped, 7 warnings in 109.28s (0:01:49)
========================================================================
 RESULT: PASS - all tests green
========================================================================
```

206. user (2026-06-18T02:13:56.249Z)

```
########## PR#213 full gate ##########
All checks passed!
Success: no issues found in 104 source files
851 passed, 7 skipped, 7 warnings in 93.69s (0:01:33)
========================================================================
 RESULT: PASS - all tests green
========================================================================
```

207. assistant (2026-06-18T02:14:07.807Z)

Both full gates green (PR #212: 848 passed; PR #213: 851 passed; ruff + mypy clean on each). Let me commit the follow-ups, push (CI re-runs), then merge each on green. First PR #212:

208. user (2026-06-18T02:14:14.090Z)

```
fix/db-migration-idempotent-alters
M  tests/test_database.py
[fix/db-migration-idempotent-alters e720483] test(db): cover the idempotent-ALTER re-raise
branch
 1 file changed, 17 insertions(+)
e720483 test(db): cover the idempotent-ALTER re-raise branch
d140722 fix(db): make v3/v4 migration ALTERs idempotent so an interrupted upgrade can't brick
the DB
To https://github.com/avidullu/badminton-highlight-indexer.git
   d140722..e720483  fix/db-migration-idempotent-alters -> fix/db-migration-idempotent-alters
```

209. user (2026-06-18T02:14:25.788Z)

```
fix/scripts-unwrap-segmenter-result
warning: in the working copy of 'tests/test_results.py', LF will be replaced by CRLF the next
time Git touches it
M  tests/test_results.py
[fix/scripts-unwrap-segmenter-result 11d779f] test(results): pin empty-Success unwrap + correct
the motion-default prose
 1 file changed, 13 insertions(+), 2 deletions(-)
11d779f test(results): pin empty-Success unwrap + correct the motion-default prose
03d11d9 fix(scripts): unwrap the segmenter Result before len()/truthiness (motion crashes the
default path)
To https://github.com/avidullu/badminton-highlight-indexer.git
   03d11d9..11d779f  fix/scripts-unwrap-segmenter-result -> fix/scripts-unwrap-segmenter-result
```

210. assistant (2026-06-18T02:14:32.695Z)

Both follow-ups pushed; CI re-runs. Let me watch PR #212 to green.

211. user (2026-06-18T02:16:40.600Z)

```
Mypy (lenient baseline) pass    26s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27732272522/job/82041727034
Ruff lint       pass    9s      https://github.com/avidullu/badminton-highlight-
```

```
indexer/actions/runs/27732272522/job/82041726999
Syntax sweep + import smoke (no GPU stack)      pass    19s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732272522/job/82041727031
Full test suite (offline, mocked)       pass    2m7s    https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27732272522/job/82041727015
Golden fixtures cross-OS (macos-latest) pass    19s     https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27732272522/job/82041727058
Golden fixtures cross-OS (ubuntu-latest)        pass    21s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732272522/job/82041727080
Golden fixtures cross-OS (windows-latest)       pass    52s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732272522/job/82041727053
Mypy (lenient baseline) pass    26s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27732272522/job/82041727034
Ruff lint       pass    9s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27732272522/job/82041726999
Syntax sweep + import smoke (no GPU stack)      pass    19s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732272522/job/82041727031
=== exit 0 ===
{"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE"}
```

212. assistant (2026-06-18T02:16:44.200Z)

```
PR #212 fully green (CLEAN) and review verdict was merge/0-must-fix. Merging it (squash):
```

213. user (2026-06-18T02:16:52.067Z)

```
failed to delete local branch fix/db-migration-idempotent-alters: failed to run git: error:
cannot delete branch 'fix/db-migration-idempotent-alters' used by worktree at
'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/db-migration-fix'

=== verify ===
{"mergeCommit":{"oid":"c7baf03217a2d78625e0e58fa90b79d90daf233c"},"state":"MERGED"}
   4bd9d2d..c7baf03  master     -> origin/master
c7baf03 fix(db): make v3/v4 migration ALTERs idempotent (interrupted upgrade can brick the DB)
(#212)
```

214. assistant (2026-06-18T02:16:58.522Z)

```
PR **#212 MERGED** (`c7baf03`), master advanced. Now PR #213 ? watch its CI (re-ran after the
follow-up) to green:
```

215. user (2026-06-18T02:17:02.151Z)

```
Ruff lint       pass    11s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27732279062/job/82041746755
Syntax sweep + import smoke (no GPU stack)      pass    24s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732279062/job/82041746754
Full test suite (offline, mocked)       pass    2m22s   https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27732279062/job/82041746771
Golden fixtures cross-OS (macos-latest) pass    15s     https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27732279062/job/82041746773
Golden fixtures cross-OS (ubuntu-latest)        pass    21s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732279062/job/82041746756
Golden fixtures cross-OS (windows-latest)       pass    47s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732279062/job/82041746779
Mypy (lenient baseline) pass    31s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27732279062/job/82041746777
Ruff lint       pass    11s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27732279062/job/82041746755
Syntax sweep + import smoke (no GPU stack)      pass    24s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732279062/job/82041746754
=== exit 0 ===
{"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE"}
```

216. assistant (2026-06-18T02:17:05.633Z)

PR #213 fully green (CLEAN). Merging it (squash):

217. user (2026-06-18T02:17:13.342Z)

failed to delete local branch fix/scripts-unwrap-segmenter-result: failed to run git: error: cannot delete branch 'fix/scripts-unwrap-segmenter-result' used by worktree at 'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap'

```
=== verify ===
{"mergeCommit":{"oid":"bd330855302b92be57179b30443d32335a0d2a52"},"state":"MERGED"}
   c7baf03..bd33085  master     -> origin/master
bd33085 fix(scripts): unwrap segmenter Result before len() (motion crashes the default eval
path) (#213)
c7baf03 fix(db): make v3/v4 migration ALTERs idempotent (interrupted upgrade can brick the DB)
(#212)
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
```

218. assistant (2026-06-18T02:17:21.842Z)

Both PRs **MERGED** ? master is now `bd33085 (#213)` ? `c7baf03 (#212)` ? `4bd9d2d (#209)`. Let me clean up the worktrees/branches and check on the chip PR #210.

219. user (2026-06-18T02:17:27.228Z)

```
removed db-migration-fix
removed scripts-unwrap
removed hunt-base
Deleted branch fix/db-migration-idempotent-alters (was e720483).
Deleted branch fix/scripts-unwrap-segmenter-result (was 11d779f).
=== chip PR #210 status ===
{"mergeStateStatus":"UNKNOWN","state":"OPEN","title":"fix(eval): fusion loaders skip windowing-
only files on a mixed golden corpus"}
=== remaining fix/* worktrees (should be none of mine) ===
(all my worktrees cleaned)
```

220. assistant (2026-06-18T02:17:39.800Z)

All my worktrees and branches are cleaned; the chip PR #210 remains its session's responsibility. Let me record this in the live checkpoint.

221. user (2026-06-18T02:17:57.673Z)

<tool_use_error>File has been modified since read, either by the user or by a linter. Read it again before attempting to write it.</tool_use_error>

222. assistant (2026-06-18T02:18:01.403Z)

The memory file changed since I read it (likely another session). Let me re-read the top and re-apply.

223. user (2026-06-18T02:18:02.765Z)

```
1       ---
2       name: current-state
3       description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not
only on request). What just landed, what's next, open decisions."
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
8       ---
9
```

10   # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12   **Checkpoint:** 2026-06-18 (? **ABLATION LITMUS MIXED-SCHEMA FIX ? MERGED #209 (`4bd9d2d`), issue #208 closed**). The pre-existing `test_ablation.py` litmus `KeyError: 'shuttle'` (flagged 2026-06-17 as OUT-OF-SCOPE `task_5878d224`) is FIXED + shipped to master. Root cause: `ablation.load_videos()` read `c["shuttle"]`/`c["person"]` unconditionally, so once R3/R4 corpus growth added 5 clips persisted via the velocity-windowing/distill path (leaner rows, no cue dicts: Badminton_BXH_2, GX010128, GX030094, mahadevpura_1, mahadevpura_2) it crashed whenever the full n=11 corpus sat in `output/` (the 6 original fusion-schema files: adarsh_avi_singles, gbaaddy, kushagra_singles, largetest_doubles, mahadevpura_singles, testlarge_short). Fix (option a, most-robust): `_is_fusion_schema()` predicate ? loader SKIPS leaner files (info-log, no crash); `_golden_present()` counts only fusion files so the skip-gate matches the loader. Litmus still reproduces Gate-A (?F1?0.074, 6/6, p?0.031 ? the 6 fusion files ARE the original n=6). +2 regression tests. **DUE DILIGENCE:** local full suite 848 pass/5 skip + ruff + mypy(104) + cov 80.66% (floor 70); **GitHub CI all 8 checks green**; **adversarial 4-lens review workflow ? verdict=merge, 0 must-fix** (independently re-verified the bug, the litmus reproduction, the 2-file scope). Owner reported issue #208 + opened PR #209 + squash-merged on green. Worktree cleaned; local+remote branch deleted. ? **Sibling latent bug still open (`task_8dd4efde`):** `fusion_compare.load_videos` + `fusion_audio.load_videos_with_audio` have the IDENTICAL unconditional `c["shuttle"]` access ? not caught by any test (synthetic tmp corpora only) but would crash the same CLIs on the mixed corpus. Separate PR (the review independently confirmed this scope call). Possible follow-up nit: litmus `>=6`-gate vs `==6`-assert would go red if a 7TH fusion clip is ever added (pre-existing; this fix improved the opposite drift). [[eval-dual-set-regression]] [[working-agreement-merging]]
13
14   **Checkpoint:** 2026-06-18 ~00:10 (? **REMAINING-3 4K REELS DONE ? ALL 6 BOXHILL-17 CLIPS NOW HAVE 4K** + cost-scaling doc + 10/10 visual QA ? all on G:). Owner asked for "just 4K highlights of the remaining 3, all local no Gemini." Delivered:
15   - ? **GX010138/139/140 4K reels** (the larger 9/10/12.8-min clips): **45/47/68 rallies**, 1.77/1.89/2.72 GB, 3840×2160, watermarked. Published to G: as `? - Highlights (4K).mp4`. Robust detached Scheduled Task `KhelSutraBoxhill4KRemaining` (keep-awake + self-restart; unregistered on completion).
16   - ? **DOGFOODED `--detect-from` end-to-end on fresh videos** (the merged #205 feature): `--video <4K original> --detect-from <1080p proxy>` ? detect on proxy, stitch 4K from original, reel named after --video (NO `_1080p` naming bug this time). Log confirmed "detecting on proxy ? stitching reel from ?". Validated beyond the unit tests.

224. user (2026-06-18T02:18:24.559Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

225. assistant (2026-06-18T02:18:48.264Z)

Done. Beyond the chip (which its own session owns), I hunted for other fixes, found three real defects, and shipped them as two merged PRs with full due diligence.

What I shipped

PR	Fix	Severity
[#212](https://github.com/avidullu/badminton-highlight-indexer/pull/212) ? `c7baf03`	**DB can't be permanently bricked by an interrupted migration.** v3/v4 used bare non-idempotent `ALTER ... ADD COLUMN` (v1/v2/v5 are `IF NOT EXISTS`). A DDL ALTER auto-commits independently of the surrounding transaction, so a crash *after* the ALTER but *before* the end-of-block `PRAGMA user_version` bump leaves the column present + version stale ? next open re-runs the ALTER ? `duplicate column name` ? `__init__` throws **forever**. Fixed with a narrow `_add_column_idempotent` helper.	High (permanent lockout)
[#213](https://github.com/avidullu/badminton-highlight-indexer/pull/213) ? `bd33085`	**`len()`/truthiness on a raw segmenter `Result`.** `run_phase3_eval.py` (default `--segmenter motion`, the documented baseline) and `run_process_and_stitch.py` crashed with `TypeError` on the `Success`/`Failure` dataclass; the empty-guard was dead code. Fixed with a new DRY `backend/results.unwrap_or_failure()` mirroring `main.py`'s unwrap.	Medium

How I found them

A two-pass adversarial bug-hunt over `origin/master` (~15 agents): **pass 1 (by-area) returned zero** ? the codebase is genuinely clean; **pass 2 (by bug-class + a completeness critic)** surfaced 3 candidates, all of which survived diverse refutation. I verified each against the actual code myself (reverting the DB fix reproduces the exact `duplicate column name`; the script crash confirmed against the real motion segmenter) ? not trusting the agents' descriptions.

Due diligence on each (your pre-LGTM gate)

- Local: full suite green (848 / 851), `ruff` clean, `mypy` clean (104 files), coverage ~80.7% (floor 70). The DB regression test is **proven to fail without the fix**.
- GitHub CI: all 8 checks green on both.
- Adversarial 4-lens review workflow per PR ? **both `verdict=merge`, 0 must-fixes**, independently re-verified.
- I addressed the reviews' minor should-fixes *before* merging (a re-raise-branch test for #212; an empty-`Success` test and a no-overclaim docstring correction for #213).
- Squash-merged, worktrees/branches cleaned.

Deliberately not merged

- **Chip [#210](https://github.com/avidullu/badminton-highlight-indexer/pull/210)** (sibling `fusion_compare`/`fusion_audio` loaders) ? still open and owned by its spawned session; I stayed off those files to avoid racing it on the shared checkout. Flag me if you'd like me to take it over.
- Two **non-bug cleanups** the hunt noted (left as-is): folding the four hand-rolled `Success`/`Failure` unwraps onto the new helper, and the pre-existing litmus `>=6`/`==6` fragility from the #209 review. Say the word if you want either.

One incidental finding: this dev container actually **has numpy/scipy/cv2/torch** now (the `CLAUDE.md` "lacks numpy/cv2/torch" note is stale), which let me verify the pure-logic and segmenter-`Failure` paths locally ? the full GPU pipeline still can't run here.

226. user (2026-06-18T07:40:44.093Z)

So all the pending PRs are merged. From your context is there anything gated on me or we good to pick up new work?

227. assistant (2026-06-18T07:40:51.312Z)

Let me check the actual current state rather than answer from memory.

228. user (2026-06-18T07:40:58.741Z)

```
=== OPEN PRs ===

=== recently merged (last 6) ===
218  docs(recording): assert single-match capture as the #1 rule (now MEASURED)
217  feat(eval): nightly all-OFF floor ? pin a baseline_off reference config
216  chore(eval): re-baseline the nightly after the R1 milestone flip
215  refactor(eval): single source of truth for the _is_fusion_schema predicate
214  docs(archive): LOVO threshold re-sweep at n=13 ? milestone thresholds hold
213  fix(scripts): unwrap segmenter Result before len() (motion crashes the default eval path)

=== origin/master HEAD ===
a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
b460526 docs(recording): assert single-match capture as the #1 rule (now MEASURED) (#218)
ab03043 refactor(eval): single source of truth for the _is_fusion_schema predicate (#215)
ff51b37 chore(eval): re-baseline the nightly after the R1 milestone flip (#204) (#216)
```

229. assistant (2026-06-18T07:41:17.903Z)

Zero open PRs ? and a lot has merged since (other sessions/you have been busy: #214?#218, including the R1 flip `#204`). Let me confirm the chip's fate and the local checkout state before I answer.

230. user (2026-06-18T07:41:24.033Z)

```
=== chip PR #210 (sibling loaders) final state ===
#210 MERGED ? fix(eval): fusion loaders skip windowing-only files on a mixed golden corpus
=== #215 (single-source _is_fusion_schema) ? did it absorb the sibling-loader fix? ===
```

Why

Follow-up diligence cleanup. PRs [#209](https://github.com/avidullu/badminton-highlight-indexer/pull/209) (ablation litmus) and [#210](https://github.com/avidullu/badminton-highlight-indexer/pull/210) (fusion compare/audio loaders) each landed an **independent, byte-identical** copy of `_is_fusion_schema` ? the guard that lets the golden-feature loaders skip leaner velocity-windowing files on a mixed corpus. The duplication was deliberate at the time (to avoid stacking #210 on the then-unmerged #209). Both are now merged on `master`, so it can be retired.

What

Consolidate to **one definition** in `backend/eval/experiment.py`, placed next to `VideoCandidates` ? the type these loaders build, and the module all three already import. This means:
- **No new import edges** (`ablation`, `fusion_compare`, `fusion_audio` already import `experiment`).
- **No circular-import risk** ? `experiment` imports none of the three consumers.

The three loaders ? `ablation.load_videos`, `fusion_compare.load_videos`, `fusion_audio.load_videos_with_audio` ? now import the shared predicate; **their bodies are unchanged** and the predicate logic is identical. Pure refactor, **no behaviour change**.

Details:
- Dropped the now-unused `Sequence` import from `fusion_compare` (it was added only for the local def). `ablation` keeps `Sequence` (still used in 5 other signatures).
- Repointed `fusion_audio`'s predicate import from `fusion_compare` ? `experiment` (single source).
- Moved the predicate's **canonical unit test** to `tests/test_experiment.py` (its new home) and removed the duplicate from `tests/test_ablation.py`. `test_ablation`'s `_golden_present` keeps referencing `ablation._is_fusion_schema` ? valid via the re-import, consistent with the existing `_derive_video_path` cross-module pattern.

Verification (local)

- `python run_all_tests.py --cov=backend --cov=training --cov-fail-under=70` ? **855 passed, 7 skipped**, coverage **80.70%** (?70 floor, no regression).

```
=== local main checkout: branch + dirty? (clean-slate / nightly self-activation) ===
docs/golden-regression-fixtures-plan
```

docs/golden-regression-fixtures-plan...origin/docs/golden-regression-fixtures-plan [ahead 18, behind 1]

```
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
=== other live worktrees still around ===
20
```

231. assistant (2026-06-18T07:42:03.871Z)

Good ? full picture now. Notable: the **chip #210 merged** (sibling loaders), **#215** already consolidated the duplicated `_is_fusion_schema` (one of my flagged cleanups ? done by another session), and the **owner-gated R1 flip `#204` merged** (with #216/#217 re-baselining the nightly). The corpus also grew to **n=13** (#214). Let me read the authoritative next-steps from master to pin down what's genuinely owner-gated.

232. user (2026-06-18T07:42:04.279Z)

```
=== golden corpus size + any owner-gated dispatch in NEXT_STEPS (from master) ===
4:owner-gated ? see the dedicated section below, the desktop-agent pick-up). Prior: 2026-06-10
(audit?M0?Gemini-battery session; later same day the **UI-alpha track opened**:
```

5-ADR-009 ratified ? KhelSutra Guru / khelsutra.guru, private `khelsutra` repo, local-first delivery ? see
--
16:**M0 in-container quick wins are COMPLETE:** ship-gate provisional blocker (**#170**), train/eval split guard (**#171**), the **doc-consistency sweep** (**#174** ? corrected 16 drifted docs vs the code), and the **P3 person-feature fusion litmus machinery is already present + suite-green** (`backend/eval/fusion_compare.py` + `ablation.py`: person-driven ?F1 / bootstrap CI / paired sign test; the real-golden `skipif` litmus waits only on GPU-extracted data). ? CI is red repo-wide due to a **GitHub Actions minutes/billing outage** (account-level, not code) ? work is local-gated + `--admin`-merged until Actions is restored (Settings ? Billing ? Actions minutes / spending limit).
17-
--
19:1. **[owner / GPU box]** `python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json --out-dir "%USERPROFILE%\OneDrive\rally-annotator\golden-data\features\2026-06-15-n6"` ? produces `*_golden_features.json` for the 6 golden videos. (GPU + WSL + videos + trajectory CSVs + `.rallies.csv` labels are owner-side only.)
20-2. Artifacts land in the **shared cloud folder** `C:\Users\avidu\OneDrive\rally-annotator\golden-data\` (OneDrive auto-syncs both ways). Convention + workflow: **[`GOLDEN_DATA_SHARING.md`](GOLDEN_DATA_SHARING.md)**; tracked in **#175**.
21:3. **[CPU dev/agent session]** `python -m backend.eval.fusion_compare --features-dir <shared>\features\2026-06-15-n6 --record` + `python -m backend.eval.ablation --features-dir <shared>\... --granularity group` ? real person-feature ?F1; copy the JSONs into `output/` to also pass the `skipif` litmus tests (`tests/test_ablation.py::test_litmus_reproduces_gate_a`).
22-
23:**Also queued (owner-gated / pending input):**
24:- Ship-gate target calibration ? **#172** (owner decision; needs corpus growth).
25:- Tests reorganization ? blocked on the owner's grouping metric (provide it ? an agent will do it).
26-- Restore CI ? the Actions outage is account-level (billing/minutes), not a repo/config bug.
--
28:**Leaving (bulky / M2 ? no start without owner go):** ActionFormer (M0.4), event-index DB migration (M0.1), Gemini-silver purge (M0.3), cost-to-Gen-2 benchmark + ByteTrack deprecation, multisport, repo rename, PMF execution.
29-
--
54:## Rally-detection quality track (design merged 2026-06-13, owner-gated; implementation in progress)
55:**This is the track a desktop/local agent is slated to pick up.** Design docs landed (PR #112 + session). All implementation is **owner-gated** (per explicit in the plan); each step = ONE PR off `master`.
56-
--
60: `docs/EXPERIMENT_HARNESS.md` (A/B + shadow + promotion gate, zero-regression) ·
61- `docs/ROORA_LABELING_GUIDELINES.md` (v8) · `docs/HOW_RALLY_DETECTION_WORKS.md` (2-layer mental model).
62:- **Key finding (no re-investigation needed):** the owner's "held-shuttle counted as a rally" bug is **NOT a missing capability** ? `backend/pipeline/detectors/rally_gate.py` already implements the net-crossing **"Gate A"** (`apply_gate(..., min_crossings=2)`) that kills service-feed/held-shuttle windows, but it is **only called from eval drivers** (and there was no `min_crossings` knob in `IndexingConfig`). **Wiring Gate A into the live segmenter (now via fusion_hybrid) was the single cheapest, highest-confidence quality win.** (P2 FusionSegmenter specifics: registry `fusion_hybrid`, **default-OFF**, seeds from substrate, persists `net_crossings` + optional person features, optional served Gate A/B via config knobs; adversarially reviewed; suite green. See `tests/test_served_gate_a_regression.py` for the honest finding that on production windowing Gate A is neutral/negative in some cases, hence kept opt-in with knob for safety, and the leaderboard revert note.)
63:- **Current status (2026-06-14):** P1 (person cue extraction to reusable `PersonDetector`) + P2 (FusionSegmenter + Gate A signal + served handoff gating) + experiment harness P1 DONE (adversarial review + NO-REGRESSION; suite green). **P3 next (this PR + follow-ups):** learned fusion (person features ? harness `compare` LOVO lift on the 6 golden videos; eager-person-compute optimisation); then P4 audio via hit_detector.
64:- **Pick-up order (each = ONE PR off `master`, owner approval first, flag-gated default-OFF, promote only on measured LOVO):**
65: 1. (P2 first-step, largely done via fusion_hybrid) Wire Gate A into production served path

```
+ config knob (min_crossings); apply in hybrids at runtime. (Now available opt-in; served
neutral on production windowing per litmus ? kept opt-in with knob for safety.)
66-  2. P3 first step (this work item): harness `compare` LOVO litmus for person-on variant
(players_in_court + two_sided + colocation etc. from `fusion_features`) on the 6 golden; add the
eager compute optimisation note/landing. Pure + harness (testable here).
--
68:- **Discipline (from EXPERIMENT_HARNESS.md):** every new cue **flag-gated, default OFF**;
promote to default **only on measured LOVO lift** through `run_regression.py` (exit 0/1/3);
pinned `CONTROL` ? rollback = config flip. Harness re-uses ~80% existing code.
69:- ? Dev container no GPU/torch/cv2 ? Gate A, person cue logic, harness, and pure fusion paths
are testable here; real-video / golden LOVO / ablation confirmation is owner-side (GPU/WSL
machine).
70:- **External due-diligence review inputs (2026-06-17) ? PICK UP AFTER current ideas are
exhausted; full details + citations in [`RALLY_QUALITY_RESEARCH.md`](RALLY_QUALITY_RESEARCH.md)
§5.6.** An owner-commissioned 31-reference review of the quality report *validated* the
methodology and surfaced these **future** levers (research inputs, not active work, not new
claims; verify citations before any submission):
71-  1. **"First-mile" paper framing (highest-value):** stroke-classifiers (BST/DSTA) + the big
datasets (ShuttleSet ~36k strokes, Fine-Badminton ~27k actions) all **assume a pre-trimmed rally
clip already exists** ? we solve the ignored prerequisite. Pitch the paper as *"the missing
untrimmed-video preprocessing pipeline that lets BST/MonoTrack run on raw footage."*
--
78:**Discipline reminder:** owner approval before starting any owner-gated item; one PR per
step; babysit the PR (poll/fix/reply) until merge observed, *then* advance.
79-
--
81:1. **[owner, in progress] GROW THE GOLDEN CORPUS** ? the single highest-leverage move; every
lever below feeds on it.
82-   Priority order for new videos: **(a) multi-court badminton** (the target distribution AND
where the ship target
--
88:2. **[owner decision] Set the ship-gate target** ? now informed: floor = heuristic 0.480
(necessary), **multi-court
89-   reference = wrapper 0.66** (the number that makes the owned model worth serving), clean-
footage ceiling = 0.93
--
91:   paired-sign p<0.05 vs heuristic" ? owner to ratify/adjust.
92-3. **M0.4 next increment ($0/local):** swap the Gen-0 LogReg for a lightweight temporal head
(MIT, minutes on the
--
98:   badminton+TT once TT has ?3 matches; identify a clean pickleball ball detector. ? TT-on-
WASB serving stays gated
99-   by C3 (provenance multiplies per sport).
--
```

233. assistant (2026-06-18T07:42:43.021Z)

```
Confirmed ? **we're good to pick up new work. Nothing is blocking me.**
```

State of play

```
- **Open PRs: 0.** Everything merged, including the two I shipped this session (#212, #213), the
chip **#210** (sibling loaders ? its own session landed it), and a wave of follow-ups from other
sessions: **#204** (the owner-gated R1 milestone flip!), #214 (LOVO re-sweep at **n=13**),
**#215** (consolidated the duplicated `_is_fusion_schema` into one source ? one of the cleanups
I'd flagged, now done), #216/#217 (nightly re-baseline + all-OFF floor), #218 (docs).
- No worktrees or branches of mine left dangling.
```

Gated on *you* ? decisions/inputs only you can give (each unblocks a *specific* track; none blocks general w

```
- **IAA 2nd-annotator study** ? the one thing still blocking **R2 ? calibration** (the tolerance
thresholds). I can't manufacture inter-annotator agreement ? needs a second human labeling 2?3
golden clips.
- **Ship-gate target ratification (#172)** ? your call on the F1 bar (floor 0.48 / multi-court
ref 0.66 are computed; needs your sign-off).
- **Any owner-gated track *start*** ? the platform async-job slice and the owned-model Phase-0
```

both say "no start without owner go." If you want either next, just say go.
- **More golden data / GPU-side extraction** ? owner-side (videos + GPU + WSL), only if we want to grow past n=13.

One housekeeping flag (not blocking)

The local **main checkout is on `docs/golden-regression-fixtures-plan`** (ahead 18 / behind 1 of its remote), not `master` ? so per your clean-slate convention the nightly-regression task stays **dormant** until it's back on master, and that branch has unmerged commits worth a decide-or-drop. There are also ~20 lingering worktrees from past sessions I could prune.

What I can pick up right now (no gating)

1. **Fresh bug-hunt on the new master** ? it moved a lot (R1 flip, nightly baselines, #215 refactor); new code = worth another adversarial pass.
2. **Small cleanup** ? fold the remaining hand-rolled `Success`/`Failure` unwraps (`main.py` ×2, `generate_highlights.py`) onto the new `unwrap_or_failure`, and the litmus `>=6`/`==6` fragility.
3. **Tidy-up** ? prune stale worktrees / get the main checkout back on master so the nightly re-activates.

Want me to take one of those, or do you have something specific in mind?

234. user (2026-06-18T07:44:59.824Z)

why just one why not all the three! I have been meaning to pause development and do a refactoring effort seeing that a bunch of files have grown a lot, so please find bugs and also some critical tech debt which we can clean up , just make the PRs and ensure teh quality does not regress and I will take a look at them!

235. assistant (2026-06-18T07:45:46.533Z)

Love it ? let's do all three, plus the refactoring sweep you're after. To be clear on the mode: I'll **open PRs for your review and not merge them** (you said you'll take a look), each one its own branch off `master`, with the full quality gate (suite + ruff + mypy + coverage) green so nothing regresses.

Let me start by syncing to the new master and mapping where the bloat actually is, so I target the *critical* tech debt rather than guess.

236. user (2026-06-18T07:45:52.346Z)

```
Preparing worktree (detached HEAD a010667)
HEAD is now at a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config
(#217)
=== HEAD ===
a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
=== top 30 largest source files (LOC) ===
 18860 total
  1113 backend/eval/golden_fixtures.py
   871 backend/main.py
   738 backend/infrastructure/database.py
   699 backend/pipeline/detectors/wasb_infer.py
   633 scripts/nightly_regression.py
   620 backend/eval/ablation.py
   579 backend/eval/experiment.py
   506 backend/pipeline/detectors/tracknet_runner.py
   472 backend/pipeline/segmenters/gemini.py
   392 backend/pipeline/stitcher/compiler.py
   380 backend/eval/rally_seg_eval.py
   369 backend/pipeline/segmenters/yolo_hybrid.py
   357 backend/pipeline/detectors/person_detector.py
   337 backend/pipeline/segmenters/trajectory_hybrid.py
   334 backend/config/models.py
   291 backend/eval/per_user_regression.py
```

```
   277 backend/eval/tolerance_metrics.py
   233 backend/eval/gemini_refine.py
   230 backend/eval/export_wasb_segments.py
   227 backend/pipeline/segmenters/motion.py
   226 backend/eval/segmentation_metrics.py
   224 training/gen0/harness.py
   223 backend/reporting/harvest.py
   218 backend/eval/eval_partial_labels.py
   216 backend/utils/run_telemetry.py
   215 backend/pipeline/detectors/fusion_features.py
   210 backend/eval/windowing.py
   209 backend/eval/per_user_gate.py
   203 backend/eval/promotion.py
   199 backend/pipeline/detectors/wasb_runner.py
   199 backend/eval/served_gate_a.py
```

237. assistant (2026-06-18T07:46:14.680Z)

Clear bloat map: `golden_fixtures.py` (1113), `main.py` (871), `database.py` (738), `wasb_infer.py` (699), `ablation.py` (620), `experiment.py` (579). Let me also get a function-length signal (the real debt is often a giant function inside a file) before launching discovery.

238. user (2026-06-18T07:46:20.785Z)

```
=== 22 longest functions (len | branchy-nodes | location) ===
 434 |  71 | backend\pipeline\segmenters\gemini.py:35 process_video
 309 |  45 | backend\pipeline\segmenters\yolo_hybrid.py:57 process_video
 231 |  30 | backend\pipeline\segmenters\trajectory_hybrid.py:106 process_video
 204 |  24 | backend\pipeline\segmenters\motion.py:20 process_video
 202 |  36 | backend\pipeline\stitcher\compiler.py:190 compile_highlights
 187 |   6 | backend\infrastructure\database.py:62 _init_db
 186 |  13 | backend\eval\golden_fixtures.py:598 freeze_real_fixture
 185 |  25 | backend\pipeline\detectors\wasb_infer.py:359 run
 168 |  23 | backend\main.py:700 main
 132 |  16 | backend\pipeline\detectors\tracknet_runner.py:256 trajectory_to_action_windows
 128 |   8 | backend\eval\per_user_gate.py:81 should_promote_for_user
 122 |   7 | backend\eval\per_user_regression.py:169 evaluate_no_regression
 116 |  13 | backend\pipeline\detectors\person_detector.py:140 extract_action_windows_from_chunk
 115 |   1 | backend\eval\ablation.py:457 export_pdf
 112 |  10 | backend\main.py:123 _execute_processing
 109 |  18 | backend\eval\rally_seg_eval.py:267 main
 107 |  18 | backend\providers\gemini.py:38 analyze_video
 102 |  16 | backend\pipeline\validators\pts_continuity.py:14 validate
  95 |   8 | backend\pipeline\validators\relevance.py:15 validate
  93 |   7 | backend\eval\promotion.py:110 promote_if_served_safe
  90 |  14 | backend\pipeline\detectors\tracknet_runner.py:120 run_predict
  90 |  10 | backend\eval\golden_fixtures.py:1019 main
```

239. assistant (2026-06-18T07:46:54.362Z)

Two clear debt signals now: monster functions ? `gemini.py process_video` (**434 lines / 71 branches**), `yolo_hybrid` (309), `trajectory_hybrid` (231), `motion` (204), `compiler.compile_highlights` (202) ? and god-modules ? `golden_fixtures.py` (1113), `main.py` (871), `database.py` (738).

Let me run a discovery workflow: a fresh bug-hunt on the changed master **plus** a tech-debt analysis that proposes only *safe, behavior-preserving, well-tested* refactors (each rated by risk + test coverage), then synthesizes a prioritized PR worklist.

240. user (2026-06-18T07:47:49.160Z)

Workflow launched in background. Task ID: wwq5a0lfa
Summary: Find real bugs + propose safe high-value refactors on the grown files; prioritized PR

worklist
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e5140909-890
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer--claude-worktrees-refactor-base\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bughunt-and-techdebt-discovery-wf_e5140909-890.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer--claude-worktrees-refactor-base\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bughunt-and-techdebt-discovery-wf_e5140909-890.js"} to iterate without resending the script.)
Run ID: wf_e5140909-890
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer--claude-worktrees-refactor-base\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bughunt-and-techdebt-discovery-wf_e5140909-890.js", resumeFromRunId: "wf_e5140909-890"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

241. assistant (2026-06-18T07:47:51.639Z)

Discovery is running across the codebase. Let me wait for the prioritized worklist.

242. user (2026-06-18T07:57:52.355Z)

<retrieval_status>timeout</retrieval_status>

<task_id>wwq5a0lfa</task_id>

<task_type>local_workflow</task_type>

<status>running</status>

243. user (2026-06-18T08:01:36.508Z)

<retrieval_status>success</retrieval_status>

<task_id>wwq5a0lfa</task_id>

<task_type>local_workflow</task_type>

<status>completed</status>

<output>

```
{
  "summary": "Find real bugs + propose safe high-value refactors on the grown files; prioritized PR worklist",
  "agentCount": 18,
  "logs": [
    "Bugs: 4 candidate(s)",
    "Bugs confirmed: 1/4",
    "Debt: 18 proposal(s)"
  ],
  "result": {
    "plan": {
      "prs": [
        {
          "rank": 1,
          "type": "bug",
          "title": "config loader: degrade to defaults+warn when config.json is valid JSON but not an object",
          "files": "backend/config/loader.py (load_config, ~lines 65-86); tests: tests/test_config.py",
          "summary": "load_config crashes at process startup when config.json parses to a non-
```

</output>

```
object (e.g. [], 42, \"x\"). json.load() succeeds so the read/parse try/except does not catch it; then for a non-empty non-mapping, _unknown_key_paths runs .items() (AttributeError), and for any non-mapping the {**defaults, **file_data} unpack raises TypeError ('list' object is not a mapping). Reproduced locally with config.json='[]' -> TypeError. This violates the loader's documented contract (docstring lines 60-61: missing/unreadable -> fully-defaulted AppConfig, bad input non-fatal) and turns an operator typo into a hard server/CLI startup crash via main.py::main -> load_app_config.",
          "plan": "Immediately after json.load (before the `if file_data:` block), coerce a non-dict result to the no-op path: `if not isinstance(file_data, dict): logger.warning(f\"{cfg_path} is not a JSON object; ignoring and using defaults.\"); file_data = {}`. This restores the 'defaults + warn, never fatal' behavior for both the falsy ([], 0, \"\") and truthy ([1], \"x\", 42) non-object cases in one guard. No other control flow changes.",
          "risk": "low",
          "test_plan": "Add tests to tests/test_config.py: write config.json containing each of `[]`, `[1,2]`, `\"x\"`, `42`, `true`, `null`; assert load_config returns a default-equal AppConfig and does not raise, and that a warning is logged (caplog). Keep existing valid-object and missing-file tests green. Run full suite + ruff + mypy; coverage must stay >=70 (this adds a previously-uncovered branch, so coverage should rise slightly).",
          "recommend_now": true
        },
        {
          "rank": 2,
          "type": "refactor",
          "title": "database: extract per-table CREATE + each migration block out of _init_db into named helpers",
          "files": "backend/infrastructure/database.py (_init_db, ~lines 62-247, 185L) -> same module/class _create_base_tables(cursor) + _migrate_to_v1..v5(self, cursor); tests: tests/test_database.py, tests/test_async_jobs.py, tests/test_job_worker_loop.py",
          "summary": "_init_db is the longest method in the class: three base-table CREATEs plus five sequential `if version < N` migration blocks with embedded DDL, all inline. Verified the ladder is clean (v1->v5, each guarded by `if version < N`, each ending in `PRAGMA user_version = N`, idempotent ALTERs via _add_column_idempotent). The migration contract is heavily pinned by tests (schema-version-is-current, idempotent-and-upgrades-old-db, v3->v5-preserving-rows, idempotent-on-interrupted-upgrade).",
          "plan": "Pure intra-class extraction. Pull each `if version < N:` body verbatim into `_migrate_to_vN(self, cursor)` and the three base CREATEs into `_create_base_tables(cursor)`. _init_db becomes: open connection (same single `with self._connect()` block), create base tables, read PRAGMA user_version, call the migration helpers in order each still guarded by the SAME `if version < N` check, commit. No SQL text, ordering, PRAGMA bumps, or cursor/commit semantics change. All DDL still runs on one cursor inside one connection block to preserve the auto-commit/interruption behavior the tests pin.",
          "risk": "low",
          "test_plan": "Run tests/test_database.py (incl. the interrupted-upgrade and old-DB-upgrade migration tests), test_async_jobs.py, test_job_worker_loop.py ? all must stay green with zero output change. Confirm a fresh DB lands at the current user_version and an old (v3) DB upgrades preserving rows. Full suite + ruff + mypy; coverage flat-or-up (no new branches, in-module so import surface unchanged).",
          "recommend_now": true
        },
        {
          "rank": 3,
          "type": "refactor",
          "title": "stitcher/compiler: extract the per-segment trim loop body into VideoCompiler._trim_one_segment",
          "files": "backend/pipeline/stitcher/compiler.py (compile_highlights, ~lines 234-335); tests: tests/test_compiler.py, tests/test_watermark.py",
          "summary": "compile_highlights is the longest function in the detector/stitcher cluster (~202L), the user-facing reel path. ~100L of its body is one `for idx,(raw_start,raw_end) in enumerate(segments)` loop doing five inline things: timestamp parse, validate/clamp (negative start, start>=end, bounds vs video_duration), padded-bounds compute, watermark-then-plain ffmpeg retry, and success/skip recording. Verified loop bounds (lines 234-334) and the temp_clips/skipped_segments accounting.",
          "plan": "Move the loop body verbatim into a new instance method `_trim_one_segment(self, raw_video_path, idx, raw_start, raw_end, n_segments, video_duration, watermark_filter, tmp_dir) -> tuple[Optional[str], Optional[tuple]]` returning
```

(clip_path_or_None, skip_record_or_None). compile_highlights keeps the `for` loop and appends to temp_clips/skipped_segments exactly as today, calling the helper per segment. The base_cmd build, the `attempts = [vf_args, []] if vf_args else [[]]` retry order, all clamp arithmetic (padded_start=max(0,start-begin_padding), padded_end clamp), and every skip-reason string move unchanged. No constant/ordering change.",
          "risk": "low",
          "test_plan": "Run tests/test_compiler.py (clamp/skip branches, ordering, concat) and tests/test_watermark.py (watermark-first-then-plain fallback) ? temp_clips ordering and final concat must be byte-identical. Full suite + ruff + mypy; coverage flat-or-up. (Note: cv2/ffmpeg deep paths run machine-side; the mocked compiler tests cover the clamp/skip/argv logic this PR moves.)",
          "recommend_now": true
        },
        {
          "rank": 4,
          "type": "refactor",
          "title": "gemini segmenter: extract process_video into named private helpers (provider wiring, trim, chunked/single, validate-dedupe, db-populate)",
          "files": "backend/pipeline/segmenters/gemini.py (process_video, ~lines 35-469); tests: tests/test_gemini.py",
          "summary": "process_video is the longest method in the segmenter cluster (~434L, 71 branches) mixing 6 concerns: provider+key wiring, optional debug-trim, chunked parallel path, single-file path, a multi-rule validation/overlap-resolution pass, and DB population. Verified it returns a BARE LIST (`return segments_data` line 469; early `return []` at 46/62/68/303) ? the cross-file contract main.py lines 201/213/838/843 (`isinstance(result, Failure)` then `result.value if hasattr(result,'value') else result`) relies on. parse_time (line 309) is a self-contained nested closure, safely liftable. 8 mocked orchestration tests pin the contract.",
          "plan": "Same-class verbatim cut-paste extraction, no logic edits: (1) _resolve_provider(self) -> provider|None for the sport/provider/api-key/registry preamble, signaling the early-return-[] cases via None; (2) _maybe_trim_video(...); (3) _process_chunked(...)/_process_single(...) wrapping the two analyze branches incl. their summary-log builders; (4) lift parse_time to module-level _parse_time(val) (no free vars beyond logger) OR keep nested; (5) _validate_and_dedupe(self, ai_segments, video_duration); (6) _populate_db(self, video_id, validated, model_name) -> segments_data. process_video becomes a ~40L orchestrator. CRITICAL: keep the return type a bare list and every early `return []` preserved; no statement reordered, no log string/level changed, no branch condition altered.",
          "risk": "low",
          "test_plan": "Run tests/test_gemini.py (8 tests, mocked orchestration). Add/confirm an output-equivalence assertion if not already present (the returned segments_data list must be identical pre/post). Verify main.py still receives a bare list (the isinstance(result,Failure)/hasattr branch must take the list path). Full suite + ruff + mypy; coverage flat-or-up. Baseline was 30/30 green across the three segmenter test files.",
          "recommend_now": true
        },
        {
          "rank": 5,
          "type": "refactor",
          "title": "yolo_hybrid segmenter: extract process_video phases into named private helpers",
          "files": "backend/pipeline/segmenters/yolo_hybrid.py (process_video, ~lines 57-366); tests: tests/test_yolo_hybrid.py",
          "summary": "process_video is ~309L spanning Phase-2 candidate generation (near-duplicate pipelined vs sequential chunk loops), cross-chunk window dedup/merge, candidate persistence, Phase-3 parallel AI handoff, and Phase-4 DB population, all inline. Verified it returns a BARE LIST (`return segments_data` line 366; early `return []` at 66/82/88/95/288) ? same cross-file contract as gemini. 7 tests including an explicit output-equivalence guard (test_yolo_hybrid_segmenter), a candidate-persist/link test, and a store-disabled test.",
          "plan": "Same-class verbatim extraction: (1) _resolve_provider(self) -> provider|None; (2) _collect_candidate_windows(...) wrapping BOTH the pipelined and sequential loops + the post-loop overlap-merge (keep `_merge_stats` nested) ? do NOT merge the two branches (that would be a logic change), only move them as-is; (3) _persist_candidates(...); (4) _run_ai_handoff(...) wrapping the ThreadPoolExecutor + process_candidate_window (keep max_workers, os.remove/os.rmdir cleanup order); (5) _populate_db(...) -> segments_data. process_video becomes a phase-by-phase orchestrator; return type stays a bare list, no thresholds/log strings/dedup-merge math changed.",

```
          "risk": "low",
          "test_plan": "Run tests/test_yolo_hybrid.py (7 tests incl. the output-equivalence
guard, candidate-persist/link, store-disabled, no-API-key path). Confirm main.py still gets a
bare list. Full suite + ruff + mypy; coverage flat-or-up. Ship as a SEPARATE PR from the gemini
extraction (no stacking).",
          "recommend_now": true
        },
        {
          "rank": 6,
          "type": "refactor",
          "title": "tracknet_runner: dedup the temporal-midpoint split-index into a shared
static helper",
          "files": "backend/pipeline/detectors/tracknet_runner.py (_split_overlong_groups hard-
cap branch lines ~448-449; _blob_fallback_chop ~493-494); tests: tests/test_tracknet_runner.py",
          "summary": "The exact two-line 'find the group index nearest the temporal midpoint and
chop there' logic (`mid=(g[0][0]+g[-1][0])/2.0; split_i=min(range(1,len(g)), key=lambda i:
abs(g[i][0]-mid))`) is copy-pasted in the W1 hard-cap fallback and the R5 blob-fallback
(verified lines 448-449 plus the _blob_fallback_chop branch). Both carry the same `# noqa: B023`
closure note. A future chop-rule tweak must be made twice or they silently diverge.",
          "plan": "Add a private static `_midpoint_split_index(group) -> int` returning the
index nearest the temporal midpoint (body = the two existing lines verbatim), and replace the
two inline copies with a call. Both call sites already operate on a `group` of (frame_idx,
velocity) tuples used for `work.append(g[:i]); work.append(g[i:])`. min() with the identical key
over the identical range yields the identical split index, so the recursive partition is
unchanged.",
          "risk": "low",
          "test_plan": "Run tests/test_tracknet_runner.py (the frame-set partition invariant
`sorted(...)==list(range(...))` and the hard-cap/blob-fallback split tests). Output partition
must be identical. Full suite + ruff + mypy; coverage flat-or-up. Small, single-file, high-
clarity dedup.",
          "recommend_now": true
        },
        {
          "rank": 7,
          "type": "refactor",
          "title": "tracknet_runner: extract the velocity-active-frame loop into
_active_frames",
          "files": "backend/pipeline/detectors/tracknet_runner.py (trajectory_to_action_windows,
the per-frame velocity loop ~lines 315-329; method starts line 256); tests:
tests/test_tracknet_runner.py, tests/test_windowing.py",
          "summary": "trajectory_to_action_windows (~231L) opens with a self-contained loop
building the `active=[(frame_idx,velocity)]` list (visibility break + normalised inter-frame
velocity vs velocity_thresh), conceptually separate from the grouping + W1/R4/R5 post-processing
that follows (already factored into tested static methods). Verified the method body and that
the grouping/split steps are downstream of `active`.",
          "plan": "Extract a private static `_active_frames(points, fps, frame_width,
velocity_thresh) -> List[Tuple[int,float]]` containing the visibility/velocity loop, returning
`active`. Keep the fps<=0 / frame_width<=0 defaulting in the public method (applied before the
call, so identical floats are fed in). trajectory_to_action_windows then keeps `if not active:
return []`, grouping, and split/trim/blob steps unchanged. Output list is element-for-element
identical.",
          "risk": "low",
          "test_plan": "Run tests/test_tracknet_runner.py (moving/stationary/gap-split
trajectories) and tests/test_windowing.py (preset windows) ? windows must be byte-identical.
Full suite + ruff + mypy; coverage flat-or-up. Single file. Independent of the rank-6 dedup
(separate PR).",
          "recommend_now": true
        },
        {
          "rank": 8,
          "type": "refactor",
          "title": "eval/ablation: move the reportlab PDF builder into
backend/eval/ablation_pdf.py with a re-export delegate",
          "files": "backend/eval/ablation.py (export_pdf, ~lines 457-572, 116L) -> new
backend/eval/ablation_pdf.py; keep `from backend.eval.ablation_pdf import export_pdf` so
```

```
ablation.export_pdf stays valid; tests: existing test_export_pdf_is_valid_nontrivial /
test_cli_main_writes_pdf_and_json",
          "summary": "ablation.py (620L) mixes the pure ablation driver, ASCII table rendering,
and a 116L reportlab PDF builder (6 reportlab imports + ~40L styling constants) ? the single
biggest reason the file is large and the only part with a heavy optional 3rd-party dep.
export_pdf already does its reportlab imports lazily inside the function body, so no import-time
cost moves with it; it has zero overlap with the stats logic.",
          "plan": "Move export_pdf byte-for-byte into backend/eval/ablation_pdf.py (which
imports AblationReport/AblationRow/VERDICT_LABEL/EARNS_KEEP from ablation). In ablation.py
replace the body with a re-export `from backend.eval.ablation_pdf import export_pdf` so
ablation.export_pdf stays a valid public name. format_table (ASCII, no heavy dep) and the
dataclasses stay in ablation.py. No control-flow/formatting/data-shape change.",
          "risk": "low",
          "test_plan": "Run the ablation PDF tests (test_export_pdf_is_valid_nontrivial,
test_cli_main_writes_pdf_and_json) ? the emitted PDF must stay valid/non-trivial and the CLI
must still write PDF+JSON via the preserved name. Confirm reportlab is installed locally so the
test actually runs (don't let it skip). Full suite + ruff + mypy; coverage flat-or-up. Watch for
an import cycle: ablation_pdf imports FROM ablation, ablation re-exports FROM ablation_pdf ?
place the re-export AFTER the dataclass/constant definitions in ablation.py.",
          "recommend_now": true
        },
        {
          "rank": 9,
          "type": "refactor",
          "title": "main.py: extract the chunked-upload subsystem into backend/api/uploads.py as
an APIRouter",
          "files": "backend/main.py (~lines 409-532: _uploads, _uploads_lock, UploadInitRequest,
UploadCompleteRequest, _upload_cfg, _abort_upload, upload_init, upload_part, upload_complete,
upload_abort) -> new backend/api/uploads.py; tests: tests/test_upload_download.py (monkeypatch
targets shift module)",
          "summary": "main.py (871L) bundles app wiring, the pipeline, the job worker, the CLI,
and a self-contained chunked-upload island (~130L: in-process registry + 4 endpoints + 2
helpers). Verified the island (_uploads at line 409, UploadInitRequest 413, upload_init 442) is
the lowest-coupling block ? it only touches global_config, paths utils, and compute_video_id;
and confirmed main.py has NO APIRouter/include_router today (greenfield wiring). Biggest LOC
reduction with least entanglement.",
          "plan": "Create backend/api/uploads.py exposing `router = APIRouter()` carrying the 4
routes + _uploads/_uploads_lock/_upload_cfg/_abort_upload + the two pydantic models. In main.py
replace the inline defs with `from backend.api import uploads as uploads_api` +
`app.include_router(uploads_api.router)`. Identical route paths, status codes, body limits,
sequential-part logic, abort cleanup. THE ONE THING TO GET EXACTLY RIGHT: uploads.py must read
the SAME module-level global_config object main.py sets at startup (import it the same way / via
the existing accessor), or it is a real regression. test_upload_download.py monkeypatches
`m._uploads` and reads global_config off backend.main; retarget those monkeypatches to the new
module (test-only edit, behavior-preserving for production).",
          "risk": "medium",
          "test_plan": "Run tests/test_upload_download.py with retargeted monkeypatch (_uploads
now in uploads.py); assert init/part/complete/abort flow, body-limit, sequential-part rejection,
abort cleanup all unchanged. Add an explicit assertion that the router endpoints see the startup
global_config (e.g. drive an upload after config init). Full suite + ruff + mypy; coverage flat-
or-up. Medium risk due to the config-global seam + test retarget ? gate on owner review.",
          "recommend_now": true
        }
      ],
      "deferred": [
        "motion.process_video refactor: DEFER ? needs characterization tests first. motion.py
has ZERO direct test of process_video, is explicitly omitted in .coveragerc (verified) as a
'legacy demo baseline with fabricated metadata', returns Success/Failure (verified line 31/224,
distinct from the other three segmenters), and its DB-populate half is saturated with
random.sample/randint/uniform/random. No oracle exists for a behavior-preserving refactor of the
random half and no mocked cv2.VideoCapture harness exists for the detection half. Prerequisite
(out-of-sweep) PR: tests/test_motion_segmenter.py mocking cv2.VideoCapture (per
test_yolo_hybrid's _make_cap_mock) and seeding `random` to pin segment count + metadata; only
then extract _detect_motion_segments + _populate_db_with_events.",
        "trajectory_hybrid.process_video decomposition: DO NOT refactor ? already the refactored
```

hook-based base class shared by wasb_hybrid/tracknet_hybrid (thin subclasses) with a dedicated equivalence suite (tests/test_trajectory_hybrid_equivalence.py). No further decomposition warranted; further churn risks the fusion subclasses for no value.",

"Shared base.py _resolve_provider preamble dedup (gemini+yolo_hybrid+base): DEFER until AFTER the gemini (rank 4) and yolo_hybrid (rank 5) intra-file extractions land. It is cross-file, touches the shared base class loaded by ALL segmenters incl. fusion subclasses, trajectory_hybrid's variant differs (skip_ai branch, key only required when not skipping), and existing tests do NOT cover the sport-not-found or provider-KeyError early returns (those branches unverified). Lower-risk to first land each segmenter's own _resolve_provider via the intra-file PRs, then lift the now-identical gemini+yolo_hybrid bodies to base in a follow-up (scope (a): exclude trajectory's skip_ai variant). Medium risk + partial coverage ? keep out of this safe sweep.",

"main.py job-worker drain/wake loop extraction (job_worker.py): DEFER (medium risk). Verified the functions exist (JobWaiting line 238, _drain_due_jobs 294, _arm_wake_timer 332). The hazard is the global-state/monkeypatch seam: test_job_worker_loop.py and test_async_jobs.py monkeypatch m._arm_wake_timer/_get_executor and reference m.JobWaiting/_run_claimed_job/_drain_due_jobs/_MAX_DRAIN_WAKE_SEC directly. If _drain_due_jobs in a new module calls a LOCAL _arm_wake_timer, monkeypatch.setattr(m,'_arm_wake_timer',...) no longer intercepts -> a real daemon timer leaks in tests and scheduling assertions go stale. Doable with a re-export shim where inter-function calls resolve through the patched module object, but delicate. Ship only with careful verification that the monkeypatch indirection still intercepts ? not in this no-regression sweep alongside the upload split (don't churn two main.py islands at once).",

"database 14 write-methods -> _execute_write dedup: DEFER (medium risk, partial coverage). The boilerplate is real (~80L of connect/commit/except-logger.error/return-False) but per-method log message text differs today; collapsing changes those strings. Coverage on the failure (return-False) branches is partial, and the helper must be bounded to bool-returning single-statement writers only (leave lastrowid-returning add_play_segment/add_candidate_segment/upsert_user_model/claim_due_job and multi-statement prune_segments alone). A reviewer must confirm no test asserts a specific 'Failed to ...' string. Defer behind the rank-2 _init_db extraction so database.py changes land one reviewable PR at a time.",

"golden_fixtures.py P1 split into golden_real_fixtures.py: DEFER (medium risk). The split mirrors the existing test-file split (test_golden_real_fixtures.py) and is high-value (1113L -> ~820L), but the back-compat re-export creates a potential CIRCULAR import (golden_real_fixtures imports shared primitives FROM golden_fixtures, golden_fixtures re-exports freeze_real_fixture/RefusalError/_scan_forbidden FROM it). Must place the re-export at the BOTTOM of golden_fixtures after all shared primitives are defined, or import inside the re-export site. Mechanical but must be done carefully ? not a 'pure move' free win; gate on owner review rather than batching into the safe sweep.",

"eval load_videos dedup (ablation + fusion_compare + fusion_audio -> one parameterized loader): DEFER (medium risk). Bodies are structurally identical but audio policy differs (ablation tolerates-missing via .get(name,0.0); fusion_audio REQUIRES audio and raises SystemExit; fusion_compare omits audio) and the SUBTLE trap: ablation.SHUTTLE_COLUMNS is a hardcoded tuple while fusion_*'s SHUTTLE_FEATURE_NAMES derives from distill_local.FEATURE_NAMES ? they match today but are different objects, so the helper MUST take column lists as args, never hardcode. Net -25L with strong caller tests, but the audio-policy + column-source parameterization is exactly where a silent drift could slip; defer to a focused PR with the owner.",

"rally_seg_eval.py reporting/CLI refactor: DEFER ? needs tests first. format_report/_format_guard/_ci and the ~50L main() preset-override block (dataclasses.replace wiring for max-window-duration/hard-cap/blob-fallback/inactivity-end plus the parallel --vs-* variants) have ZERO coverage. The base/cand --vs duplication and _apply_inactivity closure ARE genuine extract-method candidates but unsafe now. Prerequisite (out-of-sweep, tests-only, trivially behavior-preserving): tests/test_rally_seg_eval_report.py driving format_report/_format_guard on known dicts + main() via monkeypatched sys.argv on a tiny synthetic fixture asserting --preset/--vs/--max-window-duration reach the preset (via --json-out). Then a follow-up extracts _build_preset(args, side).",

"wasb_infer.run() extraction: DEFER ? needs tests first (high risk). run() (~185L) imports torch/WASB DataLoader/detector/tracker and only runs in the WSL `wasb` conda env; test_wasb_cache.py covers only the pure helpers around it, never run(). Any extraction touching flush_every batching / base_manifest['windows_done'] / resume-index bookkeeping risks a silent regression in the reboot-resume guarantee that nothing local catches. Prerequisite: a torch-mocked DataLoader-level test (fake DataLoader/detector.run_tensor/tracker) asserting cache jsonl contents + windows_done progression across a simulated resume, OR an owner-side GPU run.",

```
        "person_detector vs tracknet_runner grouping dedup: DO NOT do. The surface similarity is
misleading: active-frame tuple shapes differ (person uses (frame_idx, peak_velocity,
set_of_track_ids); tracknet uses (frame_idx, velocity)) and the _finalize payloads differ
(person computes track_id_count via set-union across the group; tracknet hardcodes
track_id_count=1). Unifying requires a generic abstraction over the tuple/stat shape ? a
redesign with real regression surface, not a pure move. Leave as-is; if ever wanted, a
separately-designed shared grouper parameterized by a finalize callback, gated by its own
measurement.",
        "main.py FastAPI-routes-vs-CLI split + main()/run_cli_diagnose_clip dedup: DO NOT do now
? needs tests first (high risk). main() (168L) and run_cli_diagnose_clip (~70L, cv2-driven) have
ZERO tests and need cv2/ffmpeg this container lacks (owner must add machine-side). A pure
routes-module split would also churn the large test surface that does `import backend.main as m`
and patches module globals ? high blast radius for low isolated value."
      ],
      "notes": "Verified against the live branch (HEAD a010667 / #217, ahead of the prompt's
snapshot at #203 ? so I re-checked line ranges; file sizes grew modestly, e.g. gemini 472L vs
claimed 434L, but every cited seam is present). CONFIRMED-BUG #1 reproduced locally:
config.json='[]' -> TypeError 'list' object is not a mapping (the crash actually lands on the
{**defaults, **file_data} unpack at line 86 for the falsy-empty case; a non-empty non-object
like [1] hits the .items() AttributeError in _unknown_key_paths ? the single isinstance guard
fixes both). Cross-file return-type contract VERIFIED EXACTLY: main.py lines 201/213 and 838/843
do `isinstance(result, Failure)` then `result.value if hasattr(result,'value') else result`;
gemini/yolo_hybrid return bare lists (return segments_data + early return []), motion returns
Success/Failure (lines 31/224) ? every segmenter refactor MUST keep its own return type
identical. .coveragerc DOES omit motion.py (verified) ? its 'no oracle' deferral stands. CI
floor is --cov-fail-under=70 (.github/workflows/ci.yml line 97); every PR must keep full suite +
ruff + mypy green and coverage >=70 with no regression vs base. parse_time is a self-contained
nested closure (gemini line 309), safely liftable. database _init_db ladder verified clean
(v1->v5, each `if version < N` guarded, each ends PRAGMA user_version=N) ? the extraction is a
true pure move. compiler loop (234-334), upload island (_uploads 409, upload_init 442), job-
worker fns (JobWaiting 238, _drain_due_jobs 294, _arm_wake_timer 332), and the duplicated
tracknet midpoint-split (448-449 vs _blob_fallback_chop) all present. main.py has NO
APIRouter/include_router yet, so the uploads-router PR is greenfield wiring.\\n\\nSEQUENCING /
NO-STACKING GUIDANCE for the owner: ranks 1-9 are all standalone, one-work-item PRs branched
independently off origin/master (no stacking). Order within a file to avoid collisions: database
rank-2 (_init_db) ships before the deferred _execute_write dedup; gemini rank-4 and yolo_hybrid
rank-5 ship before the deferred shared-base _resolve_provider dedup (which depends on both
landing first); only ONE main.py island (rank-9 uploads) is in the safe sweep ? keep the
deferred job-worker extraction out to avoid churning two main.py islands at once. Ranks 1-8 are
low risk; rank-9 is medium (config-global seam + test-monkeypatch retarget) and the single most
valuable LOC reduction, so it is recommend_now=true but flagged for careful review of the
global_config wiring. I could not run the full suite/ruff/mypy/coverage here (container lacks
GPU/torch/cv2/numpy per CLAUDE.md) ? each PR's green-gate (suite + ruff + mypy + coverage>=70,
plus the cv2/ffmpeg/GPU paths for compiler & uploads) must be confirmed by the owner machine-
side before merge; that is the binding pre-LGTM gate, not these recommendations.\"}"
    },
    "confirmed_bug_count": 1,
    "debt_proposal_count": 18
  }
}
</output>
```


244. user (2026-06-18T08:05:18.641Z)

```
Workflow launched in background. Task ID: wyh5c4cgs
Summary: Implement vetted behavior-preserving refactors in isolated worktrees; open PRs for
owner review
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_87c49086-be4
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer
--claude-worktrees-refactor-base\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\safe-
refactor-fleet-wf_87c49086-be4.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:
"C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer--claude-
worktrees-refactor-base\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\safe-refactor-
```

fleet-wf_87c49086-be4.js"} to iterate without resending the script.)
Run ID: wf_87c49086-be4
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer--claude-worktrees-refactor-base\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\safe-refactor-fleet-wf_87c49086-be4.js", resumeFromRunId: "wf_87c49086-be4"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

245. assistant (2026-06-18T08:04:26.737Z)

Outstanding discovery output: **1 confirmed bug** (config loader crashes on a non-object `config.json` ? reproduced) and a **vetted, risk-rated refactor worklist** of 9 items (8 low-risk, 1 medium), with the genuinely-risky/under-tested ones correctly *deferred* (motion's random-heavy path, wasb GPU `run()`, the main.py job-worker seam, etc.).

Key correction I'll apply: the agents assumed "no GPU/torch/cv2 here," but this container **does** have them ? so every PR's full gate runs locally before it's opened.

Plan: I'll **implement the bug myself** (most important to verify personally), and **fan out the low-risk refactors to worktree-isolated agents** that each implement ? run the full gate ? open a PR for your review (never merge). Let me launch the refactor fleet, then do the bug in parallel.

246. assistant (2026-06-18T08:05:23.694Z)

The refactor fleet (7 isolated worktrees) is running. While it works, let me implement the config-loader bug myself. First, read the loader to confirm the exact crash.

247. user (2026-06-18T08:05:24.099Z)

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8
9       from __future__ import annotations
10
11      import json
12      import logging
13      from pathlib import Path
14      from typing import Any
15
16      from pydantic import BaseModel
17
18      from .models import AppConfig
19
20      logger = logging.getLogger(__name__)
21
22      DEFAULT_CONFIG_PATH = Path("config.json")
23
24
25      def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
26          """Dotted paths in `data` that the typed config will not honor.
27
28          Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
29          kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
30          typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
31          user's intent is lost ? collect every such key so load_config can warn.
```

```
        """
        unknown: list[str] = []
        fields = model_cls.model_fields
        for key, val in data.items():
            if key not in fields:
                unknown.append(prefix + key)
                continue
            ann = fields[key].annotation
            if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
                unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
        return unknown


    def _get_builtin_defaults() -> dict[str, Any]:
        """Return a plain dict of the current built-in defaults.

        The AppConfig model is the single source of truth for defaults.
        """
        return AppConfig().model_dump(mode="json")


    def load_config(path: str | Path | None = None) -> AppConfig:
        """Load configuration with the following precedence:

        1. Built-in defaults defined in the Pydantic models.
        2. Values from the JSON file (if present and readable).
        3. (Future) environment / CLI overrides.

        Missing file or unreadable file ? returns a fully defaulted AppConfig.
        Unknown keys in the file are tolerated (extra="allow" on the model).
        """
        cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH

        file_data: dict[str, Any] = {}
        if cfg_path.exists():
            try:
                with open(cfg_path, "r", encoding="utf-8") as f:
                    file_data = json.load(f)
            except Exception as exc:
                # Non-fatal: continue with defaults + whatever we could parse.
                # In production we might log here.
                logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")

        if file_data:
            unknown = _unknown_key_paths(file_data, AppConfig)
            if unknown:
                logger.warning(
                    "[CONFIG WARNING] %s contains keys the typed config does not "
                    "recognize (typo'd or removed knobs ? top-level extras are kept "
                    "but unread; unknown section keys are silently dropped): %s",
                    cfg_path, ", ".join(sorted(unknown)),
                )

        # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
        merged = {**_get_builtin_defaults(), **file_data}

        return AppConfig.model_validate(merged)


    def save_default_config(path: str | Path | None = None) -> Path:
        """Write a fresh default config.json.

        Used by scripts/doctor.py diagnostics and the --setup CLI path.
```

```
95          Overwrites if the file exists.
96          """
97          cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
98          cfg = AppConfig()
99          cfg_path.parent.mkdir(parents=True, exist_ok=True)
100         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
101         return cfg_path
102
103
104     # Backwards-compatible alias used by some older call sites during transition.
105     def get_default_config() -> dict[str, Any]:
106         """Return defaults as a plain dict (for transitional use only)."""
107         return _get_builtin_defaults()
108
```

248. assistant (2026-06-18T08:05:42.032Z)

Confirmed the bug: a valid-but-non-object `config.json` either hits `.items()` on a non-dict (truthy case) or `{**defaults, **file_data}` unpacking a non-mapping (falsy case) ? both crash startup, violating the "defaults + warn, never fatal" contract. Let me set up the bug worktree and read the existing config tests.

249. user (2026-06-18T08:05:48.045Z)

```
branch 'fix/config-loader-non-object-json' set up to track 'origin/master'.
HEAD is now at a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config
(#217)
=== reproduce the crash on origin/master ===
'[]'     -> TypeError: 'list' object is not a mapping
'[1,2]'  -> AttributeError: 'list' object has no attribute 'items'
'"x"'    -> AttributeError: 'str' object has no attribute 'items'
'42'     -> AttributeError: 'int' object has no attribute 'items'
'true'   -> AttributeError: 'bool' object has no attribute 'items'
'null'   -> TypeError: 'NoneType' object is not a mapping
=== existing test_config.py structure ===
17:def test_load_default_config():
18:    # Force load without the project's config.json so we test the model built-in defaults
19:    cfg = load_config("/nonexistent/path/that/does/not/exist/12345/config.json")
27:def test_logging_config_default_quiets_http():
29:    cfg = load_config("/nonexistent/path/12345/config.json")
33:def test_skip_ai_handoff_default_off():
35:    cfg = load_config("/nonexistent/path/12345/config.json")
39:def test_skip_ai_handoff_file_override():
48:def test_logging_config_file_override():
57:def test_fusion_config_defaults_reproduce_control():
65:    cfg = load_config("/nonexistent/path/12345/config.json")
75:def test_fusion_config_gate_a_opt_in_via_min_crossings():
86:def test_served_default_segmenter_is_gemini():
87:    """The shipped config.json served default is the prior detector (gemini) ? the #146 flip
90:    cfg = load_config(os.path.join(repo_root, "config.json"))
96:def test_fusion_config_file_override():
114:def test_file_override():
137:def test_extra_keys_tolerated():
138:    """Old or future keys in config.json must not break loading."""
155:def test_unknown_keys_warn_with_dotted_paths(tmp_path, caplog):
159:    p = tmp_path / "typo.json"
166:    with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
169:    joined = " ".join(r.message for r in caplog.records)
175:def test_recognized_config_does_not_warn(tmp_path, caplog):
176:    p = tmp_path / "clean.json"
```

250. assistant (2026-06-18T08:05:56.081Z)

All 6 non-object inputs crash on master. Let me read the worktree's loader + a test pattern to

match, then fix.

251. user (2026-06-18T08:05:57.532Z)

```
63          cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
64
65          file_data: dict[str, Any] = {}
66          if cfg_path.exists():
67              try:
68                  with open(cfg_path, "r", encoding="utf-8") as f:
69                      file_data = json.load(f)
70              except Exception as exc:
71                  # Non-fatal: continue with defaults + whatever we could parse.
72                  # In production we might log here.
73                  logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
74
75          if file_data:
76              unknown = _unknown_key_paths(file_data, AppConfig)
77              if unknown:
78                  logger.warning(
79                      "[CONFIG WARNING] %s contains keys the typed config does not "
80                      "recognize (typo'd or removed knobs ? top-level extras are kept "
81                      "but unread; unknown section keys are silently dropped): %s",
82                      cfg_path, ", ".join(sorted(unknown)),
83                  )
84
85          # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
86          merged = {**_get_builtin_defaults(), **file_data}
87
88          return AppConfig.model_validate(merged)
```

252. user (2026-06-18T08:05:58.439Z)

```
137     def test_extra_keys_tolerated():
138         """Old or future keys in config.json must not break loading."""
139         with tempfile.TemporaryDirectory() as td:
140             p = os.path.join(td, "with_extra.json")
141             data = {
142                 "sport": "badminton",
143                 "indexing": {"default_segmenter": "yolo_hybrid"},
144                 "some_future_key": "ignored_for_now",
145                 "validation": {"relevance": {"court_pixels_min_ratio": 0.1}},
146             }
147             with open(p, "w") as f:
148                 json.dump(data, f)
149
150             cfg = load_config(p)
151             assert cfg.indexing.default_segmenter == "yolo_hybrid"
152             assert cfg.validation.relevance.court_pixels_min_ratio == 0.1
153
154
155     def test_unknown_keys_warn_with_dotted_paths(tmp_path, caplog):
156         """Typo'd keys must be LOUD: top-level extras are kept-but-unread and unknown
157         section keys are silently dropped, so without this warning a typo just makes
158         the knob a no-op (the audit's 'silently dropped config' class)."""
159         p = tmp_path / "typo.json"
160         p.write_text(json.dumps({
161             "some_future_key": 1,                                  # top-level extra
162             "indexing": {"velocty_threshold": 0.5},                # typo'd section key
163             "validation": {"relevance": {"unknown_gate": True}},   # doubly nested
164         }))
165         import logging
166         with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
167             cfg = load_config(str(p))
```

```
168         assert isinstance(cfg, AppConfig)  # still non-fatal
169         joined = " ".join(r.message for r in caplog.records)
170         assert "some_future_key" in joined
171         assert "indexing.velocty_threshold" in joined
172         assert "validation.relevance.unknown_gate" in joined
173
174
175     def test_recognized_config_does_not_warn(tmp_path, caplog):
176         p = tmp_path / "clean.json"
177         p.write_text(json.dumps({
178             "sport": "badminton",
179             "indexing": {"default_segmenter": "gemini", "wasb": {"velocity_threshold":
0.02}},
180         }))
181         import logging
```

253. assistant (2026-06-18T08:06:03.358Z)

Now implement the fix ? coerce a non-object `config.json` to the no-op path:

254. user (2026-06-18T08:06:08.808Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\config-bug\backend\config\loader.py has been updated successfully. (file state is current in your context ? no need to Read it back)

255. assistant (2026-06-18T08:06:16.793Z)

Now the regression test. Let me check the imports and add it:

256. user (2026-06-18T08:06:17.462Z)

```
1       """Basic tests for the typed configuration package.
2
3       These ensure:
4       - Defaults are sensible and come from the models.
5       - File overrides work.
6       - Old config files with extra keys are tolerated.
7       - save_default_config produces loadable output.
8       """
9       import json
10      import tempfile
11      import os
12
13
14      from backend.config import load_config, save_default_config, AppConfig
15
16
```

257. user (2026-06-18T08:06:23.257Z)

```
175     def test_recognized_config_does_not_warn(tmp_path, caplog):
176         p = tmp_path / "clean.json"
177         p.write_text(json.dumps({
178             "sport": "badminton",
179             "indexing": {"default_segmenter": "gemini", "wasb": {"velocity_threshold":
0.02}},
180         }))
181         import logging
182         with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
183             load_config(str(p))
184         assert not [r for r in caplog.records if "CONFIG WARNING" in r.message]
185
186
```

```
187     def test_save_default_config_roundtrip():
188         with tempfile.TemporaryDirectory() as td:
189             p = os.path.join(td, "saved.json")
190             saved = save_default_config(p)
191             assert str(saved) == p
192             assert os.path.exists(p)
193 
194             cfg = load_config(p)
195             assert cfg.indexing.default_segmenter == "yolo_hybrid"
196             assert cfg.validation.min_blur_threshold == 20.0
197 
198 
199     def test_model_is_the_source_of_truth():
200         """Changing a default in the model should be reflected without touching json."""
201         # This is a documentation test more than anything.
202         cfg = AppConfig()
203         assert cfg.indexing.tracknet.velocity_threshold == 0.02
204 
205 
206     def test_output_dir_field_default_and_override():
207         """output_dir now lives on OutputConfig (was a write-only runtime hack), so
208         /api/compile and the CLI resolve the same value. Default + file override + the
209         runtime mutation that backend/main.py::main performs for --output-dir."""
210         cfg = AppConfig()
211         assert cfg.output.output_dir == "output"  # built-in default
212 
213         # The CLI override path: main() assigns args.output_dir onto the typed model.
214         cfg.output.output_dir = "/tmp/custom_out"
```

258. user (2026-06-18T08:06:36.265Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\config-
bug\tests\test_config.py has been updated successfully. (file state is current in your context ?
no need to Read it back)
```

259. assistant (2026-06-18T08:06:40.272Z)

Verify the test pins the bug (passes with fix, fails without), then the full gate:

260. user (2026-06-18T08:06:49.400Z)

```
=== new test WITH fix (expect pass) ===
.                                                                        [100%]
1 passed in 0.56s
=== revert loader, re-run (expect FAIL) ===
Saved working directory and index state WIP on fix/config-loader-non-object-json: a010667
feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)

backend\config\loader.py:86: TypeError
=========================== short test summary info ===========================
FAILED tests/test_config.py::test_non_object_config_degrades_to_defaults - Ty...
1 failed in 0.87s
Dropped refs/stash@{0} (48b6cd0640e8f3b9a151f9b40b1b7210a7507b9c)
```

261. assistant (2026-06-18T08:06:54.576Z)

Test pins the bug (passes with fix, fails with `TypeError` without). Now the full gate:

262. user (2026-06-18T08:11:20.612Z)

```
All checks passed!
Success: no issues found in 104 source files
857 passed, 7 skipped, 7 warnings in 140.25s (0:02:20)
=============================================================================
```

```
 RESULT: PASS - all tests green
========================================================================
```

263. assistant (2026-06-18T08:11:29.999Z)

```
Full gate green (857 passed, ruff + mypy clean, coverage ?70). Commit, push, open PR (for your
review ? not merging):
```

264. user (2026-06-18T08:11:40.839Z)

```
M  backend/config/loader.py
M  tests/test_config.py
[fix/config-loader-non-object-json d5e5ad7] fix(config): degrade to defaults + warn when
config.json is valid JSON but not an object
 2 files changed, 29 insertions(+)
d5e5ad7 fix(config): degrade to defaults + warn when config.json is valid JSON but not an object
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      fix/config-loader-non-object-json -> fix/config-loader-non-object-json
```

265. user (2026-06-18T08:11:54.594Z)

```
https://github.com/avidullu/badminton-highlight-indexer/pull/221
```

266. assistant (2026-06-18T08:12:00.043Z)

```
Config bug PR [#221](https://github.com/avidullu/badminton-highlight-indexer/pull/221) is open
for your review. Now let me collect the refactor fleet's results.
```

267. user (2026-06-18T08:16:19.832Z)

```
<retrieval_status>success</retrieval_status>

<task_id>wyh5c4cgs</task_id>

<task_type>local_workflow</task_type>

<status>completed</status>

<output>
{
  "summary": "Implement vetted behavior-preserving refactors in isolated worktrees; open PRs for
owner review",
  "agentCount": 7,
  "logs": [
    "PRs opened: 7/7 ? https://github.com/avidullu/badminton-highlight-indexer/pull/223
https://github.com/avidullu/badminton-highlight-indexer/pull/224
https://github.com/avidullu/badminton-highlight-indexer/pull/225
https://github.com/avidullu/badminton-highlight-indexer/pull/227
https://github.com/avidullu/badminton-highlight-indexer/pull/222
https://github.com/avidullu/badminton-highlight-indexer/pull/226
https://github.com/avidullu/badminton-highlight-indexer/pull/228"
  ],
  "result": {
    "results": [
      {
        "slug": "db-init-extract",
        "branch": "refactor/db-init-helpers",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/223",
        "diffstat": "backend/infrastructure/database.py | 349
++++++++++++++++++-------------- 1 file changed, 184 insertions(+), 165 deletions(-)",
        "suite": "856 passed, 7 skipped",
        "ruff": "All checks passed!",
        "mypy": "Success: no issues found in 104 source files",
```

```
        "coverage": "80.75% (floor 70; Required test coverage of 70% reached)",
        "behavior_note": "Pure intra-class cut-paste extraction. _init_db's single `with
self._connect()` block is unchanged: it now calls _create_base_tables(cursor) (the
videos/play_segments/events CREATEs), reads `version = cursor.execute(\"PRAGMA
user_version\").fetchone()[0]`, then calls _migrate_to_v1.._migrate_to_v5(cursor) each guarded
by the SAME `if version < N:` checks in the SAME order. Every block body ? SQL text, the `PRAGMA
user_version = N` bumps, the _add_column_idempotent ALTER calls, and the trailing conn.commit()
plus the `# future: if version < 6` comment ? was moved verbatim. No logic, branch conditions,
SQL, log strings, return types, ordering, thresholds, or constants changed; all DDL still runs
on the one cursor in the one connection/transaction block. The crash-safety/idempotency tests
(test_migration_idempotent_on_interrupted_upgrade,
test_migration_v3_db_upgrades_to_v5_preserving_rows, test_schema_version_is_current) and async-
job tests stay green.",
        "caveat": "Branch was cut from origin/master (a010667); only
backend/infrastructure/database.py was staged (explicit path) and only one commit (8c0fa48) sits
ahead of origin/master ? no sibling commits. The new helper methods are intentionally untyped
(cursor param has no annotation) to match the surrounding pre-existing _init_db style, which
mypy accepts without error. Full suite/coverage ran here as the env has
pytest/ruff/mypy/cv2/torch/numpy; no GPU/real-video path was exercised (none touched by this
refactor). PR left OPEN for owner review ? not merged."
      },
      {
        "slug": "compiler-trim-extract",
        "branch": "refactor/compiler-trim-one-segment",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/224",
        "diffstat": "backend/pipeline/stitcher/compiler.py | 220
++++++++++++++++++---------------- 1 file changed, 120 insertions(+), 100 deletions(-)",
        "suite": "856 passed, 7 skipped",
        "ruff": "All checks passed!",
        "mypy": "Success: no issues found in 104 source files",
        "coverage": "80.76% (floor 70; --cov-fail-under=70 reached)",
        "behavior_note": "Cut-paste extraction of the per-segment for-loop body in
VideoCompiler.compile_highlights into a new instance method _trim_one_segment(self, idx,
raw_start, raw_end, segments, video_duration, tmp_dir, raw_video_path, watermark_filter) ->
(clip_path_or_None, skip_record_or_None). In the original loop each segment resolved to exactly
one outcome ? it either appended one clip (no skip) or `continue`d after appending one skip
record (no clip). The helper returns a (clip_path, skip_record) pair with exactly one slot non-
None; the caller does two independent `if x is not None` appends to temp_clips/skipped_segments,
reproducing the original control flow byte-for-byte. The `continue` statements became `return
None, <skip_record>`; the success-append became `return clip_path, None`. All clamp arithmetic,
log strings/levels, every skip-reason string, and the watermark-then-plain ffmpeg retry order
(attempts = [vf_args, []] if vf_args else [[]]) are identical. No logic, branch conditions,
return types, ordering, thresholds, or constants changed.",
        "caveat": "ffmpeg/ffprobe are mocked in the suite (the offline test convention), so the
extraction is verified against tests/test_compiler.py (clamp/skip/ordering/concat) and
tests/test_watermark.py (watermark-failure fallback) rather than a real encode; an end-to-end
run on real video/GPU was not performed. PR opened against master for owner review only ? NOT
merged."
      },
      {
        "slug": "gemini-decompose",
        "branch": "refactor/gemini-process-video-helpers",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/225",
        "diffstat": "backend/pipeline/segmenters/gemini.py | 523 +++++++++---------- (298
insertions(+), 225 deletions(-)); 1 file changed",
        "suite": "856 passed, 7 skipped",
        "ruff": "All checks passed!",
        "mypy": "Success: no issues found in 104 source files (only informational annotation-
unchecked notes)",
        "coverage": "80.76% (floor 70, no regression) ? Required test coverage of 70% reached",
        "behavior_note": "Pure verbatim cut-paste extraction of
GeminiPlaySegmenter.process_video (~430 lines) into named private helpers: _resolve_provider
(preamble; the 3 early `return []` cases ? unknown sport / missing API key / unknown provider ?
```

```
are signalled via None and re-raised as [] by the orchestrator), _maybe_trim_video (returns
video_to_upload + cleanup_temp_files closure), _process_chunked / _process_single (the two
analyze branches incl. their summary-log builders; the inner process_single_chunk closure stays
nested since it captures run-local state), _validate_and_dedupe, _populate_db, and the formerly-
nested parse_time lifted to a module-level _parse_time (no free vars beyond logger). No logic,
branch conditions, log strings/levels, return types, statement ordering, thresholds, or
constants changed. process_video still returns a BARE LIST (not Result) and preserves every
early return [] ? main.py relies on the bare-list contract. model_name is computed identically.
tests/test_gemini.py (incl. the output-equivalence assertions) is unchanged and green.",
        "caveat": "CLAUDE.md notes the dev container lacks GPU/torch/cv2 ? but this worktree's
Python env had them, so the full offline suite (856 tests), ruff, mypy, and coverage all ran
locally and passed; no machine-side handoff needed. The task brief said \"8 mocked orchestration
tests\" but tests/test_gemini.py actually contains 7 test functions (plus a _run_chunked helper)
? all 7 pass. Note: process_video's declared base-class return type is Result[List[...]] but the
implementation has always returned a bare list; I preserved that existing (intentional, main.py-
relied-upon) discrepancy rather than \"fixing\" it, per the behavior-preserving mandate. Git
warns LF?CRLF on this Windows checkout (line-ending normalization only, no content change)."
      },
      {
        "slug": "yolo-decompose",
        "branch": "refactor/yolo-hybrid-process-video-helpers",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/227",
        "diffstat": "backend/pipeline/segmenters/yolo_hybrid.py | 155
++++++++++++++++++-------- (1 file changed, 111 insertions(+), 44 deletions(-))",
        "suite": "856 passed, 7 skipped",
        "ruff": "All checks passed!",
        "mypy": "Success: no issues found in 104 source files",
        "coverage": "80.77% (floor 70%, --cov=backend --cov=training)",
        "behavior_note": "Pure verbatim cut-paste: process_video (~309 lines) split into 5
private helpers (_resolve_provider, _collect_candidate_windows, _persist_candidates,
_run_ai_handoff, _populate_db) with process_video left as a phase orchestrator. Helper bodies
are byte-for-byte the original blocks; no log strings/levels, branch conditions, thresholds,
defaults, ordering, or return types changed. The two chunk-loop branches were moved as-is (NOT
merged); nested _merge_stats and process_candidate_window closures kept intact; max_workers +
os.remove/os.rmdir cleanup order preserved. All early `return []` preserved ? the three
provider/sport/API-key failures become _resolve_provider returning None which the orchestrator
maps to `return []`; invalid-duration and bare-list returns untouched. _resolve_provider returns
provider|None per spec; orchestrator re-reads the plain config values (idx_cfg, sport_profile,
provider_name, model_name) needed downstream ? deterministic side-effect-free self.config reads,
and BadmintonSportProfile is stateless (no __init__) so re-constructing it is an observable no-
op.",
        "caveat": "Two minor, behavior-neutral consequences of having _resolve_provider return
only the provider (per the spec signature): the orchestrator re-reads
idx_cfg/provider_name/model_name and re-calls sport_registry.get() once more (it constructs a
fresh stateless BadmintonSportProfile each call) ? pure idempotent reads, no observable
difference. Separately, the test file contains 6 test functions (not 7 as the task stated); all
6 pass, including the output-equivalence guard. Note: `python run_all_tests.py
--cov=backend.pipeline.segmenters.yolo_hybrid` (targeting the single module) raises a coverage-
plugin import-isolation error that also exists on master and is unrelated to this change ? the
gate's package-root coverage run (--cov=backend --cov=training) is clean at 80.77%.
GPU/torch/cv2 real-video paths are mocked in the offline suite (cannot be exercised here),
consistent with the repo's test design."
      },
      {
        "slug": "tracknet-extract",
        "branch": "refactor/tracknet-dedup-and-active-frames",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/222",
        "diffstat": "1 file changed, 36 insertions(+), 19 deletions(-) ?
backend/pipeline/detectors/tracknet_runner.py only",
        "suite": "856 passed, 7 skipped",
        "ruff": "All checks passed!",
        "mypy": "Success: no issues found in 104 source files",
        "coverage": "80.74% (>= 70 floor); same suite 856 passed, 7 skipped",
```

```
        "behavior_note": "Pure cut-paste into helpers, no logic change. (A)
_midpoint_split_index(group) holds the verbatim `mid=(g[0][0]+g[-1][0])/2.0;
split_i=min(range(1,len(g)), key=lambda i: abs(g[i][0]-mid))` two-liner and is now called from
both the W1 hard-cap fallback in _split_overlong_groups and from _blob_fallback_chop, which had
it copy-pasted identically. (B) _active_frames(points, fps, frame_width, velocity_thresh) is the
opening per-frame velocity loop lifted verbatim; the fps<=0/frame_width<=0 defaulting stays in
trajectory_to_action_windows BEFORE the call so identical floats feed the math, then `if not
active: return []` and everything downstream is untouched. No constants, thresholds, branch
conditions, log strings/levels, return types, or ordering changed; output is element-for-element
identical. Confirmed by the unchanged frame-set partition invariant + hard-cap/blob-fallback
split tests in tests/test_tracknet_runner.py and the preset-window tests in
tests/test_windowing.py.",
        "caveat": "The previous `# noqa: B023` on the _blob_fallback_chop midpoint lambda was
dropped because that lambda moved into the standalone _midpoint_split_index where `i` is the
comprehension-local of min(range(...)), not a closure over a loop variable ? ruff is clean,
confirming the suppression is no longer needed. Branch was cut from origin/master (a010667);
`git log origin/master..HEAD` lists only my one commit. Full suite/ruff/mypy/coverage all ran
and passed locally in this worktree (env has cv2/torch/numpy). PR is OPEN against master, not
merged, per instructions."
      },
      {
        "slug": "ablation-pdf-split",
        "branch": "refactor/ablation-pdf-module",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/226",
        "diffstat": "2 files changed, 145 insertions(+), 117 deletions(-) ?
backend/eval/ablation.py (+4/-117, body replaced by re-export), backend/eval/ablation_pdf.py
(new, +141)",
        "suite": "856 passed, 7 skipped",
        "ruff": "All checks passed!",
        "mypy": "Success: no issues found in 105 source files",
        "coverage": "80.74% (>= 70 floor); ablation_pdf.py 100%, ablation.py 99%",
        "behavior_note": "Pure cut-paste move: the export_pdf function body (lazy reportlab
imports, paragraph/table/style construction, log string 'wrote ablation PDF -> %s', return
value) was moved verbatim into the new backend/eval/ablation_pdf.py. ablation.py keeps the
public name via `from backend.eval.ablation_pdf import export_pdf as export_pdf`, placed AFTER
the dataclass/constant definitions ablation_pdf imports
(AblationReport/AblationRow/VERDICT_LABEL/EARNS_KEEP) so there is no import cycle ? verified at
runtime that ablation.export_pdf is ablation_pdf.export_pdf (same object). format_table,
dataclasses, driver and CLI stay in ablation.py. No logic/threshold/ordering/log change. The two
PDF tests run (reportlab+pypdf installed), not skipped, and pass.",
        "caveat": "Edited only the two files named in the task; staged explicit paths. The
redundant-alias re-export carries a `# noqa: E402` because it intentionally sits below module-
level code (the dataclasses it depends on) ? that is the cycle-avoiding placement the task
requested, not a style slip. AblationRow is imported per the task's import list and genuinely
consumed as the `rows: List[AblationRow]` annotation. Full pipeline (GPU/cv2/torch) not
exercised, but this module is pure-Python presentation and the offline suite + both PDF tests
ran green here."
      },
      {
        "slug": "main-uploads-router",
        "branch": "refactor/main-uploads-apirouter",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/228",
        "diffstat": "4 files changed, 165 insertions(+), 132 deletions(-):
backend/api/__init__.py (+1, new), backend/api/uploads.py (+155, new), backend/main.py (138
lines removed ? import + include_router; unused `Request` import dropped),
tests/test_upload_download.py (+1/-1, retarget `_uploads` monkeypatch).",
        "suite": "856 passed, 7 skipped",
        "ruff": "All checks passed!",
        "mypy": "Success: no issues found in 106 source files",
        "coverage": "80.75% (floor 70, --cov=backend --cov=training)",
        "behavior_note": "Pure cut-paste: the 4 endpoint bodies, route paths, status codes
(400/404/409/413/200), body-size limits, strictly-sequential part check, never-overwrite rename,
and abort cleanup are moved verbatim into backend/api/uploads.py ? only @app.* ? @router.*,
```

```
registered via app.include_router. The critical config seam is preserved: _upload_cfg resolves
backend.main.global_config (the same module-level global main() sets at startup), so it reads
the LIVE config, not a stale copy; verified the test's monkeypatch(m, 'global_config', cfg)
still flows through. The _uploads/_uploads_lock registry moved with the endpoints; the test's
monkeypatch(m, '_uploads', {}) was retargeted to uploads_api._uploads (test-only, production-
neutral). Unused `Request` import dropped from main.py (threading/uuid kept ? still used
elsewhere).",
        "caveat": "backend.main imports the router at module load, so backend.api.uploads must
NOT import backend.main at its own import time (caught a real circular-import failure when
uploads is imported first). Resolved by doing `import backend.main as _main` lazily inside
_upload_cfg; verified both import orders (main-first and uploads-first) resolve cleanly.
docs/CODE_MAP.md still describes these symbols as living in backend.main (`::upload_init` etc.)
? it was out of editing scope for this task, so a follow-up doc update would keep the cold-start
map accurate. mypy emits pre-existing annotation-unchecked notes (not from the changed files);
no errors. Full GPU/real-video pipeline not exercisable here, but these are pure HTTP/in-process
paths fully covered by tests/test_upload_download.py (15 tests, all green)."
      }
    ]
  }
}
</output>
```

268. assistant (2026-06-18T08:16:41.244Z)

All 7 refactor agents succeeded ? **8 PRs total now open** for your review (each self-reported a green local gate). But agent self-reports aren't enough for a "no regression" guarantee, so let me **independently verify** via GitHub CI and confirm each branch is exactly one clean commit off master.

269. user (2026-06-18T08:17:06.228Z)

```
=== all OPEN PRs ===
#228  refactor/main-uploads-apirouter  ?  refactor(api): extract chunked-upload island from
main.py into backend.api.uploads router
#227  refactor/yolo-hybrid-process-video-helpers  ?  refactor(segmenter): decompose yolo_hybrid
process_video into phase helpers
#226  refactor/ablation-pdf-module  ?  refactor(eval): split ablation export_pdf into
backend/eval/ablation_pdf
#225  refactor/gemini-process-video-helpers  ?  refactor(gemini): decompose process_video into
named private helpers
#224  refactor/compiler-trim-one-segment  ?  refactor(stitcher): extract per-segment trim into
_trim_one_segment
#223  refactor/db-init-helpers  ?  refactor(db): extract _init_db base-table + per-version
migration helpers
#222  refactor/tracknet-dedup-and-active-frames  ?  refactor(tracknet): dedup midpoint-split +
extract _active_frames helper
#221  fix/config-loader-non-object-json  ?  fix(config): non-object config.json must not crash
startup (degrade to defaults + warn)
#220  feat/long-rally-lens  ?  feat(eval): P7 long-rally (>5s) quality lens + reel knob (both
default-OFF)
#219  docs/desktop-app-plan  ?  docs(desktop): scope a cross-platform desktop app + the de-WSL
native-compute port

=== per-PR: commits-off-master (must be 1) + CI rollup ===
--- PR #221 (fix/config-loader-non-object-json): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
           1 Mypy       (lenient
           1 Ruff       lint
           1 Syntax     sweep
--- PR #222 (refactor/tracknet-dedup-and-active-frames): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
           1 Mypy       (lenient
           1 Ruff       lint
```

```
           1 Syntax     sweep
--- PR #223 (refactor/db-init-helpers): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
           1 Mypy       (lenient
           1 Ruff       lint
           1 Syntax     sweep
--- PR #224 (refactor/compiler-trim-one-segment): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
           1 Mypy       (lenient
           1 Ruff       lint
           1 Syntax     sweep
--- PR #225 (refactor/gemini-process-video-helpers): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
           1 Mypy       (lenient
           1 Ruff       lint
           1 Syntax     sweep
--- PR #226 (refactor/ablation-pdf-module): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
           1 Mypy       (lenient
           1 Ruff       lint
           1 Syntax     sweep
--- PR #227 (refactor/yolo-hybrid-process-video-helpers): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
           1 Mypy       (lenient
           1 Ruff       lint
           1 Syntax     sweep
--- PR #228 (refactor/main-uploads-apirouter): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
           1 Mypy       (lenient
           1 Ruff       lint
           1 Syntax     sweep
```

270. assistant (2026-06-18T08:17:13.907Z)

All 8 branches are exactly **1 clean commit off master**. My grouping hid the pass/fail column ? let me pull the actual CI conclusion per PR (and wait on any still running).

271. user (2026-06-18T08:17:24.359Z)

```
PR #221: PASS
PR #222: PASS
PR #223: PASS
PR #224: PASS
PR #225: PASS
PR #226: PASS
PR #227: PASS
PR #228: PENDING
```

272. assistant (2026-06-18T08:17:34.449Z)

7/8 CI **PASS** (independently confirming the agents' green gates); only the medium-risk **#228** (uploads router) is still running. That's exactly the one worth my own eyes ? it had to resolve a circular import and preserve the `global_config` seam. Let me spot-check its diff while CI finishes.

273. user (2026-06-18T08:17:38.102Z)

```
=== #228 diffstat ===
```

```
 backend/api/__init__.py       |   1 +
 backend/api/uploads.py        | 155 ++++++++++++++++++++++++++++++++++++++++++++
 backend/main.py               | 138 ++++-----------------------------------
 tests/test_upload_download.py |   3 +-
 4 files changed, 165 insertions(+), 132 deletions(-)

=== the config seam: how uploads.py resolves global_config ===
10:The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
13:``backend.main.global_config`` flow through unchanged).
20:from fastapi import APIRouter, HTTPException, Request
26:router = APIRouter()
41:def _upload_cfg():
45:    import backend.main as _main
46:    sec = getattr(_main.global_config, "security", None)
54:def _abort_upload(upload_id: str) -> None:
114:        _abort_upload(upload_id)
154:    _abort_upload(upload_id)

=== main.py: what replaced the island (import + include_router) ===
9:-from fastapi import FastAPI, HTTPException, Request
10:+from fastapi import FastAPI, HTTPException
18:-# Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
19:-# proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
22:-# aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
25:+# backend.api.uploads as a self-contained APIRouter. It resolves the live startup config via
26:+# backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
28:+from backend.api import uploads as uploads_api
30:-_uploads: Dict[str, Dict[str, Any]] = {}
31:-_uploads_lock = threading.Lock()
34:-class UploadInitRequest(BaseModel):
39:-class UploadCompleteRequest(BaseModel):
44:-    sec = getattr(global_config, "security", None)
45:-    upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
53:-    with _uploads_lock:
54:-        st = _uploads.pop(upload_id, None)
63:-def upload_init(req: UploadInitRequest):
81:-    with _uploads_lock:
82:-        _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0, "bytes":
0}
88:-async def upload_part(upload_id: str, index: int, request: Request):
```

274. assistant (2026-06-18T08:17:47.214Z)

The lazy `import backend.main as _main` correctly breaks the cycle while reading the live `global_config`. Let me confirm the `include_router` wiring and the `upload_dir` fallback are intact, then wait for #228's CI.

275. user (2026-06-18T08:17:51.397Z)

```
=== include_router wired in main.py? ===
406:from backend.api import uploads as uploads_api
408:app.include_router(uploads_api.router)
=== _upload_cfg full body (default fallback preserved?) ===
def _upload_cfg():
    # Resolved lazily (import inside the function) to read the LIVE startup config that
    # main() sets on the module global, and to avoid a circular import at module load
    # (main.py imports this router; this module must not import main at import time).
    import backend.main as _main
    sec = getattr(_main.global_config, "security", None)
    upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
    max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
    part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
    exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions", None) or
["mp4", "mov"])]
    return upload_dir, max_bytes, part_bytes, exts
```

```
=== route decorators (paths/status preserved?) ===
64:@router.post("/api/upload/init")
71:        raise HTTPException(status_code=400, detail=str(e)) from e
74:        raise HTTPException(status_code=400,
77:        raise HTTPException(status_code=413,
89:@router.put("/api/upload/{upload_id}/part/{index}")
96:        raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
98:        raise HTTPException(status_code=409,
115:         raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-init to
retry.")
123:@router.post("/api/upload/{upload_id}/complete")
130:         raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
```

276. assistant (2026-06-18T08:17:57.711Z)

#228 is a clean, behavior-preserving extraction: router wired, exact config defaults preserved, live `global_config` read via the lazy import, all route paths/status codes intact. Let me wait for its CI to confirm green.

277. user (2026-06-18T08:18:31.752Z)

```
Ruff lint       pass    9s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27746206451/job/82084999737
Syntax sweep + import smoke (no GPU stack)      pass    19s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27746206451/job/82084999723
Full test suite (offline, mocked)       pass    2m13s   https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27746206451/job/82084999706
Golden fixtures cross-OS (macos-latest) pass    24s     https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27746206451/job/82084999773
Golden fixtures cross-OS (ubuntu-latest)        pass    19s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27746206451/job/82084999731
Golden fixtures cross-OS (windows-latest)       pass    59s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27746206451/job/82084999755
Mypy (lenient baseline) pass    32s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27746206451/job/82084999859
Ruff lint       pass    9s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27746206451/job/82084999737
Syntax sweep + import smoke (no GPU stack)      pass    19s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27746206451/job/82084999723
=== exit 0 ===
```

278. assistant (2026-06-18T08:18:47.183Z)

All **8 PRs are green on CI** ? independently confirmed, not just agent self-reports. Let me clean up my worktrees (keeping the PR branches intact) and record the checkpoint.

279. user (2026-06-18T08:18:50.909Z)

```
=== worktrees before cleanup ===
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/config-bug
d5e5ad7 [fix/config-loader-non-object-json]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base
a010667 (detached HEAD)
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-1
8c0fa48 [refactor/db-init-helpers]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-2
2f30631 [refactor/compiler-trim-one-segment]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-3
39853b2 [refactor/gemini-process-video-helpers]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-4
c0ac2f9 [refactor/yolo-hybrid-process-video-helpers]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-5
79bd7f1 [refactor/tracknet-dedup-and-active-frames]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-6
```

```
d105026 [refactor/ablation-pdf-module]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-7
9c61554 [refactor/main-uploads-apirouter]
=== remove MY worktrees (branches/PRs stay intact) ===
removed config-bug
removed refactor-base
=== any leftover fleet worktrees? (auto-isolation should have cleaned) ===
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed
7e0881a [perf/wasb-fast-frames]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-1
8c0fa48 [refactor/db-init-helpers]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-2
2f30631 [refactor/compiler-trim-one-segment]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-3
39853b2 [refactor/gemini-process-video-helpers]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-4
c0ac2f9 [refactor/yolo-hybrid-process-video-helpers]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-5
79bd7f1 [refactor/tracknet-dedup-and-active-frames]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-6
d105026 [refactor/ablation-pdf-module]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-7
9c61554 [refactor/main-uploads-apirouter]
=== final: my 8 open PRs ===
#228  refactor(api): extract chunked-upload island from main.py into backend.api.uploads router
#227  refactor(segmenter): decompose yolo_hybrid process_video into phase helpers
#226  refactor(eval): split ablation export_pdf into backend/eval/ablation_pdf
#225  refactor(gemini): decompose process_video into named private helpers
#224  refactor(stitcher): extract per-segment trim into _trim_one_segment
#223  refactor(db): extract _init_db base-table + per-version migration helpers
#222  refactor(tracknet): dedup midpoint-split + extract _active_frames helper
#221  fix(config): non-object config.json must not crash startup (degrade to defaults + warn)
#220  feat(eval): P7 long-rally (>5s) quality lens + reel knob (both default-OFF)
#219  docs(desktop): scope a cross-platform desktop app + the de-WSL native-compute port
```

280. assistant (2026-06-18T08:18:57.525Z)

The fleet worktrees persisted (they hold committed branches). The branches are pushed and back the PRs, so removing the working dirs is safe. Let me prune them.

281. user (2026-06-18T08:19:05.642Z)

```
removed wf-1
removed wf-2
removed wf-3
removed wf-4
removed wf-5
removed wf-6
removed wf-7
=== remaining worktrees (mine should all be gone) ===
(all my sweep worktrees cleaned)
=== confirm the 8 PR branches still exist on origin (intact) ===
8
```

282. assistant (2026-06-18T08:19:11.912Z)

All cleaned up; all 8 PR branches intact. Let me record the checkpoint, then summarize.

283. user (2026-06-18T08:19:12.904Z)

```
9
10      # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12      **Checkpoint:** 2026-06-18 (? **P7 LONG-RALLY (>5s) QUALITY LENS + REEL KNOB SHIPPED ?
```

PR [#220](https://github.com/avidullu/badminton-highlight-indexer/pull/220), both default-OFF, awaiting owner review of the NUMBERS**). Owner: "bake in a long-rally lens (eval) + product knob today." Delivered + measured n=15:

284. user (2026-06-18T08:19:30.504Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

285. assistant (2026-06-18T08:19:50.717Z)

Done ? the full sweep landed as **8 review-ready PRs**, all green on CI, none merged (yours to review).

1 bug fix

PR	What
[#221](https://github.com/avidullu/badminton-highlight-indexer/pull/221)	**Startup crash:** a valid-but-non-object `config.json` (`[]`, `42`, `"x"`, `null`) crashed `load_config` (`.items()` / dict-unpack), violating its "non-fatal ? defaults" contract ? i.e. an operator typo took down the server/CLI. One `isinstance` guard ? defaults + warn. Regression test **proven to fail without the fix**. I implemented and verified this one personally.

7 behavior-preserving refactors of the grown files

PR	Target	Debt removed
[#225](https://github.com/avidullu/badminton-highlight-indexer/pull/225)	`gemini.process_video`	the **472-line / 71-branch monster** ? 6 named helpers + a ~40-line orchestrator
[#227](https://github.com/avidullu/badminton-highlight-indexer/pull/227)	`yolo_hybrid.process_video`	309 lines ? 5 phase helpers
[#223](https://github.com/avidullu/badminton-highlight-indexer/pull/223)	`database._init_db`	187-line migration ladder ? `_create_base_tables` + `_migrate_to_vN`
[#224](https://github.com/avidullu/badminton-highlight-indexer/pull/224)	`compiler.compile_highlights`	~100-line trim loop ? `_trim_one_segment`
[#228](https://github.com/avidullu/badminton-highlight-indexer/pull/228)	`main.py` (871 LOC)	chunked-upload island ? `backend/api/uploads.py` APIRouter (?138 lines from main)
[#226](https://github.com/avidullu/badminton-highlight-indexer/pull/226)	`ablation.py` (620 LOC)	reportlab PDF builder ? `ablation_pdf.py`
[#222](https://github.com/avidullu/badminton-highlight-indexer/pull/222)	`tracknet_runner.py`	dedup the copy-pasted midpoint-split + extract `_active_frames`

How I kept quality from regressing

- **Each refactor is a pure cut-paste** (no logic/branch/log/return/ordering/constant changes), backed by existing tests ? gemini & yolo even have **output-equivalence guards**.
- **Independently verified, not just agent self-reports:** every PR is **exactly one clean commit off `master`** and **GitHub CI is fully green** (full suite + ruff + mypy + golden-fixtures on ubuntu/macos/windows). Local gates ~857 passed, coverage ~80.7% (floor 70).
- I **spot-checked the one medium-risk PR (#228)** myself ? confirmed the router wiring, the exact config defaults, the live `global_config` read, and the lazy-import that breaks a real circular dependency.

What I deliberately did *not* do (flagged, not forced)

The discovery pass found ~11 more refactors but **deferred them** as unsafe-without-tests-first: `motion.process_video` (random-heavy, no oracle), `wasb_infer.run()` (GPU-only), the `main.py` job-worker seam, the `golden_fixtures.py` split, the `database` write-method dedup, `rally_seg_eval` reporting. Each needs characterization tests before a safe refactor ? a good follow-up sweep if you want it. Also: if you merge #228, `docs/CODE_MAP.md` needs a one-line update (upload symbols moved out of `backend.main`).

Take your time reviewing ? and if you want, I can merge the ones you approve and turn the deferred list into "add tests ? then refactor" PRs next.

286. user (2026-06-18T08:19:54.036Z)

please merge the PRs when all the test suites and regression tests are green

287. assistant (2026-06-18T08:20:37.454Z)

On it. Per your condition I'll re-verify green right before each merge ? and because these all landed independently, I'll also do a **combined-state check** (merge all 8 together and run the full suite) so the *aggregate* doesn't regress, not just each PR in isolation. First, sync and re-confirm CI + mergeability.

288. user (2026-06-18T08:20:51.973Z)

```
=== current origin/master ===
a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
=== re-verify each PR: CI rollup + mergeable ===
PR #221: CI=PASS  mergeable=MERGEABLE
PR #222: CI=PASS  mergeable=MERGEABLE
PR #223: CI=PASS  mergeable=MERGEABLE
PR #224: CI=PASS  mergeable=MERGEABLE
PR #225: CI=PASS  mergeable=MERGEABLE
PR #226: CI=PASS  mergeable=MERGEABLE
PR #227: CI=PASS  mergeable=MERGEABLE
PR #228: CI=PASS  mergeable=MERGEABLE
```

289. assistant (2026-06-18T08:20:59.007Z)

All 8: **CI PASS, MERGEABLE**, and master hasn't drifted (still a010667). Now the combined-state check ? merge all 8 together and run the full suite:

290. user (2026-06-18T08:21:05.937Z)

```
HEAD is now at a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config
(#217)
Switched to a new branch 'combined-verify'
merged OK: fix/config-loader-non-object-json
merged OK: refactor/tracknet-dedup-and-active-frames
merged OK: refactor/db-init-helpers
merged OK: refactor/compiler-trim-one-segment
merged OK: refactor/gemini-process-video-helpers
merged OK: refactor/ablation-pdf-module
merged OK: refactor/yolo-hybrid-process-video-helpers
merged OK: refactor/main-uploads-apirouter
=== combined diffstat vs master ===
 tests/test_config.py                          |  20 +
 tests/test_upload_download.py                 |   3 +-
 13 files changed, 1088 insertions(+), 802 deletions(-)
```

291. assistant (2026-06-18T08:21:11.897Z)

All 8 merged cleanly (no conflicts). Now the binding aggregate gate ? full suite + ruff + mypy + coverage on the combined state:

292. user (2026-06-18T08:23:19.594Z)

```
=== ruff ===
All checks passed!
=== mypy ===
Success: no issues found in 107 source files
=== full suite + coverage ===
Required test coverage of 70% reached. Total coverage: 80.94%
857 passed, 7 skipped, 7 warnings in 88.32s (0:01:28)
======================================================================
 RESULT: PASS - all tests green
```

```
================================================================================
```

293. assistant (2026-06-18T08:23:32.291Z)

```
**Combined state green** ? ruff clean, mypy clean (107 files), full suite **857 passed**,
coverage **80.94%**. The aggregate of all 8 does not regress. Safe to merge. Let me clean up the
check worktree and merge all 8 (squash).
```

294. user (2026-06-18T08:24:18.460Z)

```
error: failed to delete 'C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/combined-check': Permission denied
Deleted branch combined-verify (was 9aee56f).
=== merging all 8 (squash, delete branch) ===
PR #221 -> state=MERGED  ()
PR #222 -> state=MERGED  ()
PR #223 -> state=MERGED  ()
PR #224 -> state=MERGED  ()
PR #225 -> state=MERGED  ()
PR #226 -> state=MERGED  ()
PR #227 -> state=MERGED  ()
PR #228 -> state=MERGED  ()
```

295. assistant (2026-06-18T08:24:24.286Z)

```
All **8 PRs MERGED**. Let me confirm master advanced and watch the master-branch CI (the final
combined-state run on master itself).
```

296. user (2026-06-18T08:24:31.846Z)

```
   a010667..e67add1  master     -> origin/master
=== master now (should show the 8 squash merges on top) ===
e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router
(#228)
8092a39 refactor(segmenter): decompose yolo_hybrid process_video into phase helpers (#227)
f28ce05 refactor(eval): split ablation export_pdf into backend/eval/ablation_pdf (#226)
90dcc57 refactor(gemini): decompose process_video into named private helpers (#225)
18883b5 refactor(stitcher): extract per-segment trim into _trim_one_segment (#224)
af90e32 refactor(db): extract _init_db base-table + per-version migration helpers (#223)
596f7d3 refactor(tracknet): dedup midpoint-split + extract _active_frames helper (#222)
93d7116 fix(config): degrade to defaults + warn when config.json is valid JSON but not an object
(#221)
a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
b460526 docs(recording): assert single-match capture as the #1 rule (now MEASURED) (#218)
=== any of my branches left on origin? (should be 0) ===
0
=== latest master CI run ===
queued          refactor(api): extract chunked-upload island from main into backend.a?  CI
master  push    27746642325     12s     2026-06-18T08:24:19Z
```

297. assistant (2026-06-18T08:24:36.770Z)

```
Master advanced with all 8 squash-merges; no branches left behind. The master-branch CI is
queued ? let me watch it to green as the official confirmation (I already verified the combined
state locally, but this is the binding check).
```

298. user (2026-06-18T08:27:01.287Z)

```
Watching master CI run 27746642325 ...
  ? Complete job
? Full test suite (offline, mocked) in 2m13s (ID 82086482120)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
```

```
  ? System libs for opencv-python
  ? Install CPU-only torch (no CUDA wheels on a CPU runner)
  ? Install package (editable) + dev tooling
  ? Run full suite (with coverage floor)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
=== final conclusion ===
status=completed conclusion=success
```

299. assistant (2026-06-18T08:27:07.141Z)

Master CI: **success** ? the merged master is officially green. Let me prune the one leftover worktree dir and update the checkpoint.

300. user (2026-06-18T08:27:10.986Z)

```
all sweep worktrees cleaned
=== open PRs authored by me (should be 0) ===
2
```

301. assistant (2026-06-18T08:27:19.244Z)

Let me confirm those 2 remaining open PRs aren't from my sweep (all 8 of mine should be merged).

302. user (2026-06-18T08:27:28.054Z)

```
=== still-open PRs ===
#220  feat/long-rally-lens  ?  feat(eval): P7 long-rally (>5s) quality lens + reel knob (both
default-OFF)
#219  docs/desktop-app-plan  ?  docs(desktop): scope a cross-platform desktop app + the de-WSL
native-compute port
=== confirm my 8 are all MERGED ===
#221 MERGED
#222 MERGED
#223 MERGED
#224 MERGED
#225 MERGED
#226 MERGED
#227 MERGED
#228 MERGED
```

303. assistant (2026-06-18T08:27:35.682Z)

Confirmed: all 8 of my PRs are **MERGED**. The 2 still-open (#219 docs, #220 long-rally feature) are other sessions' work ? #220 is awaiting your review of its numbers ? so I correctly left them alone. Let me update the checkpoint.

304. user (2026-06-18T08:27:36.592Z)

```
10      # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12      **Checkpoint:** 2026-06-18 (? **REFACTORING SWEEP + BUG ? 8 PRs OPEN FOR OWNER REVIEW
(#221-#228), all CI-green, NOT merged**). Owner: "pause dev, refactor the grown files, find bugs
+ critical tech debt, make PRs (I'll review), ensure quality doesn't regress." Ran a 2-phase
discovery workflow (~18 agents): bug-hunt + tech-debt analysis over the grown files (mapped LOC
+ longest-functions: gemini.process_video=472L/71br worst, golden_fixtures=1113L, main.py=871L,
database.py=738L). Produced a risk-rated worklist; the genuinely-risky/under-tested items were
correctly DEFERRED (motion's random-heavy path, wasb GPU run(), main.py job-worker seam,
golden_fixtures split, _execute_write dedup, rally_seg_eval reporting ? all "needs
characterization tests first"). Shipped:
```

305. user (2026-06-18T08:27:46.713Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

306. assistant (2026-06-18T08:28:03.532Z)

All 8 merged and master is green. Here's the wrap-up.

Merged to `master` (now `e67add1`)

PR	Change
[#221](https://github.com/avidullu/badminton-highlight-indexer/pull/221)	**bug:** non-object `config.json` no longer crashes startup
[#222](https://github.com/avidullu/badminton-highlight-indexer/pull/222)	tracknet: dedup midpoint-split + extract `_active_frames`
[#223](https://github.com/avidullu/badminton-highlight-indexer/pull/223)	database: `_init_db` ? base-table + per-version migration helpers
[#224](https://github.com/avidullu/badminton-highlight-indexer/pull/224)	stitcher: extract `_trim_one_segment`
[#225](https://github.com/avidullu/badminton-highlight-indexer/pull/225)	gemini: the 472-line `process_video` ? 6 named helpers
[#226](https://github.com/avidullu/badminton-highlight-indexer/pull/226)	eval: `export_pdf` ? `ablation_pdf.py`
[#227](https://github.com/avidullu/badminton-highlight-indexer/pull/227)	yolo_hybrid: `process_video` ? phase helpers
[#228](https://github.com/avidullu/badminton-highlight-indexer/pull/228)	main.py: upload island ? `backend/api/uploads.py` router

How I honored "merge when all suites + regression tests are green"

- **Per-PR:** re-verified each was `CI=PASS` + `MERGEABLE` immediately before merging.
- **Aggregate (extra safety against combined regressions):** merged all 8 branches into one throwaway worktree ? **no conflicts** ? and ran the full gate: **857 passed / 7 skipped, ruff clean, mypy clean (107 files), coverage 80.94%**.
- **Post-merge:** the push-triggered **master CI run completed `success`** (full suite + ruff + mypy + golden-fixtures cross-OS). Branches deleted, worktrees pruned.

Left untouched (intentionally)

[#219](https://github.com/avidullu/badminton-highlight-indexer/pull/219) (desktop-app plan) and [#220](https://github.com/avidullu/badminton-highlight-indexer/pull/220) (long-rally lens) are from other sessions ? #220 is awaiting your review of its numbers ? so they're not mine to merge.

Two small follow-ups (your call)

1. **`docs/CODE_MAP.md`** still lists the upload endpoints under `backend.main`; #228 moved them to `backend/api/uploads.py`. A one-line doc update would keep the cold-start map accurate ? want me to push that?
2. The **11 deferred tech-debt items** (motion's random path, `wasb_infer.run()`, the main.py job-worker seam, `golden_fixtures` split, etc.) each need **characterization tests first**. I can do those as "add tests ? then refactor" PRs whenever you want the next sweep.

307. user (2026-06-18T10:17:31.052Z)

please go ahead with the code map and fixing the 11 tech debts with the same rigor and ensuring the quality doiesn;t regress

308. assistant (2026-06-18T10:19:57.006Z)

On it. These are the harder, deferred ones ? so the discipline is **characterization tests first** (pin current behavior, prove they pass on today's code, *then* refactor), the same combined-gate + CI rigor, and an honest "hold" for anything that turns out to be a risky redesign rather than a pure move.

Quick reality check on the 11 from the discovery analysis: a few were explicitly **"don't

refactor"** (trajectory_hybrid is already decomposed; person/tracknet grouping needs a redesign, not a pure move) and two need **owner-side GPU/cv2 validation** (`wasb_infer.run()`, the main.py routes/CLI split). I'll do the genuinely-safe ones with tests-first rigor, and report honestly on the rest rather than force a regression.

Let me launch the tech-debt fleet (tests-first ? refactor ? full gate ? open PR), and fix CODE_MAP myself while it runs.

309. user (2026-06-18T10:20:57.202Z)

Workflow launched in background. Task ID: wcldbim6x
Summary: Resolve deferred tech-debt items: characterization tests FIRST, then behavior-preserving refactor; open PRs
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_1d39e81c-851
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\techdebt-tests-first-fleet-wf_1d39e81c-851.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\techdebt-tests-first-fleet-wf_1d39e81c-851.js"} to iterate without resending the script.)
Run ID: wf_1d39e81c-851
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\techdebt-tests-first-fleet-wf_1d39e81c-851.js", resumeFromRunId: "wf_1d39e81c-851"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

310. assistant (2026-06-18T10:21:05.518Z)

The tech-debt fleet (7 agents) is running. Let me do the CODE_MAP fix myself in parallel. First, find what's now stale after the 8 merges.

311. user (2026-06-18T10:21:09.057Z)

HEAD is now at e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router (#228)
=== CODE_MAP references that the 8 merges made stale ===
115: - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** chunked upload ? `::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST .../complete` (`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ? client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ? upload state (`::_uploads` + `::_uploads_lock`) is IN-PROCESS ? restart aborts partials (client re-inits); per-user partitioning lands with PR-3. `::download_highlight` `@GET /api/highlights/{video_id}/{filename}` ? safe-name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads', max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge proxies cap bodies ~100MB).

312. assistant (2026-06-18T10:21:16.241Z)

Let me see the section structure around that line to update the module attribution correctly.

313. user (2026-06-18T10:21:17.290Z)

104 ```
105 Entrypoints: `@run_eval.py` (single video score), `@scripts/run_phase3_eval.py` (manifest batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression),

`@scripts/nightly_regression.py` (nightly golden-corpus rally-quality tripwire, exit 1=regression/4=stale), `@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning drivers).
106
107   ---
108
109   ## 2. SUBSYSTEMS
110
111   ### 2.0 Orchestration & config
112   - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries may still use `print` for direct output.
113   - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every endpoint NPEs.
114   - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx (path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_run_job,...)` -> **202 + job_id**. `::_run_job` (worker: mark running -> `_execute_processing` -> done/failed; catches + logs traceback). `::_execute_processing` = the former sync body (hash -> validate -> add_video -> segment -> `finally:` always `emit_indexer_report`, grafts `response["reports"]`; never raises from the report); a RAISING segmenter now prune_after-restores pre-run rows before re-raising. `::get_job` `@GET /api/jobs/{job_id}` -> `{status, progress, video_id, error, result, reports, ...}` ? job `status` = EXECUTION state (`queued|running|done|failed`); the pipeline verdict lives in `result["status"]`. `::_get_executor` lazy module-global `ThreadPoolExecutor(max_workers=1)` (single box + Gemini serial-upload learning; tests monkeypatch it inline). `main()` runs `db.fail_stale_jobs()` at startup (orphaned queued/running -> failed). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read). ? CLI `--video-path` mode still runs the pipeline synchronously inline (separate code path, unchanged by #16).
115   - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** chunked upload ? `::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST .../complete` (`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ? client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ? upload state (`::_uploads` + `::_uploads_lock`) is IN-PROCESS ? restart aborts partials (client re-inits); per-user partitioning lands with PR-3. `::download_highlight` `@GET /api/highlights/{video_id}/{filename}` ? safe-name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads', max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge proxies cap bodies ~100MB).
116   - `::load_config` ? ? legacy thin wrapper returning a **dict** (`load_app_config(path).model_dump(...)`); kept for transition ? new code imports `load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest` pydantic bodies.
117   - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on `OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`. `compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
118   - ? `--segmenter` argparse `choices` frozen at build time from `segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI default_segmenter fallback `'motion'`.
119   - `@scripts/run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()` (blocks automation), skips validation. Config via `backend.config.load_config` (typed); segmenter, padding, and db path all read from the model (no literal fallbacks).

314. assistant (2026-06-18T10:21:37.958Z)

The upload endpoints are documented under `@backend/main.py` (line 115); after #228 they live in `backend/api/uploads.py` (download_highlight stays in main). Let me check for any other staleness from the 8 merges, then make a focused, accurate edit.

315. user (2026-06-18T10:21:40.218Z)

```
=== other potentially-stale internal-symbol refs in CODE_MAP ===
66:  -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
112:- `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static
UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO,
timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray
prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries
may still use `print` for direct output.
114:  - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx
(path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_run_job,...)` -> **202 +
job_id**. `::_run_job` (worker: mark running -> `_execute_processing` -> done/failed; catches +
logs traceback). `::_execute_processing` = the former sync body (hash -> validate -> add_video
-> segment -> `finally:` always `emit_indexer_report`, grafts `response["reports"]`; never
raises from the report); a RAISING segmenter now prune_after-restores pre-run rows before re-
raising. `::get_job` `@GET /api/jobs/{job_id}` -> `{status, progress, video_id, error, result,
reports, ...}` ? job `status` = EXECUTION state (`queued|running|done|failed`); the pipeline
verdict lives in `result["status"]`. `::_get_executor` lazy module-global
`ThreadPoolExecutor(max_workers=1)` (single box + Gemini serial-upload learning; tests
monkeypatch it inline). `main()` runs `db.fail_stale_jobs()` at startup (orphaned queued/running
-> failed). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST
/api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy
`import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read). ? CLI `--video-
path` mode still runs the pipeline synchronously inline (separate code path, unchanged by #16).
117:  - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on
`OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`.
`compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is
honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
190:- `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. `::_parse_time`
(mirrors gemini.parse_time; ? unparseable -> 0.0), `::_get_video_duration`. ? if duration probe
fails (0.0) NO boundary validation; `begin_padding`/`end_padding` ctor defaults 0.5/1.0 but
every live call site (`/api/compile`, CLI trim, `scripts/run_process_and_stitch.py`) threads
`config.json stitching.{begin_padding,end_padding}` (typed default 0.5/0.5); temp clips in a
TemporaryDirectory under output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses
`print()`. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so
final concat can stream-copy ? change a preset and concat may break. ? optional **branding
watermark** (`stitching.watermark`, default-on): `::_watermark_filter` builds a `-vf` (drawtext
via `textfile=` + optional logo `movie/overlay`) applied to the per-clip re-encode only (concat
stays `-c copy`); a failed watermark encode retries the clip WITHOUT it (never nukes the reel,
and logs the ffmpeg stderr reason). Pass `watermark=` to `VideoCompiler(...)` (a
`WatermarkConfig` model or dict; `None` = off). Default font = bundled Apache-2.0
`::_BUNDLED_FONT` (`backend/assets/fonts/RobotoSlab-Regular.ttf`) so it renders without
fontconfig; `font_path` overrides.
216:- `@backend/infrastructure/database.py::Database` ? SQLite, 6 tables
(`videos/play_segments/events/candidate_segments/jobs/user_models`), `PRAGMA user_version`
migration ladder in `::_init_db` (currently =**5**: `version<1` creates `candidate_segments`;
`version<2` creates `jobs` + `idx_jobs_status` ? PR-1/#16; `version<3` ALTERs `jobs` to add
`policy`/`next_poll_at` + `idx_jobs_due` (self-scheduling EXECUTION_POLICY); `version<4` ALTERs
`videos`+`candidate_segments` to add a NULLABLE `user_id` (NULL=local install) + partial
`idx_videos_user`/`idx_candidates_user`; `version<5` creates `user_models` +
`idx_user_models_user_version`/`idx_user_models_one_active` (per-user model store,
PERSONALIZATION_PLAN ? persisted/loaded but NOT yet read on the served path)).
`::_get_connection` (? re-issues `PRAGMA foreign_keys=ON` per connection ? else CASCADE/SET NULL
silently skipped). Methods: `add_video`,`add_play_segment`,`add_event`,`add_candidate_segment`,`
link_candidate_to_segment`,`query_candidate_segments(detector=,only_unlabeled=)`,`query_play_seg
ments`,`get_video`,`clear_video_data`,`segment_watermarks`,`prune_segments`; job store: `create_
job`,`mark_job_running`,`set_job_progress`,`set_job_video_id`,`mark_job_done(result)`,`mark_job_
failed(error)`,`get_job` (deserializes `result`/`player_pool` JSON),`fail_stale_jobs` (startup
sweep -> count). ? candidate_segments is the detector-agnostic fine-tuning extension point
(common fields columns, detector-specifics in `stats`/`detection_params` JSON; index
`idx_candidates_video_detector`). ? `add_video` is an **UPSERT** (`INSERT ... ON CONFLICT(id) DO
UPDATE`) that mutates the row in place ? it does NOT cascade-delete dependent segments on re-
ingest (that was the old `INSERT OR REPLACE` footgun); clearing segments is now an explicit
```

```
destroy-AFTER-confirm via `segment_watermarks`/`prune_segments` (or `clear_video_data`). Events
column is `type` not `event_type`; `query_play_segments` filters use truthiness so
`duration_min=0`/empty-string = "no filter"; write helpers swallow exceptions.
286:- **`output_dir`:** a real field on `OutputConfig` (default `"output"`); the CLI `--output-
dir` overrides it on the typed model in `main()`, and `compile_highlights` reads the same
`global_config.output.output_dir` ? honored end-to-end.
314:| Change stitching padding | `config.json stitching.{begin_padding,end_padding}`; impl
`@backend/pipeline/stitcher/compiler.py::VideoCompiler.compile_highlights`; wiring
`@backend/main.py` (`/api/compile` + CLI trim); mirror
`@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
316:| Filter highlights by shot count | `config.json stitching.min_shot_count` ? enforced on
BOTH `@backend/main.py::compile_highlights` (`/api/compile`) and
`@scripts/run_process_and_stitch.py` (segments without `shot_count` metadata are exempt) |
335:| Add a DB column/table | `@backend/infrastructure/database.py::Database._init_db` migration
ladder + bump `PRAGMA user_version` (else version-assert tests fail) |
```

316. assistant (2026-06-18T10:22:01.768Z)

Only the upload endpoints (line 115) are stale ? `compile_highlights`/`_init_db` etc. are still valid (those refactors kept the public methods as orchestrators). Let me make three targeted, accurate edits.

317. user (2026-06-18T10:22:04.879Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\codemap-fix\docs\CODE_MAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

318. user (2026-06-18T10:22:10.666Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\codemap-fix\docs\CODE_MAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

319. assistant (2026-06-18T10:22:21.353Z)

Let me check whether a doc-consistency test parses CODE_MAP (so I run the right gate for a docs PR):

320. user (2026-06-18T10:22:23.913Z)

```
(no test references CODE_MAP)
=== verify the two edits read coherently ===
  - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** the chunked-upload endpoints now live in
`@backend/api/uploads.py` (#228 ? an `APIRouter` registered via `app.include_router`;
`uploads.py::_upload_cfg` resolves the live `backend.main.global_config` lazily so it reads the
startup config, and a lazy `import backend.main` avoids an import cycle) ?
`uploads.py::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext
allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-
bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-
part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST ...
```

321. assistant (2026-06-18T10:22:36.808Z)

No test reads CODE_MAP ? pure docs change, zero code impact. Let me commit, push, and open the PR (CI will still confirm green).

322. user (2026-06-18T10:22:49.154Z)

```
M  docs/CODE_MAP.md
[docs/code-map-uploads 9ed1f52] docs(code-map): point UPLOAD endpoints at backend/api/uploads.py
(post #228)
 1 file changed, 1 insertion(+), 1 deletion(-)
To https://github.com/avidullu/badminton-highlight-indexer.git
```

```
 * [new branch]      docs/code-map-uploads -> docs/code-map-uploads
https://github.com/avidullu/badminton-highlight-indexer/pull/229
```

323. assistant (2026-06-18T10:22:56.537Z)

```
CODE_MAP PR [#229](https://github.com/avidullu/badminton-highlight-indexer/pull/229) opened. Now
let me collect the tech-debt fleet's results.
```

324. user (2026-06-18T10:32:56.935Z)

```
<retrieval_status>timeout</retrieval_status>

<task_id>wcldbim6x</task_id>

<task_type>local_workflow</task_type>

<status>running</status>
```

325. assistant (2026-06-18T10:33:01.677Z)

```
Still running (tests-first refactors take longer). Let me keep waiting.
```

326. user (2026-06-18T10:42:30.092Z)

```
<retrieval_status>success</retrieval_status>

<task_id>wcldbim6x</task_id>

<task_type>local_workflow</task_type>

<status>completed</status>

<output>
{
  "summary": "Resolve deferred tech-debt items: characterization tests FIRST, then behavior-
preserving refactor; open PRs",
  "agentCount": 7,
  "logs": [
    "opened 6/7: https://github.com/avidullu/badminton-highlight-indexer/pull/230
https://github.com/avidullu/badminton-highlight-indexer/pull/232
https://github.com/avidullu/badminton-highlight-indexer/pull/235
https://github.com/avidullu/badminton-highlight-indexer/pull/233
https://github.com/avidullu/badminton-highlight-indexer/pull/231
https://github.com/avidullu/badminton-highlight-indexer/pull/234 | not-opened: base-resolve-
provider-dedup=held"
  ],
  "result": {
    "results": [
      {
        "slug": "motion-decompose",
        "branch": "refactor/motion-process-video-tests-then-helpers",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/230",
        "char_tests": "Added tests/test_motion_segmenter.py (3 tests). It mocks the module-level
cv2 (backend.pipeline.segmenters.motion.cv2) ? the WHOLE module is faked, not just VideoCapture,
because process_video calls cvtColor/GaussianBlur/absdiff/threshold/dilate/countNonZero
directly; a scripted countNonZero drives a deterministic per-frame motion_ratio (mirroring
test_yolo_hybrid's _make_cap_mock capture pattern). It SEEDS stdlib random via random.seed(0)
before the call so the fabricated server/receiver/metadata/events are reproducible. Pins: (1)
test_process_video_failure_on_unopenable_video -> Failure(message='Could not open video:
bad.mp4', is_recoverable=False) with no DB writes; (2)
test_process_video_no_motion_returns_empty_success -> Success(value=[]) when motion stays below
the 0.08 threshold; (3) test_process_video_fabricates_one_segment_deterministically -> Success
with exactly one segment dict {id:42, start_time?0.1667, end_time?8.1667, duration?8.0,
```

server:'Deepak', receiver:'Suresh',
metadata:{shot_count:5,intensity:'medium',avg_speed_kmh:51.7}}, the exact add_play_segment args,
and 4 add_event calls in order (serve Deepak valid/high_serve; smash Suresh 195.8; smash Deepak
156.6; foul Suresh net_touch). All 3 PASSED against the UNCHANGED motion.py before refactoring,
and PASSED again unchanged after.",
        "diffstat": "2 files changed, 251 insertions(+), 1 deletion(-).
backend/pipeline/segmenters/motion.py | 30 (29 ins, 1 del); tests/test_motion_segmenter.py | 222
(new file).",
        "suite": "python run_all_tests.py -> PASS: 860 passed, 7 skipped in ~65s (and 860 passed
/ 7 skipped under the coverage run).",
        "ruff": "python -m ruff check . -> All checks passed! (exit 0)",
        "mypy": "python -m mypy backend training -> Success: no issues found in 107 source files
(exit 0; only informational annotation-unchecked notes).",
        "coverage": "python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70 -> TOTAL 80.94% (Required 70% reached). Matches base ~80.9% ? no regression.
motion.py stays omitted in .coveragerc (omit unchanged), so coverage accounting is unaffected by
the new helpers.",
        "behavior_note": "Pure behaviour-preserving extraction. process_video became a thin
orchestrator: it still opens the capture and returns the same Failure on !isOpened, then calls
_detect_motion_segments(cap, max_frames) (the detection loop verbatim: sub-sample via grab(),
motion-signal build, moving-average smooth, threshold->raw segments, dead-time merge, min-
duration filter; releases cap; returns the (start,end) list) and
_populate_db_with_events(video_id, final_segments, player_pool) (the legacy fabrication half
verbatim: random.sample server/receiver, fabricated metadata, serve/0-3 smashes/termination
events, DB add_play_segment + add_event, builds per-segment dicts), then wraps the result in
Success(value=...). No change to logic, thresholds, constants, log strings/levels, ordering, or
return types. The stdlib random call order is byte-identical (the seeded characterization test
pins it). The only edit inside a moved body is `return Success(value=segments_data)` -> `return
segments_data`, with the Success wrap hoisted to the orchestrator. Tuple/Any were already
imported. HEAD branched cleanly from origin/master (git log origin/master..HEAD listed only my
one commit); staged explicit paths only.",
        "caveat": "Characterization coverage is intentionally scoped to the externally
observable contract (Success/Failure, the returned dicts, and db.add_play_segment/add_event
side-effects) ? exactly the surface the extraction had to preserve. It does not exhaustively
exercise every internal branch (e.g. max_frames early-break, the progress-logging cadence,
serve-foul / winner-vs-foul / variable smash counts at other seeds, or multi-segment merge);
motion.py remains omitted from coverage by design, so those paths are still uncounted. Run on
win32 / Python 3.13 with real cv2 4.13 + numpy present (cv2 calls are mocked in the new tests,
so they don't depend on it). No real-video / GPU path was exercised. The git autocrlf 'LF will
be replaced by CRLF' warning on the new test file is benign."
      },
      {
        "slug": "base-resolve-provider-dedup",
        "branch": "refactor/base-resolve-provider-dedup",
        "status": "held",
        "pr_url": "\"\" ? no PR opened; held because the two _resolve_provider bodies on
origin/master are NOT byte-identical, so a pure lift-to-base is impossible without changing
behavior (return shapes, model_name source, a log string).",
        "char_tests": "NONE written. Characterization tests were intentionally not added: the
task's binding precondition for the refactor ? \"Only lift if the two bodies are byte-identical;
if they diverge, report status='held'\" ? fails before any test work, so the discipline's STOP
rule applies (do not force a behavior-changing redesign). No production code was touched;
working tree is clean vs origin/master.",
        "diffstat": "No changes. `git diff --stat origin/master` is empty; `git status --short`
is empty. HEAD == origin/master (e67add172f5b620025f31489fde2fb9ff45465ac) on branch
refactor/base-resolve-provider-dedup.",
        "suite": "Not run ? held before any edit. Suite gate is moot because no refactor was
applied and no characterization tests were added (nothing to verify; production code
unchanged).",
        "ruff": "Not run ? no code changes to lint (held before editing).",
        "mypy": "Not run ? no code changes to type-check (held before editing).",
        "coverage": "Not run ? no code changes; base coverage (~80.9%) is unaffected since the
worktree is identical to origin/master.",
        "behavior_note": "NOT a pure move. The premise (an \"IDENTICAL private _resolve_provider
preamble\" in gemini.py and yolo_hybrid.py) does not hold on origin/master. `diff` of the two

method bodies (gemini.py lines 66-102 vs yolo_hybrid.py lines 57-95) returns exit=1 with several substantive hunks: (1) signature differs ? gemini takes `model_name: str`, yolo_hybrid takes `video_path: str`; (2) gemini CAPTURES `sport_profile = sport_registry.get(sport_name)` and returns it, yolo_hybrid CALLS `sport_registry.get(sport_name)` and DISCARDS the result; (3) yolo_hybrid derives `model_name = self.config.get(\"ai_model\", idx_cfg.get(\"ai_model\", \"gemini-2.5-flash\"))` internally, gemini receives model_name as an argument; (4) yolo_hybrid emits a trailing side-effect `logger.info(\"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video: ...\")` that gemini does not; (5) return shape differs ? gemini returns `(sport_profile, provider)` tuple, yolo_hybrid returns `provider` only. A single shared base method could not reproduce all five without changing logic, return types, and a log string at the call sites ? which the HARD RULES forbid. Hence held, not refactored.",
        "caveat": "If the owner still wants dedup, it is achievable only as a deliberate behavior-changing redesign (NOT a pure move): e.g. a base method that always reads `model_name` from config and always returns `(sport_profile, provider)`, with gemini dropping its `model_name` param and yolo_hybrid (a) accepting the now-returned sport_profile instead of re-fetching it at process_video line 374 and (b) relocating its `logger.info(\"Initializing Hybrid Segmenter ...\")` to its own process_video after the base call. That changes return types and moves a log emission, so it needs explicit owner approval and full characterization tests around BOTH segmenters' early-returns and the relocated log line before proceeding. Note the task description's stated premise was inaccurate for current origin/master (the bodies were independently refactored and diverged). I made zero edits; nothing to revert."
      },
      {
        "slug": "db-execute-write-dedup",
        "branch": "refactor/db-execute-write-helper",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/232",
        "char_tests": "Added 6 write-failure characterization tests to tests/test_database.py (test_add_video_write_failure_returns_false_and_logs, test_add_event_..., test_link_candidate_..., test_create_job_..., test_mark_job_running_..., test_set_job_waiting_...). Each monkeypatches db._get_connection to return a connection whose cursor.execute raises sqlite3.OperationalError(\"boom\"), then asserts the method returns False AND that caplog contains the method's EXACT fully-interpolated message (e.g. \"Failed to link candidate 7 to segment 42: boom\", \"Failed to create job job-99: boom\", \"Failed to set job job-3 waiting: boom\"). Covers 6 of the 11 routed methods (>= the required 2), specifically including all three message shapes: no-arg, single-id, and two-id interpolation. All 6 PASSED against the UNCHANGED production code pre-refactor (6 passed, 21 deselected), and the same tests passed unchanged post-refactor.",
        "diffstat": "backend/infrastructure/database.py | 200 ++++++++++++++++----------------- (79 insertions, 121 deletions); tests/test_database.py | 98 ++++++++++++++++++++ (98 insertions, 0 deletions); 2 files changed, 177 insertions(+), 121 deletions(-)",
        "suite": "python run_all_tests.py -> 863 passed, 7 skipped in 66.89s. RESULT: PASS - all tests green. (Targeted job/async/watermark/execution-policy/reingest subset also run: 74 passed.)",
        "ruff": "python -m ruff check . -> All checks passed!",
        "mypy": "python -m mypy backend training -> Success: no issues found in 107 source files (only pre-existing informational [annotation-unchecked] notes, no errors).",
        "coverage": "python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70 -> TOTAL 81.19%; \"Required test coverage of 70% reached.\" 863 passed, 7 skipped. Not a regression vs ~80.9% base (the 6 new failure-branch tests plus the helper consolidation improved database.py branch coverage).",
        "behavior_note": "Pure behavior-preserving move. Introduced private _execute_write(self, sql, params, error_msg) -> bool that captures the shared connect/cursor().execute/commit/except->logger.error/return-False boilerplate, then routed the 11 bool-returning single-statement writers (add_video, add_event, link_candidate_to_segment, create_job, mark_job_running, set_job_progress, set_job_video_id, mark_job_done, mark_job_failed, set_job_policy, set_job_waiting) through it. The helper logs f\"{error_msg}: {e}\"; each caller passes its original message stripped of the trailing \": {e}\" (e.g. \"Failed to add video\", f\"Failed to link candidate {candidate_id} to segment {segment_id}\"), so every emitted log line is byte-identical to before. No change to SQL text, params, return types, control flow, log levels, ordering, thresholds, or constants. Excluded per task: lastrowid-returning writers (add_play_segment, add_candidate_segment, upsert_user_model, claim_due_job), int-returning fail_stale_jobs, read paths, and multi-statement prune_segments ? all left untouched.",

```
        "caveat": "The shared _execute_write commits via the _connect() context manager (which
already wraps `with conn:` + conn.close()) AND calls conn.commit() inside, mirroring the
original methods exactly (they too had a redundant conn.commit() inside `with self._connect()`),
so the redundant-commit behavior is preserved rather than cleaned up ? intentionally, to keep
this a pure move. Full pipeline (GPU/torch/cv2/real-video) was NOT exercised; only the offline
mocked suite, which is the project's standard gate and ran green here. No merge performed (PR
left open for review per instructions)."
      },
      {
        "slug": "eval-load-videos-dedup",
        "branch": "refactor/eval-load-videos-shared",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/235",
        "char_tests": "Confirmed the existing caller tests pass on UNCHANGED code first:
tests/test_ablation.py (test_load_videos_tolerates_missing_audio,
test_load_videos_skips_windowing_only_schema), tests/test_fusion_compare.py
(test_load_videos_skips_windowing_only_in_mixed_corpus + the compare/CI tests),
tests/test_audio_features.py (test_load_videos_requires_audio,
test_load_videos_with_audio_skips_windowing_only_in_mixed_corpus, test_audio_increment_compare)
? 60 passed, 2 skipped (golden-litmus needs real output/*.json). ADDED to
tests/test_experiment.py: test_golden_loaders_audio_policy_differs ? pins the three-way audio-
policy DIFFERENCE on the SAME audio-less full-fusion file: ablation.load_videos defaults audio
cols to 0.0; fusion_compare.load_videos omits audio cols; fusion_audio.load_videos_with_audio
raises SystemExit. Ran it against UNCHANGED production code: PASS (after adding the missing
`import os` to the test file). Re-ran all four test files after the dedup: 61 passed, 2 skipped
? the same char test still green, unchanged. Also added
test_load_golden_videos_rejects_unknown_audio_policy to pin the new loader's policy-validation
guard.",
        "diffstat": "5 files changed, 150 insertions(+), 75 deletions(-).
backend/eval/ablation.py +12/-24 (load_videos now delegates; dropped unused glob/os imports +
_is_fusion_schema import); backend/eval/experiment.py +64 (new load_golden_videos + `import
glob`); backend/eval/fusion_audio.py +12/-25 (delegates; dropped glob + _is_fusion_schema
imports); backend/eval/fusion_compare.py +11/-22 (delegates; dropped glob/json/os +
_is_fusion_schema imports); tests/test_experiment.py +46 (import os, import load_golden_videos,
2 new tests).",
        "suite": "python run_all_tests.py ? 859 passed, 7 skipped (RESULT: PASS - all tests
green). Base was 858 passed; +1 is the new audio-policy char test (the policy-guard test runs in
test_experiment.py which run_all_tests already collects).",
        "ruff": "python -m ruff check . ? All checks passed!",
        "mypy": "python -m mypy backend training ? Success: no issues found in 107 source files
(only pre-existing informational annotation-unchecked notes).",
        "coverage": "python run_all_tests.py --cov=backend --cov=training --cov-fail-under=70 ?
Total coverage 80.87% (gate PASS, ?70). Base (origin/master, measured by stashing) = 80.94%. The
absolute MISSED-line count is IDENTICAL in both (1542 misses); base TOTAL = 8092 statements,
refactored TOTAL = 8059. The 0.07pp tick is a pure denominator effect: the dedup deleted 33
fully-covered duplicate lines, so misses stayed flat at 1542 while the percentage moved
1542/8092?1542/8059. No previously-tested line became untested; the new shared loader
(experiment.py lines 199-260) is fully covered.",
        "behavior_note": "Pure move/extraction/dedup. The three loaders' bodies were lifted
verbatim into one experiment.load_golden_videos that is parameterized on (1) caller-supplied
column lists ? never hardcoded, (2) an audio_policy literal reproducing each caller's branch
exactly (omit ? no audio loop; tolerate-missing ? float((c.get('audio') or {}).get(fn, 0.0));
require ? the `if 'audio' not in cands[0]: raise SystemExit(...)` guard BEFORE reading, then
float(c['audio'][fn])), (3) a per-caller skip_logger + skip_message so each module's info-level
skip line and the `... ? run `--augment` first` SystemExit message stay byte-identical (verified
by string equality). The _is_fusion_schema skip ordering, interval/feature/gts parsing,
sorted(glob) iteration, and VideoCandidates construction are unchanged. No logic, output, log
string/level, return type, ordering, threshold, or constant changed. Newly-unused imports
(glob/os/_is_fusion_schema, and json in fusion_compare) were removed from the callers to keep
ruff clean ? required by the move, not a behavior change.",
        "caveat": "Global coverage is 80.87% vs base 80.94% ? a 0.07pp dip whose root cause is
the denominator shrinking by 33 covered lines from a pure dedup (absolute misses unchanged at
1542; no line lost test coverage). It clears the repo's 70% CI floor comfortably. If a strict
no-decimal-regression policy is enforced, this is the expected and benign signature of removing
covered duplicate code. The 2 golden-litmus skips and the GPU/real-video paths were NOT run (no
```

real output/*_golden_features.json corpus / no GPU here) ? but those are pre-existing skips unrelated to this change; the loaders are exercised by synthetic-JSON tests. The new audio_policy='require' happy path (reading c['audio'][fn] when audio IS present) is covered via fusion_audio's existing test_audio_increment_compare."
      },
      {
        "slug": "golden-fixtures-split",
        "branch": "refactor/golden-fixtures-real-split",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/233",
        "char_tests": "Pre-refactor baseline: ran tests/test_golden_fixtures.py + tests/test_golden_real_fixtures.py against the UNCHANGED code ? 54 passed (these already exercise freeze_real_fixture / RefusalError / _scan_forbidden and the whole subsystem). Added new tests/test_golden_fixtures_split.py (7 tests) pinning the post-split re-export contract: gf.freeze_real_fixture is grf.freeze_real_fixture (and RefusalError, _scan_forbidden, and the moved helpers _new_surrogate_slug/_reset_trajectory_rows/_reset_labels/_SCHEMA_FILENAME_LITERALS ? all same object); the shared-primitive single-source-of-truth (constants/helpers golden_real_fixtures uses ARE the golden_fixtures objects, not copies); RefusalError still a ValueError subclass via both paths; identical _scan_forbidden gate behaviour via either module path. After the split, all 61 (54 existing unchanged + 7 new) pass. Both import orders (golden_fixtures-first AND golden_real_fixtures-first) verified to resolve the cycle and yield identical objects.",
        "diffstat": "3 files changed, 461 insertions(+), 296 deletions(-). backend/eval/golden_fixtures.py (-296/+31: removed the moved P1+M2 block + 4 now-unused imports os/shutil/tempfile/TrackNetRunner; added a lazy PEP-562 __getattr__ re-export + CLI now calls through golden_real_fixtures); backend/eval/golden_real_fixtures.py (NEW, 343L: the moved subsystem importing shared primitives from golden_fixtures); tests/test_golden_fixtures_split.py (NEW, 87L).",
        "suite": "python run_all_tests.py -> 864 passed, 7 skipped in 71.35s (RESULT: PASS). Coverage run: 864 passed, 7 skipped.",
        "ruff": "python -m ruff check . -> All checks passed!",
        "mypy": "python -m mypy backend training -> Success: no issues found in 108 source files (only informational annotation-unchecked notes, no errors).",
        "coverage": "python run_all_tests.py --cov=backend --cov=training --cov-fail-under=70 -> TOTAL 80.96% (Required 70% reached; no regression vs ~80.9% base). Split files: golden_real_fixtures.py 98% (92 stmts, 2 miss), golden_fixtures.py 87% (353 stmts, 47 miss).",
        "behavior_note": "Pure move: the real-fixture-only code (RefusalError, _new_surrogate_slug, _SCHEMA_FILENAME_LITERALS, _scan_forbidden, _reset_trajectory_rows, _reset_labels, freeze_real_fixture) was relocated verbatim from golden_fixtures.py into golden_real_fixtures.py with NO change to logic, output strings, return shapes, ordering, thresholds, or constants. golden_fixtures keeps the shared synthetic/replay/manifest primitives; the new module imports them FROM golden_fixtures (single source of truth). Historical golden_fixtures.<name> import paths are preserved via a lazy PEP-562 module __getattr__ that resolves to the SAME objects defined in golden_real_fixtures, so callers (incl. tests/CI/docs that reference backend.eval.golden_fixtures) are unaffected. The only non-relocation edits were mechanical: dropping the 4 imports that became unused in golden_fixtures (os/shutil/tempfile/TrackNetRunner ? they moved with the code) and pointing the --freeze-real CLI branch at golden_real_fixtures (same callable + same exception class).",
        "caveat": "Used a lazy PEP-562 __getattr__ for the re-export rather than the eager bottom-of-file `from golden_real_fixtures import ...` suggested in the task: an eager bottom import would fail when golden_real_fixtures is imported FIRST (its own names aren't defined yet at the point golden_fixtures runs its bottom import), so the lazy hook is required to make BOTH import orders resolve ? both verified. Functionally equivalent (same re-exported objects), just defers the import to first attribute access. This dev environment HAS pytest/ruff/mypy/cv2/torch/numpy and the full offline suite ran green here; no GPU/real-video paths were exercised (none are touched by this pure module move)."
      },
      {
        "slug": "rally-seg-eval-report-extract",
        "branch": "refactor/rally-seg-eval-build-preset",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/231",
        "char_tests": "Added tests/test_rally_seg_eval_report.py (11 tests) pinning the previously-zero-coverage reporting + main() preset-override paths. Confirmed ALL 11 PASS against the UNCHANGED code before refactoring, and the SAME 11 still pass unchanged after. Coverage: _ci

(None/{}?\"?\" branch + 3-dp \"mean [lo,hi]\" render); _format_guard (PASS/no-regress branch AND BLOCK + count-based + merge_rate-regress branch, rendered byte-for-byte); format_report (full render incl. all bounded knobs cap(hard)/inact_end/blob_fallback + the no-knobs header that omits extras); main() driven via monkeypatched sys.argv on a synthetic golden-features fixture (1 oscillating-trajectory video), asserting --preset/--vs/--max-window-duration/--hard-cap/--blob-fallback/--blob-factor/--inactivity-end/--inactivity-rally-thresh/--inactivity-min-density and the --vs-* variants reach the resolved preset (read back via --json-out: \"preset\" for non-vs, \"base\"/\"cand\" for vs), plus single-preset evaluate path and the unknown-preset SystemExit.",
        "diffstat": "2 files changed, 345 insertions(+), 33 deletions(-). backend/eval/rally_seg_eval.py: +39/-33 (net +6: new _build_preset helper, dedup in main()). tests/test_rally_seg_eval_report.py: new file (+312).",
        "suite": "python run_all_tests.py ? 868 passed, 7 skipped in ~66s. RESULT: PASS - all tests green.",
        "ruff": "python -m ruff check . ? All checks passed! (Initial run flagged one F401 unused-import in the new test file; removed the stray `from backend.eval import rally_seg_eval` line, re-ran clean.)",
        "mypy": "python -m mypy backend training ? Success: no issues found in 107 source files (only pre-existing annotation-unchecked notes, no errors).",
        "coverage": "python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70 ? TOTAL 82.41% (Required 70% reached; at/above the ~80.9% base, no regression). The touched module backend/eval/rally_seg_eval.py went from 0%?98% on the report/main() paths (4 missed lines are incidental: lazy imports 155/187/189 + the __main__ guard 386).",
        "behavior_note": "Pure extract-method move. The duplicated base/cand dataclasses.replace wiring in main() ? cap (max_window_duration), hard_cap, blob_fallback, blob_factor, and the inner _apply_inactivity closure (inactivity_end + rally_thresh + min_density), plus the parallel --vs-* fallbacks ? was consolidated into one _build_preset(args, side) helper (side in {\"base\",\"cand\"}). Each replace() touches the same field under the same condition in the same order; the cand-side --vs-* ? base fallback semantics (vs_max_window_duration|?max_window_duration, vs_hard_cap or hard_cap, vs_blob_fallback or blob_fallback, vs_inactivity_end|?inactivity_end; blob_factor/rally_thresh/min_density shared) are preserved exactly. _build_preset(args,\"cand\") is only called inside `if args.vs:`, matching the original which never built a cand preset without --vs. No change to logic, outputs, print/log strings, return types, ordering, thresholds, or constants ? verified byte-identical by the unchanged characterization tests (format_report/main --json-out outputs pinned).",
        "caveat": "Full offline suite + ruff + mypy + coverage all run and verified GREEN on THIS machine (env has pytest/ruff/mypy/cv2/torch/numpy). The GPU/WSL real-video path is out of scope for this pure-logic refactor and was not exercised. The characterization fixture is synthetic (one oscillating-trajectory CSV + a hand-written golden-features JSON), not the real golden corpus, but it drives the exact main()/report code paths the item targets. PR is opened, NOT merged (per instructions)."
      },
      {
        "slug": "job-worker-extract",
        "branch": "refactor/main-job-worker-module",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/234",
        "char_tests": "Added tests/test_job_worker_module_seam.py with 3 pins of the exact deferral hazard (the monkeypatch indirection): (1) the worker symbols stay addressable as attributes of backend.main; (2) a real _drain_due_jobs of a not-yet-due 'waiting' job routes the wake through the PATCHED m._arm_wake_timer (the leaked-timer regression pin ? asserts the patched recorder fired, not a bare local); (3) _arm_wake_timer submits the re-drain through the PATCHED m._get_executor / m._drain_due_jobs (via a non-leaking fake threading.Timer that fires immediately). Confirmed all 3 PASS against the UNCHANGED main.py before the move, and PASS unchanged after. The two named hazard files (tests/test_job_worker_loop.py 12 tests, tests/test_async_jobs.py 17 tests) plus tests/test_api_endpoints.py also passed both pre- and post-refactor (42 -> 45 with the new file).",
        "diffstat": "backend/api/job_worker.py | 152 +++ (new); backend/main.py | 140 +- (mostly deletions: 6 moved defs + 3 now-unused imports removed, replaced by an 8-name re-export shim); mypy.ini | 6 + (quarantine entry for the new module, mirroring backend.main); tests/test_job_worker_module_seam.py | 97 +++ (new). 4 files changed, 272 insertions(+), 123 deletions(-). main.py net -100 lines.",
        "suite": "python run_all_tests.py -> 860 passed, 7 skipped (RESULT: PASS - all tests green). Ran on the refactored code; the targeted subset (test_job_worker_loop + test_async_jobs

```
+ test_api_endpoints + new seam file) = 45 passed.",
        "ruff": "python -m ruff check . -> All checks passed! (re-export import carries a `#
noqa: F401 (re-exports)` matching the repo convention in backend/eval/rally_seq_proto.py)",
        "mypy": "python -m mypy backend training -> Success: no issues found in 108 source files
(was 107; the new module is now checked but quarantined). The 7 union-attr errors on db_instance
Optional-global accesses are excused by the new [mypy-backend.api.job_worker] entry that mirrors
the pre-existing [mypy-backend.main] quarantine ? same Optional module-global debt, resolved
through backend.main at runtime.",
        "coverage": "81.00% total (Required test coverage of 70% reached). At/above base ~80.9%
? no regression. backend/api/job_worker.py itself is 95% covered (only the lazy
ThreadPoolExecutor-construction lines 36-38 uncovered, which the existing tests deliberately
monkeypatch around, identical to the original main.py).",
        "behavior_note": "Pure move: the 6 worker functions + JobWaiting + _MAX_DRAIN_WAKE_SEC +
_job_executor were relocated verbatim (same logic, log strings/levels, return shapes, ordering,
the 60.0 constant). The only deltas are (a) cross-function seam calls now go through `import
backend.main as _main` / `_main.<name>` at call-time instead of bare module-locals ?
semantically identical because backend.main re-exports the same objects and the original locals
were also resolved at call-time, but it is what makes the existing monkeypatch.setattr(m, ...)
STILL intercept (the explicit deferral concern: a moved _drain_due_jobs calling a local
_arm_wake_timer would have leaked a real daemon timer in tests); and (b) _job_to_request's `->
ProcessRequest` return annotation was dropped to avoid a module-level ProcessRequest import
(annotation-only, no runtime change, module is mypy-quarantined). No endpoint, CLI, threshold,
or output changed.",
        "caveat": "mypy.ini was edited (a file not literally named in the task) ? strictly
necessary to keep the mypy gate green for a pure move of already-excused code: the moved
functions inherit backend.main's pre-existing Optional module-global (db_instance/global_config
set only in main()) debt, which [mypy-backend.main] already quarantines with that exact reason;
I added the parallel [mypy-backend.api.job_worker] entry rather than inject runtime asserts
(which would have changed behavior). One test monkeypatch was retargeted in a behavior-
equivalent way: my new seam test patches threading.Timer on the shared `threading` module object
itself (not m.threading) so the pin doesn't couple to which module hosts _arm_wake_timer ?
adjusted and re-confirmed green on the UNCHANGED code before the move. The full GPU/WSL/real-
video pipeline was not exercised (offline mocked suite only), but this change touches no GPU/CV
path. Did NOT merge (PR left open for review)."
      }
    ]
  }
}
</output>
```


327. assistant (2026-06-18T10:43:07.525Z)

The tech-debt fleet finished: **6 PRs opened, 1 correctly HELD**. The held one (`base-resolve-provider-dedup`) is exactly the rigor working ? the agent found the two `_resolve_provider` bodies *aren't* byte-identical (5 real differences) so a "pure lift" would silently change behavior; it stopped instead of forcing it. Good.

Let me independently verify CI + commit-cleanliness on the 6 + the docs PR, then scrutinize the riskiest (#234 job-worker: it touched `mypy.ini` and reworked the monkeypatch seam).

328. user (2026-06-18T10:43:25.361Z)

```
=== per-PR: commits-off-master + CI rollup + coverage-ish ===
PR #229 (docs/code-map-uploads): commits=1  CI=PASS
PR #230 (refactor/motion-process-video-tests-then-helpers): commits=1  CI=PASS
PR #231 (refactor/rally-seg-eval-build-preset): commits=1  CI=PASS
PR #232 (refactor/db-execute-write-helper): commits=1  CI=PASS
PR #233 (refactor/golden-fixtures-real-split): commits=1  CI=PASS
PR #234 (refactor/main-job-worker-module): commits=1  CI=PASS
PR #235 (refactor/eval-load-videos-shared): commits=1  CI=PENDING
```

329. assistant (2026-06-18T10:43:34.565Z)

6/7 CI PASS (incl. the complex #234), all 1 clean commit; #235 still running. Let me scrutinize

#234's `mypy.ini` change and the monkeypatch-seam (the deferral hazard) before trusting it.

330. user (2026-06-18T10:43:38.845Z)

```
=== #234 mypy.ini change (should be a quarantine entry mirroring backend.main) ===
diff --git a/mypy.ini b/mypy.ini
index 9698460..57e63fd 100644
--- a/mypy.ini
+++ b/mypy.ini
@@ -14,6 +14,12 @@ explicit_package_bases = True
 [mypy-backend.main]
 ignore_errors = True

+# Job-worker drain/wake loop extracted from backend.main (same Optional module-global
+# debt: resolves db_instance/global_config THROUGH backend.main at call-time). Shares
+# backend.main's quarantine until the deferred app-factory/lifespan refactor lands.
+[mypy-backend.api.job_worker]
+ignore_errors = True
+
 # Legacy demo segmenter (fabricated metadata; coverage-omitted too).
 [mypy-backend.pipeline.segmenters.motion]
 ignore_errors = True
=== #234 main.py re-export shim (how worker symbols stay addressable on backend.main) ===
+# now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
+# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
+# `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams THROUGH
+# intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
+from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable on
backend.main)
+    JobWaiting,
+    _arm_wake_timer,
+    _drain_due_jobs,
+    _get_executor,
+    _MAX_DRAIN_WAKE_SEC,
+    _schedule_next_drain_if_waiting,
=== job_worker.py: do cross-fn seam calls route through backend.main (so monkeypatch
intercepts)? ===
11:`backend.main` module object at call-time (`import backend.main as _main`), never as a
33:def _get_executor() -> ThreadPoolExecutor:
63:    import backend.main as _main
64:    return _main.ProcessRequest(
81:    import backend.main as _main
82:    db_instance = _main.db_instance
84:    req = _main._job_to_request(job)
86:        result = _main._execute_processing(
91:    except _main.JobWaiting as w:
100:def _drain_due_jobs() -> int:
109:    import backend.main as _main
112:        job = _main.db_instance.claim_due_job()
116:        _main._run_claimed_job(job)
120:    _main._schedule_next_drain_if_waiting()
128:    import backend.main as _main
130:        delay = _main.db_instance.seconds_until_next_due()
137:    _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))
140:def _arm_wake_timer(delay_sec: float) -> None:
144:    import backend.main as _main
145:    timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
```

331. assistant (2026-06-18T10:43:55.582Z)

#234 is correct: the `mypy.ini` change is a minimal quarantine entry mirroring the existing `backend.main` one (same already-excused Optional-global debt), and every cross-function call routes through `backend.main` (`_main._arm_wake_timer(...)`) so existing `monkeypatch.setattr(m, ...)` still intercepts ? the leaked-timer hazard is correctly avoided. Let me wait for #235's

CI, then run the combined-state gate before merging.

332. user (2026-06-18T10:44:30.902Z)

```
Syntax sweep + import smoke (no GPU stack)      pass    20s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27753949047/job/82111019403
Full test suite (offline, mocked)       pass    2m14s   https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27753949047/job/82111019447
Golden fixtures cross-OS (macos-latest) pass    18s     https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27753949047/job/82111019365
Golden fixtures cross-OS (ubuntu-latest)        pass    20s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27753949047/job/82111019357
Golden fixtures cross-OS (windows-latest)       pass    54s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27753949047/job/82111019370
Mypy (lenient baseline) pass    31s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27753949047/job/82111019412
Ruff lint       pass    12s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27753949047/job/82111019360
Syntax sweep + import smoke (no GPU stack)      pass    20s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27753949047/job/82111019403
=== #235 final ===
CLEAN
```

333. assistant (2026-06-18T10:44:38.728Z)

All 7 are CI-green. Now the binding combined-state gate ? merge all 7 together and run the full suite:

334. user (2026-06-18T10:44:44.543Z)

```
HEAD is now at e67add1 refactor(api): extract chunked-upload island from main into
backend.api.uploads router (#228)
Switched to a new branch 'combined-td-verify'
merged OK: docs/code-map-uploads
merged OK: refactor/motion-process-video-tests-then-helpers
merged OK: refactor/rally-seg-eval-build-preset
merged OK: refactor/db-execute-write-helper
merged OK: refactor/golden-fixtures-real-split
merged OK: refactor/main-job-worker-module
merged OK: refactor/eval-load-videos-shared
=== combined diffstat ===
 tests/test_rally_seg_eval_report.py   | 306 ++++++++++++++++++++++++++++++++++
 19 files changed, 1657 insertions(+), 650 deletions(-)
```

335. assistant (2026-06-18T10:44:47.997Z)

All 7 merged cleanly. Now the full gate on the combined state:

336. user (2026-06-18T10:46:51.341Z)

```
=== ruff ===
All checks passed!
=== mypy ===
Success: no issues found in 109 source files
=== full suite + coverage ===
Required test coverage of 70% reached. Total coverage: 82.68%
889 passed, 7 skipped, 7 warnings in 85.17s (0:01:25)
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

337. assistant (2026-06-18T10:46:58.146Z)

Combined state green ? ruff clean, mypy clean (109 files), **889 passed**, coverage

82.68% (UP from 80.94% base ? the characterization tests net-improved coverage). Aggregate doesn't regress. Merging all 7 (squash):

338. user (2026-06-18T10:47:31.473Z)

```
error: failed to delete 'C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/combined-td': Permission denied
Deleted branch combined-td-verify (was 9ab95b7).
=== merging all 7 (squash, delete branch) ===
PR #229 -> MERGED
PR #230 -> MERGED
PR #231 -> MERGED
PR #232 -> MERGED
PR #233 -> MERGED
PR #234 -> MERGED
PR #235 -> MERGED
```

339. assistant (2026-06-18T10:47:35.702Z)

All 7 MERGED. Let me confirm master and watch the final master-branch CI run.

340. user (2026-06-18T10:50:01.677Z)

```
   e67add1..5f555cf  master     -> origin/master
=== master now ===
5f555cf refactor(eval): extract one shared golden-feature loader behind the three load_videos
(#235)
5409fc4 refactor(api): extract job-worker drain/wake loop from main into backend.api.job_worker
(#234)
5c46ebc refactor(eval): split real-fixture subsystem into golden_real_fixtures (pure move)
(#233)
97fa187 refactor(db): route bool-returning single-statement writers through _execute_write
(#232)
ae10447 refactor(eval): extract _build_preset(args, side) from rally_seg_eval.main() (#231)
1e324ea refactor(motion): extract detection + DB-populate halves from process_video (#230)
53b9cef docs(code-map): point UPLOAD endpoints at backend/api/uploads.py (post #228) (#229)
e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router
(#228)
=== watching master CI run 27754238078 ===
  ? Golden-fixtures characterization (cross-OS determinism guard)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
master CI conclusion=success
```

341. assistant (2026-06-18T10:50:08.646Z)

Master CI: **success**. Let me prune leftover worktrees and record the checkpoint.

342. user (2026-06-18T10:50:15.514Z)

```
=== my open PRs (should be 0) ===
=== leftover sweep worktrees? ===
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-1
3057ada [refactor/motion-process-video-tests-then-helpers]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3
f7b8a78 [refactor/db-execute-write-helper]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-4
5445dbc [refactor/eval-load-videos-shared]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-5
727a421 [refactor/golden-fixtures-real-split]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-6
0a014ba [refactor/rally-seg-eval-build-preset]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-7
```

```
174cef6 [refactor/main-job-worker-module]
=== held branch still on origin? (base-resolve-provider had no PR) ===
0
```

343. assistant (2026-06-18T10:50:21.798Z)

Fleet worktrees persisted (held the merged branches). Let me prune them and their local branches.

344. user (2026-06-18T10:50:26.706Z)

```
Deleted branch refactor/main-job-worker-module (was 174cef6).
Deleted branch refactor/base-resolve-provider-dedup (was e67add1).
Deleted branch docs/code-map-uploads (was 9ed1f52).
=== remaining sweep worktrees? ===
(all clean)
```

345. user (2026-06-18T10:50:32.329Z)

```
10      # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
```

346. assistant (2026-06-18T10:50:31.219Z)

All clean. Let me record the checkpoint.

347. user (2026-06-18T10:50:54.136Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

348. assistant (2026-06-18T10:51:16.317Z)

Done ? the CODE_MAP fix and the deferred tech-debt sweep are merged to master (`5f555cf`), with the characterization-tests-first rigor throughout.

Merged (CODE_MAP + 6 refactors)

PR	What	Tests-first
[#229](https://github.com/avidullu/badminton-highlight-indexer/pull/229)	CODE_MAP: upload endpoints re-pointed to `backend/api/uploads.py`	docs-only
[#230](https://github.com/avidullu/badminton-highlight-indexer/pull/230)	motion: split `process_video` into detect + db-populate halves	new char tests mock cv2 + seed `random` to pin the fabricated segment/events (the untested 204-line path)
[#231](https://github.com/avidullu/badminton-highlight-indexer/pull/231)	rally_seg_eval: extract `_build_preset(args, side)`	11 char tests for the 0%-coverage report/main() paths ? module **0%?98%**
[#232](https://github.com/avidullu/badminton-highlight-indexer/pull/232)	database: route the 11 bool writers through `_execute_write`	6 failure-branch tests pin each **exact log message** ? byte-identical
[#233](https://github.com/avidullu/badminton-highlight-indexer/pull/233)	golden_fixtures: split real-fixture subsystem out (?296 lines)	lazy PEP-562 re-export (handles both import orders) + 7 char tests
[#234](https://github.com/avidullu/badminton-highlight-indexer/pull/234)	main.py: job-worker loop ? `backend/api/job_worker.py`	3 seam tests prove `monkeypatch` still intercepts (no leaked timer)
[#235](https://github.com/avidullu/badminton-highlight-indexer/pull/235)	eval: one shared `load_golden_videos` behind the 3 loaders	char test pins the 3-way audio-policy difference

How quality was protected

- **Tests-first discipline:** every refactor added characterization tests that pin today's behavior, proven to pass on the *unchanged* code before any refactor, then again after.

- **Independently verified:** each PR = 1 clean commit + green GitHub CI. I personally spot-checked the riskiest (#234: the `mypy.ini` quarantine entry mirrors the existing `backend.main` one, and the seam routes through `backend.main` so monkeypatches still intercept).
- **Combined-state gate:** merged all 7 together ? full suite **889 passed**, ruff + mypy (109 files) clean, **coverage 82.68% ? up from 80.94%** (the char tests net-*added* coverage). Master push-CI: **success**. Worktrees/branches cleaned.

1 held for your call

`base-resolve-provider` dedup ? the agent found gemini's and yolo_hybrid's `_resolve_provider` are **not** identical (5 real differences: signature, `sport_profile` capture-vs-discard, `model_name` source, an extra log line, return shape). Folding them to a base method would **change behavior**, not move code ? so it correctly stopped. Doable only as a deliberate redesign (with char tests around both segmenters' early-returns + the relocated log line) **if you want it**.

4 reported as not-a-pure-refactor (not forced)

- **trajectory_hybrid** ? already decomposed (hook-based base class); nothing to do.
- **person_detector ? tracknet grouping dedup** ? different tuple shapes ? a redesign with real regression surface, not a move.
- **`wasb_infer.run()`** ? GPU/WSL-only; needs an owner-side torch-mocked test + a real run to refactor safely.
- **main.py routes-vs-CLI split** ? high blast radius; needs cv2-backed tests first.

So: **7 of the 11** are resolved or correctly closed-out (3 of those 11 were "don't-do" by design), 1 awaits your decision, and the 3 GPU/redesign ones I've scoped for when you want them. Want me to take on the held dedup as an explicit redesign, or set up the characterization tests for the `wasb_infer.run()` / main-routes splits next?